UNIVERSITÉ LIBRE DE BRUXELLES
FACULTÉ DES SCIENCES APPLIQUÉES

Ant Colony Optimization and its Application to Adaptive Routing in Telecommunication Networks

Gianni Di Caro

Dissertation présentée en vue de l'obtention du grade de
Docteur en Sciences Appliquées

Bruxelles, September 2004

PROMOTEUR: PROF. MARCO DORIGO
IRIDIA, Institut de Recherches Interdisciplinaires
et de Développements en Intelligence Artificielle

ANNÉE ACADÉMIQUE 2003-2004

Abstract

In *ant societies*, and, more in general, in insect societies, the activities of the individuals, as well as of the society as a whole, are not regulated by any explicit form of centralized control. On the other hand, adaptive and robust behaviors transcending the behavioral repertoire of the single individual can be easily observed at society level. These complex global behaviors are the result of self-organizing dynamics driven by local interactions and communications among a number of relatively simple individuals. The simultaneous presence of these and other fascinating and unique characteristics have made ant societies an attractive and inspiring model for building new *algorithms* and new *multi-agent systems*. In the last decade, ant societies have been taken as a reference for an ever growing body of scientific work, mostly in the fields of robotics, operations research, and telecommunications.

Among the different works inspired by ant colonies, the *Ant Colony Optimization metaheuristic* (ACO) is probably the most successful and popular one. The ACO metaheuristic is a multi-agent framework for combinatorial optimization whose main components are: a set of *ant-like agents*, the use of *memory* and of *stochastic decisions*, and strategies of *collective* and *distributed learning*. It finds its roots in the experimental observation of a specific foraging behavior of some ant colonies that, under appropriate conditions, are able to select the *shortest path* among few possible paths connecting their nest to a food site. The *pheromone*, a volatile chemical substance laid on the ground by the ants while walking and affecting in turn their moving decisions according to its local intensity, is the mediator of this behavior. All the elements playing an essential role in the ant colony foraging behavior were understood, thoroughly reverse-engineered and put to work to solve problems of combinatorial optimization by Marco Dorigo and his co-workers at the beginning of the 1990's. From that moment on it has been a flourishing of new combinatorial optimization algorithms designed after the first algorithms of Dorigo's *et al.*, and of related scientific events. In 1999 the ACO metaheuristic was defined by Dorigo, Di Caro and Gambardella with the purpose of providing a common framework for describing and analyzing all these algorithms inspired by the same ant colony behavior and by the same common process of reverse-engineering of this behavior. Therefore, the ACO metaheuristic was defined *a posteriori*, as the result of a synthesis effort effectuated on the study of the characteristics of all these ant-inspired algorithms and on the abstraction of their common traits. The ACO's synthesis was also motivated by the usually good performance shown by the algorithms (e.g., for several important combinatorial problems like the quadratic assignment, vehicle routing and job shop scheduling, ACO implementations have outperformed state-of-the-art algorithms).

The definition and study of the ACO metaheuristic is one of the two fundamental goals of the thesis. The other one, strictly related to this former one, consists in the design, implementation, and testing of ACO instances for problems of *adaptive routing in telecommunication networks*.

This thesis is an in-depth journey through the ACO metaheuristic, during which we have (re)defined ACO and tried to get a clear understanding of its potentialities, limits, and relationships with other frameworks and with its biological background. The thesis takes into account all the developments that have followed the original 1999's definition, and provides a formal and comprehensive systematization of the subject, as well as an up-to-date and quite comprehensive review of current applications. We have also identified in dynamic problems in telecommuni-

cation networks the most appropriate domain of application for the ACO ideas. According to this understanding, in the most applicative part of the thesis we have focused on problems of adaptive routing in networks and we have developed and tested four new algorithms.

Adopting an original point of view with respect to the way ACO was firstly defined (but maintaining full conceptual and terminological consistency), ACO is here defined and mainly discussed in the terms of *sequential decision processes* and *Monte Carlo sampling* and *learning*. More precisely, ACO is characterized as a policy search strategy aimed at learning the distributed parameters (called *pheromone variables* in accordance with the biological metaphor) of the stochastic decision policy which is used by so-called *ant* agents to generate solutions. Each ant represents in practice an independent *sequential decision process* aimed at *constructing* a possibly feasible solution for the optimization problem at hand by using only information *local* to the decision step. Ants are *repeatedly* and *concurrently* generated in order to sample the solution set according to the current policy. The outcomes of the generated solutions are used to *partially evaluate* the current policy, *spot* the most promising search areas, and *update the policy parameters* in order to possibly focus the search in those promising areas while keeping a satisfactory level of overall *exploration*.

This way of looking at ACO has facilitated to disclose the strict relationships between ACO and other well-known frameworks, like *dynamic programming*, *Markov* and *non-Markov decision processes*, and *reinforcement learning*. In turn, this has favored reasoning on the general properties of ACO in terms of amount of complete *state information* which is used by the ACO's ants to take optimized decisions and to encode in pheromone variables memory of both the decisions that belonged to the sampled solutions and their quality.

The ACO's biological context of inspiration is fully acknowledged in the thesis. We report with extensive discussions on the shortest path behaviors of ant colonies and on the identification and analysis of the few nonlinear dynamics that are at the very core of self-organized behaviors in both the ants and other societal organizations. We discuss these dynamics in the general framework of *stigmergic modeling*, based on asynchronous environment-mediated communication protocols, and (pheromone) variables priming coordinated responses of a number of "cheap" and concurrent agents.

The second half of the thesis is devoted to the study of the application of ACO to problems of *online routing in telecommunication networks*. This class of problems has been identified in the thesis as the most appropriate for the application of the multi-agent, distributed, and adaptive nature of the ACO architecture. Four novel ACO algorithms for problems of adaptive routing in telecommunication networks are throughly described. The four algorithms cover a wide spectrum of possible types of network: two of them deliver *best-effort traffic in wired IP networks*, one is intended for *quality-of-service (QoS) traffic in ATM networks*, and the fourth is for *best-effort traffic in mobile ad hoc networks*. The two algorithms for wired IP networks have been extensively tested by simulation studies and compared to state-of-the-art algorithms for a wide set of reference scenarios. The algorithm for mobile ad hoc networks is still under development, but quite extensive results and comparisons with a popular state-of-the-art algorithm are reported. No results are reported for the algorithm for QoS, which has not been fully tested. The observed experimental performance is excellent, especially for the case of wired IP networks: our algorithms always perform comparably or much better than the state-of-the-art competitors. In the thesis we try to understand the rationale behind the brilliant performance obtained and the good level of popularity reached by our algorithms. More in general, we discuss the reasons of the general efficacy of the ACO approach for network routing problems compared to the characteristics of more classical approaches. Moving further, we also informally define *Ant Colony Routing* (ACR), a multi-agent framework explicitly integrating learning components into the ACO's design in order to define a general and in a sense futuristic architecture for autonomic network control.

Most of the material of the thesis comes from a re-elaboration of material co-authored and published in a number of books, journal papers, conference proceedings, and technical reports. The detailed list of references is provided in the Introduction.

Contents

CHAPTER 7

ACO algorithms for adaptive routing

This chapter introduces four novel algorithms for routing in telecommunication networks (*AntNet, AntNet-FA, AntNet+SELA*, and *AntHocNet*), a general framework for the design of routing algorithms (*Ant Colony Routing*), and reviews related work on ant-inspired routing algorithms.

AntNet [119, 125, 121, 122, 118, 115] and AntNet-FA [124] are two ACO algorithms for adaptive best-effort routing in wired datagram networks (extensive experimental results for these two algorithms are presented in the next chapter). On the other hand, Ant Colony Routing (ACR) is a general framework of reference for the design of *autonomic routing systems* [250].[1] ACR defines the generalities of a multi-agent society based on the integration of the ACO's philosophy with ideas from the domain of reinforcement learning, with the aim of providing a meta-architecture of reference for the design and implementation of fully adaptive and distributed routing systems for a wide range of network scenarios (e.g., wired and wireless, best-effort and QoS, static and mobile). In the same way ACO has been defined as an ant-inspired meta-heuristic for generic combinatorial problems, ACR can be seen as the equivalent meta-architecture for network control problems based on the use of ant-like and learning agents. In the following ACR will be also referred to as the *ant-based* network control/routing framework. Both AntNet and AntNet-FA can be seen as specific instances of ACR for the case of best-effort routing in wired datagram networks.In order to show how the general ACR's ideas can find their application, as well as in order to introduce a new routing algorithm for each one of the most important and popular network scenarios, we briefly describe two additional routing algorithms, *AntNet+SELA* [130] and *AntHocNet* [126, 154, 127]. AntNet+SELA is a model to deliver QoS routing in ATM networks, while AntHocNet is intended for routing in mobile ad hoc networks.[2]

The author's work on ACO algorithms for routing tasks dates back to 1997, when he developed the first versions of AntNet [116, 117, 115, 114]. AntNet was specifically designed to address the problem of adaptive best-effort routing in wired datagram networks (e.g., Internet). Since then, AntNet's design has evolved, and improved/revised versions of it have been developed by the author [119, 125, 121, 122, 118, 120] and also by several other researchers from all over the world (these additional contributions are discussed in Section 7.4). In particular, AntNet-FA [124, 113] has brought some major improvements into the AntNet design and made it also suitable for a possible use in connection-oriented and QoS networks. Some models for fair-share and generic QoS networks [123] have been also derived from the basic AntNet, but are not going to be described in this thesis since similar ideas have contributed to the design of AntNet+SELA [130], intended for QoS routing in ATM networks. In AntNet+SELA ACO's ant-like agents are complemented by the presence of *node agents* each implementing a stochastic learning automata exploiting the information gathered by the ants to adaptively learn an ef-

[1] More in general, ACR can be considered as a framework for distributed control tasks in telecommunication networks (e.g., monitoring, admission control, maintenance, load balancing). However, in the following the focus will be almost exclusively on routing tasks. For instance, a discussion on how ACO algorithms can be applied to perform active monitoring can be found in [129].

[2] Hereafter, for notation convenience, we will also refer to the set of ACO routing algorithms that we are going to describe, that is, AntNet, AntNet-FA, AntNet+SELA, and AntHocNet, in terms of *ACR algorithms*, since they can all be seen as instances of this more general ant-based framework.

fective routing policy for QoS data. The definition of ACR as a sort of general framework of reference for ant-based routing and control systems is the result of a process of abstraction and generalization from all these ACO algorithms developed during the years, as well as from the number of other ant-based algorithms that have appeared in the literature in the same time, and from results and ideas from the field of reinforcement learning. ACR finds its roots in the work on AntNet++, which was introduced in the author's DEA thesis [113]. ACR and AntNet++ share the basic architecture and several other ideas. However, we choose to use here a different name since AntNet++ gives more the idea of an evolution over AntNet, rather than the definition of a general framework that finds its roots in ACO. The work on ACR has to be considered as still preliminary. More systematic and formal definitions are necessary. The author's most recent ongoing work on routing is that on AntHocNet [126, 128, 154, 127], which is an application to the case of mobile ad hoc networks. This work is co-authored with Frederick Ducatelle and Luca Maria Gambardella, as explained in Footnote 4.

As already discussed in the Summary section of Chapter 5, the application of the ACO framework to routing tasks in telecommunication networks is rather natural and straightforward due to the isomorphism between the pheromone graph and stigmergic architecture of ACO on one side, and the structure and constraints of telecommunication networks on the other side. An isomorphism that makes rather natural to map ACO's components onto a telecommunication network in the following way:

REMARK 7.1 (ONE-TO-ONE RELATIONSHIP BETWEEN ACO'S COMPONENTS AND ROUTING PROBLEMS): *Ants are mobile agents that migrate from one node to an adjacent one searching for feasible paths between source and destination nodes. ACO's solution components (and phantasmata) correspond to network nodes, and, accordingly, routing tables correspond to pheromone tables $\mathcal{T}^k$ in which each pheromone variable τ_{nd}^k holds the estimated goodness of selecting k's neighbor n to forward a data packet toward d.*

The immediate relationship between ACO and network routing is likely one of the main reasons behind the popularity of the application of ACO to routing problems (see Section 7.4), as well as behind the usually good performance and the strong similarities showed by the different implementations. In particular, the adopted *pheromone model* is in practice the same for all the implementations: a pheromone variable is always associated to a pair of nodes, which are the "natural" solution components for routing problems. Nevertheless, important differences also exist among the algorithms, in particular concerning the heuristic variables, the way the paths sampled by the ants are evaluated and reinforced, the modalities for the generation of the ants, and so on. The relationship between ACO (as well as its biological context of inspiration) and networks is particularly evident for the case of *datagram protocols*. In fact, in this case each node builds and holds its own routing table and an independent routing decision is taken for each single data packet on the sole basis of the contents of the local routing table. On the other hand, in a virtual-circuit model all the packets of the same session are routed along the same path and no independent per packet decisions are issued at the nodes. The direct relationship between ACO and datagram models is likely one of the main reasons behind the fact that most of the works on ant-based routing have focused so far on best-effort datagram networks, in spite of the fact that the first work in this domain by Schoonderwoerd et al. [382] was an application to circuit-switched networks. Nevertheless, the application of the ACO ideas to both connection-oriented and QoS networks is in a sense equally straightforward, as it will be also shown by the description of AntNet+SELA.

In the Summary of the previous chapter a sort of "wish list" for the characteristics of novel routing algorithms was compiled. The characteristics of the routing algorithms that are introduced in the following of the chapter well match the characteristics indicated in the wish list.

REMARK 7.2 (GENERAL CHARACTERISTICS OF ACO ALGORITHMS FOR ROUTING): *The following set of core properties characterizes ACO instances for routing problems:*

- *provide traffic-adaptive and multipath routing,*
- *rely on both passive and active information monitoring and gathering,*
- *make use of stochastic components,*
- *do not allow local estimates to have global impact,*
- *set up paths in a less selfish way than in pure shortest path schemes favoring load balancing,*
- *show limited sensitivity to parameter settings.*

These are all characteristics that directly result from the application of the ACO's design guidelines, and in particular from the use of *controlled random experiments* (the ants) that are repeatedly generated in order to *actively* gather useful non-local information about the characteristics of the solution set (i.e., the set of paths connecting all pairs of source-destination nodes, in the routing case). In turn, this information is used to set and continually update the decision (routing) policies at the decision nodes in the form of pheromone tables. All the other properties derive in some sense from this basic behavior. *Traffic-adaptiveness*, as well as *automatic load balancing*, come from the generalized policy iteration structure of ACO, which is expected to repeatedly refine and/or adapt the current decision policy to new sampled experiences and, therefore, to the changing traffic patterns. The use of stochastic components is a fundamental aspect of ACO, as well as the notion of *locality of the information* and the related use of Monte Carlo (i.e., non-bootstrapping) updating. The availability of *multiple paths* derives from two of the most fundamental components of ACO, that is, the pheromone tables, which old estimates for the goodness of each feasible local decision, and the use of a stochastic decision policy. In fact, pheromone variables virtually assign a score to each possible path in the network, while the use of a stochastic routing policy not only for the ants but also for data packets allows to concurrently spread data along those paths that are currently estimated to be the best ones. That is, a *bundle of paths*, each one with an associated value of goodness, are made available for each source-destination pair and can be used for either multipath or alternate path routing. The relative stability of performance with respect to a wide range of parameter settings derives from the locality of the estimates, such that "wrong" settings do not have global impact, as well as from the fact that in ACO algorithms several different components contribute to the overall performance such that there is not a single component which is extremely critical and that has to be finely and carefully tuned.

Organization of the chapter

The chapter is organized in four main sections. The first three describe respectively AntNet, AntNet-FA, and ACR (and also AntNet+SELA and AntHocNet, that are seen as examples of ACR), the fourth is devoted to the discussion of related work on ant-inspired routing algorithms.

The introductory part of Section 7.1 discusses the generalities of AntNet and reports a compact and informal description of the overall algorithm behavior. The characteristics of the model of communication network that is assumed for both AntNet and AntNet-FA (and which is used for the experiments reported in the next chapter), are discussed in Subsection 7.1.1, as a preliminary step before providing a detailed description of the algorithm. Subsection 7.1.2 describes the data structures (e.g., routing and pheromone tables) that are maintained at the nodes. AntNet

is finally described in detail in Subsection 7.1.3. This subsection is organized in several sub-subsections, each describing a single component of the algorithm. A pseudo-code description of the full behavior of an ant agent is provided in Subsection 7.1.3.8. Subsection 7.1.4 discusses the central and thorny issue of how to properly evaluate an ant path, and shows the difference between using constant and adaptive evaluations.

Section 7.2 describes AntNet-FA, which is an improvement over AntNet, while Section 7.3 describes ACR, which is a preliminary characterization of a general multi-agent framework derived from ACO for the design of fully adaptive and distributed network control systems. The architecture of ACR, based on the use of both learning agents as node managers and mobile ant-like agents, is discussed in Subsection 7.3.1 and its subsections. In Subsection 7.3.2 two other novel routing algorithms are briefly described. They are intended to be examples of the general ACR design concepts. AntNet+SELA, an algorithm for QoS routing in ATM networks is described in Subsection 7.3.2.1, while Subsection 7.3.2.2 reports the description of AntHocNet, an algorithm for routing in mobile ad hoc networks.

The chapter is concluded by Section 7.4, which reviews related work in the domain of routing algorithms inspired by ant behaviors.

7.1 AntNet: traffic-adaptive multipath routing for best-effort IP networks

AntNet is an ACO algorithm for distributed and traffic-adaptive multipath routing in wired best-effort IP networks. AntNet's design is based on ACO's general ideas as well as on the work of Schoonderwoerd et al. [382, 381], which was a first application of algorithms inspired by the foraging behavior of ant colonies to routing tasks (in telephone networks). AntNet behavior is based on the use of mobile agents, the ACO's ants, that realize a pheromone-driven Monte Carlo sampling and updating of the paths connecting sources and destination nodes.[3]

Informally, the behavior of AntNet can be summarized as follows (a detailed description and discussion of all AntNet's components is provided in the subsections that follow).

- From each network node s mobile agents are launched towards specific destination nodes d at regular intervals and concurrently with the data traffic. The agent generation processes happen concurrently and without any form of synchronization among the nodes. These agents moving from their source to destination nodes are called *forward ants* and are indicated with $F^i_{s \to d}$ where i is the ant identifier.

- Each forward ant is a *random experiment* aimed at collecting and gathering at the nodes non-local information about paths and traffic patterns. Forward ants *simulate data packets* moving hop-by-hop towards their destination. They make use of the same priority queues used by data packets. The characteristics of each experiment can be tuned by assigning different values to the agent's parameters (e.g., the destination node).

[3] In ACO the notion of agent is more an abstraction than a practical issue. But in the general ACR case ants are "true" mobile agents. On the other hand, mobile agents are expected to carry their own code an execute it at the nodes. However, these are genuine implementation issues. AntNet's ants can be precisely designed in such a way: they could read the routing information at the nodes, make their own calculations, and communicate the results to the routing component of the node that would in turn decide to accept or not the proposed modifications to the routing and pheromone tables. However, in the following, even if we will keep using the term "mobile agents", we will more simply assume that our ants are routing control packets managed at the network layer and their contents are used to update routing tables and related information.

- Ants, once generated, are fully autonomous agents. They act concurrently, independently and asynchronously. They communicate in an indirect, stigmergic way, through the information they locally read from and write to the nodes.
- The specific task of each forward ant is to search for a *minimum delay path* connecting its source and destination nodes.
- The forward ant migrates from a node to an adjacent one towards its destination. At each intermediate node, a *stochastic decision policy* is applied to select the next node to move to. The parameters of the local policy are: (i) the local pheromone variables, (ii) the status of the local link queues (playing the role of heuristic variables), and (iii) the information carried into the ant memory (to avoid cycles). The decision is the results of some tradeoff among all these components.
- While moving, the forward ant *collects information* about the traveling time and the node identifiers along the followed path.
- Once arrived at destination, the forward ant becomes a *backward ant* $B^i_{d\to s}$ and goes back to its source node by moving along the same path $\mathcal{P}^i_{s\to d} = [s, v_1, v_2, \ldots, d]$ as before but in the opposite direction. For its return trip the ant makes use of queues of priority higher than those used by data packets, in order to quickly retrace the path.
- At each visited node $v_k \in \mathcal{P}_{s\to d}$ and arriving from neighbor $v_j,\ v_j \in \mathcal{N}(v_k) \cap \mathcal{P}^i_{s\to d}$ the backward ant updates the local routing information related to each node v_d in the path $\mathcal{P}^i_{v_k\to d}$ followed by the forward ant from v_k to d, and related to the choice of v_j as next hop to reach each v_d. In particular, the following data structures are updated: a *statistical model* $\mathcal{M}^{v_k}$ of the expected end-to-end delays, the *pheromone table* $\mathcal{T}^{v_k}$ used by the ants, and the *data routing table* $\mathcal{R}^{v_k}$ used to route data packets. Both the pheromone and the routing tables are updated on the basis of the *evaluation* of the goodness of the path that was followed by the forward ant from that node toward the destination. The evaluation is done by comparing the experienced traveling time with the expected traveling time estimated according to the local delay model.
- Once they have returned to their source node, the agent is *removed* from the network.
- Data packets are routed according to a stochastic decision policy based on the information contained in the *data-routing tables*. These tables are derived from the pheromone tables used to route the ants: only the best next hops are in practice retained in the data-routing tables. In this way data traffic is concurrently spread over the best available *multiple paths*, possibly obtaining an optimized utilization of network resources and *load balancing*.

The AntNet's general structure is quite simple and closely follows the ACO's guidelines. During the forward phase each mobile ant-like agent *constructs* a path by taking a sequence of decisions based on a *stochastic policy* parametrized by local *pheromone* and *heuristic* information (the length of the local link queues). Once arrived at destination, the backward phase starts. The ant retraces the path and at each node it *evaluates* the followed path with respect to the destination (and to all the intermediate nodes) and *updates* the local routing information. Due to the practical implementation of both the forward and backward phases envisaged by ACO, the AntNet model is also often referred to as the *Forward-Backward model*. For instance, in the model adopted by Schoonderwoerd et al. [382] for cost-symmetric networks (the end-to-end delay along a path connecting two nodes is the same in both directions) only the forward phase is present. The need for a backward phase comes from the need of completing and evaluating the path before carrying out any update. This is the case of cost-asymmetric networks. Moreover,

in both cost-symmetric and cost-asymmetric networks, until the destination is reached there is no guarantee that the agent is not being actually catch in a loop, such that carrying out updates during the forward phase might result in reinforcing a loop, which is clearly wrong.

The AntNet's ant-like agents behave similarly to data packets. However, an important conceptual difference exists:

REMARK 7.3 (ANTS AND EXPLORATION): *Ants simulate data packets with the aim of performing controlled network exploration (i.e., discovering and testing of paths). Ants do not belong to user applications, therefore they can freely explore the network. No user will complain if an ant gets lost or follows a long-latency path. On the other hand, users would possibly get disappointed if their packets would incur in long delays or get lost for the sake of general exploration and/or information gathering.*

In this sense, AntNet and, more in general, all ACR algorithms, constitute a radical departure from previous approaches to adaptive routing, in which exploration is either absent or marginal. Classical approaches are mainly based on the somehow *passive* observation of data traffic: nodes observe local data flows, build local cost estimates on the basis of these observations, and propagate the estimates to other nodes. With this strategy path exploration becomes problematic, since it has to be carried out directly using users' data packets. On the other hand:

REMARK 7.4 (PASSIVE AND ACTIVE INFORMATION GATHERING IN ANT-BASED ROUTING ALGORITHMS): *ACR algorithms complement the passive observation of the local traffic streams with an* active *exploratory component based on the ACO's core idea of repeated Monte Carlo simulation by online generated ant-like agents. Routing tables are built and maintained on the basis of the observation of the behavior of both data packets generated by traffic sources and routing agents generated by the control system itself.*

The ants explore the network making use of their own ant-routing tables, while data packets are routed making use of data-routing tables derived from the ant-routing tables such that only the best paths discovered so far will be followed by data. In this way, *path exploration* and *path exploitation* policies are conveniently kept separate. In the jargon of reinforcement learning, this is termed *off-policy* control [414, Chapter 5]: the policy used to generate samples is different from the target one which has to be evaluated and possibly improved.

It is interesting to notice that the very possibility of executing *realistic simulations* concurrently with the normal activities of the system is a sort of unique property of telecommunication networks. While the usefulness of simulation for learning tasks is well understood (e.g., [415, 27]), building a faithful simulator (based on either a mechanistic or phenomenological model) of the system under study is usually a quite complex/expensive task. On the other hand, in the case of telecommunication networks, the network itself can be used as a fully realistic online simulator. With simulation packets running concurrently with data packets at a cost (i.e., the generated overhead) under control and usually negligible with respect to the produced benefits.

In spite of the fact that the AntNet's general architecture is rather simple, the design of each single component (e.g., evaluation of paths, use of heuristic information, pheromone updating, etc.) had to be carefully engineered in order to obtain an algorithm which is not just a proof-of-concept but rather an algorithm able to provide performance comparable or better than that of state-of-the-art algorithms under realistic assumptions and for a wide set of scenarios. The subsections that follow discuss one-by-one and in detail all the components of the algorithm.

7.1.1 The communication network model

Before diving into the detailed description of AntNet (and AntNet-FA), is customary to discuss the characteristics of the model of IP wired networks that has been adopted. The IP datagram model is rather general and scalable (and these are two of the major reasons behind its popu-

larity), but does not tell anything about how packets are, for instance, queued and transmitted, while at the same time it comes with a number of different possible choices at the transport layer (several TCP implementations exist) and in terms of classes of service and forwarding. More in general, since we are not going to focus on a specific type of network and our experiments are only based on simulation, as is common practice to evaluate telecommunication protocols (e.g., [325]), is necessary to first make explicit the characteristics of the adopted network and, accordingly, of the simulation model.

At the network layer we have adopted an IP-like datagram model, while we make use of a very simple, UDP-like, protocol at the transport layer. The terms "IP-like" and "UDP-like" stands for the fact that we did not implement full protocols (i.e., that could be used in real networks) but rather oversimplified models that, however, conserve the core characteristics of real implementations in terms of dynamics of packet processing and forwarding.

The assumed network topology is in general irregular, and both connection-less and connection-oriented forwarding schemes are in principle admitted, even if the basic IP model considers only the connection-less one.[4] In the following the discussion will mainly focus on the connection-less case. We see the connection-oriented case as a quite straightforward modification of it, since, generally speaking, it can be realized by keeping per-application, per flow, or per-destination state information at the nodes, or by using source routing.

The segment of considered networks is that of *wide-area networks* (WAN), which usually have point-to-point links. In these cases, *hierarchical* organization schemes are adopted. The instance of the communication network is mapped on a directed weighted graph with N processing/forwarding nodes. All the links are viewed as *bit pipes* characterized by a *bandwidth* (bit/sec) and a end-to-end *propagation delay* (sec), and are accessed following a *statistical multiplexing* scheme. For this purpose, every node, which is of type *store-and-forward* (e.g., this is not the case of ATM networks), holds a buffer space where the incoming and the outgoing packets are stored. This buffer is a shared resource among all the queues associated to every ongoing and outgoing link attached to the node. Traveling packets are subdivided in two classes: *data* and *routing* packets. All the packets in the same class have the same priority, so they are queued and served on the basis of a *first-in-first-out* policy, but routing packets have a *greater priority* than data packets. The workload is defined in terms of *applications* whose *arrival rate* at each node is dictated by a selected probabilistic model. By application (or traffic session/connection, in the following), we mean a stochastic process sending data packets from an origin node to a destination node. The number of packets to send, their sizes and the intervals between them are assigned according to some defined stochastic process. No distinction is made among nodes, in the sense that they act at the same time as *hosts* (session end-points) and *gateways/routers* (forwarding elements). The workload model incorporates a simple *flow control mechanism* implemented by using a *fixed production window* for the session's packets generation. The window determines the maximum number of data packets waiting to be sent. Once sent, a packet is considered to be acknowledged. This means that the transport layer neither manages error control, nor packet sequencing, nor acknowledgments and retransmissions.[5]

For each incoming packet, the node routing layer make use of the information stored in the local routing table to assign the output link to be used to forward the packet toward its destination. When the link resources are available, they are reserved and the transfer is set up. The time it takes to move a packet from one node to a neighbor one depends on the packet size and on the transmission characteristics of the link. If, on a packet's arrival, there is not enough

[4] Resources reservation schemes, that in principle would require connection-oriented architectures, can be possibly managed in the Internet by using the proposed *IntServ* [58, 450], which is aimed at providing deterministic service guarantees in the Internet connection-less architecture. IntServ is based on the idea of using *per flow* reservations using the soft-state protocol *RSVP* [450].

[5] This choice is the same as in the *Simple_Traffic* model in the MaRS network simulator [5]. It can be seen as a very basic form of File Transfer Protocol (FTP) [93].

buffer space to hold it, the packet is *discarded*. Otherwise, a *service time* is stochastically generated for the newly arrived packet. This time represents the delay between the packet arrival time and the time when it will be put in the buffer queue of the outgoing link that the local routing component has selected for it.

A packet-level *network simulator* according to the above characteristics has been developed in C++. It is a *discrete event simulator* using as its main data structure a priority queue dynamically holding the time-ordered list of future events. The simulation time is a continuous variable and is set by the currently scheduled event. The design characteristic of the simulator is to closely mirror the essential features of the concurrent and distributed behavior of a generic queue network without sacrificing run-time efficiency and flexibility in code development.[6]

What is *not* in the model

Situations causing a temporary or steady *alteration of the network topology* or of its physical characteristics are not taken into account (e.g., link or node failure, addition or removal of network components). In fact, these are low probability events in real networks (which otherwise would be quite unstable), whose proper management requires, at the same time, to include in the routing protocol quite specific components of considerable complexity. We will briefly discuss how our algorithms can in a sense already deal with the problem, though not in a way which is expected to be efficient. An efficient solution has been devised in AntHocNet for the case of mobile ad hoc networks, in which topological dynamics is the norm, not the exception. While this same solution could be applied to deal with online topological modifications also in the considered wired IP networks, it will not explicitly considered in this thesis and the whole issue of topological modifications is actually bypassed.

As it is clear from the model description, the implemented *transport layer*, that is, the management of error, flow, and congestion control, is quite simple. This choice has been motivated by two main concerns. First, as pointed out at Page 180, when multipath routing is used, as is the case of our algorithms, some problems can arise with packet reordering. Such that the only reasonable choice is to accordingly re-design the transport protocol in order to effectively deal in practice with this thorny issue. Second, is a fact that each additional control component (other than routing) has a considerable impact on the network performance such that it might result quite difficult to evaluate and study the properties of each implemented control component without taking into account the complex way it interacts with all the other control components (and possibly mutually adapting the different components). The layered architecture of networks is an extremely good design feature from a software engineers point of view, it allows to independently modify the algorithms used at each layer, but at the same time nothing can be said about the global network dynamics resulting from intra- and inter-layers interactions.[7] Therefore, our choice has been to test the behavior of AntNet and AntNet-FA in conditions such that the number of interacting components is minimal, as well as the complexity of components other than routing. In this way the routing component can be in a sense evaluated in isolation. This way of proceeding is expected to allow a better understanding of the properties of the proposed algorithms.

[6] We are well aware of how critical is the choice of both the network model and of its practical implementation in a simulation software (see also the co-authored report [325] on general simulation issues and on simulation/simulators for mobile ad hoc networks in particular). However, realistic and arbitrarily complex situations for traffic patterns can hardly be studied by analytical models. Therefore, simulation is an inescapable choice to carry out extensive analysis and performance studies.

[7] For instance, some works [102] reported an improvement ranging from 2 to 30% in various measures of performance for real Internet traffic changing from the Reno version to the Vegas version of the TCP [351]. Other authors even claimed improvements ranging from 40 to 70% [58].

7.1.2 Data structures maintained at the nodes

In any routing algorithm, the final quality of the routing policy critically depends on the characteristics of the information maintained at the network nodes. Figure 7.1 graphically summarizes the data structures used by AntNet at each node k, that are as follows:

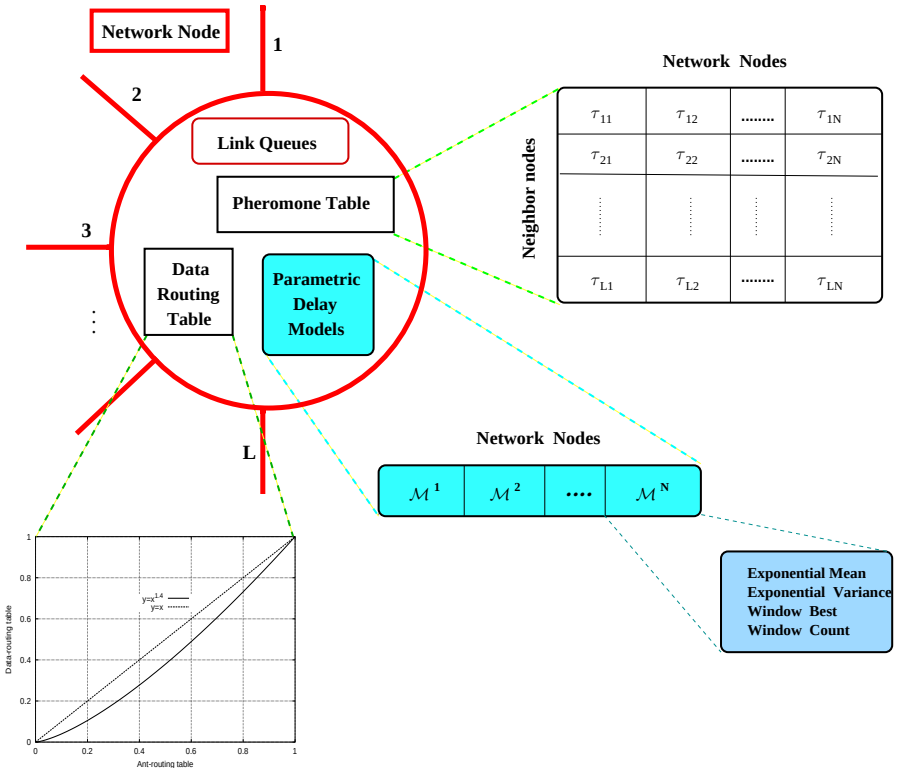

Figure 7.1: *Node data structures used the ant agents in AntNet for the case of a node with L neighbors and a network with N nodes. For simplicity the identifiers of the neighbors are supposed to be $1, 2, \ldots L$. Both the ant-routing and data-routing tables are organized as in distance-vector algorithms, but the entries are not distances but probabilities indicating the goodness of each next hop choice. The data-routing table is obtained by the ant-routing table by means of an exponential transformation as the one showed in the graph in the lower-left part of the figure. The entries of the vector of delay models are data structures representing parametric models for the expected traveling times to reach each possible destination from the current node. Also the current status of the link queues (in terms of bits waiting to be transmitted) is used by AntNet, and it is represented in the upper part of the node diagram.*

Pheromone matrix $\mathcal{T}^k$: is organized similarly to the routing tables in distance-vector algorithms, but its entries τ_{nd} are not distances or generic costs-to-go. The entries, in agreement with the common meaning attributed to pheromone variables in ACO, are a measure, for each one of the physically connected neighbor nodes $n \in \mathcal{N}_k$, of the goodness of forwarding to such a neighbor a packet traveling toward destination d. The τ_{ij} values are in the interval [0,1] and sum up to 1 along each destination column:

$$\sum_{n \in \mathcal{N}_k} \tau_{nd} = 1, \quad d \in [1, N], \quad \mathcal{N}_k = \{neighbors(k)\}. \tag{7.1}$$

Accordingly, the entries of the pheromone table can be seen as the *probabilities* of selecting one specific outgoing link for a specific final destination according to what has been learned so far through the ant agents.[8] $\mathcal{T}^k$ are parameters of the stochastic routing policy

[8] The matrix $\mathcal{T}$ is actually what is a generically called a *stochastic matrix*. If the nodes are seen as the states of a Markov decision process, the pheromone stochastic matrix precisely coincides with the process' transition matrix.

currently adopted at node k by the ant agents. The $\mathcal{T}^k$'s entries are the learning target of the AntNet algorithm. The idea here, as in all ACO algorithms, is to learn an effective *local* decision policy by the continual updating of the pheromone values in order to obtain an effective *global* routing policy. The pheromone table, in conjunction with information related to the status of the local link queues, is used by the ants to route themselves. In turn, it is used to set the values of the *data-routing table*, used exclusively by data packets.

Data-routing table $\mathcal{R}^k$: is the routing table used to forward *data* packets. $\mathcal{R}^k$ is a stochastic matrix and has the same structure as $\mathcal{T}^k$. The entries of $\mathcal{R}^k$ are obtained by an exponential transformation and re-normalization to 1 of the corresponding entries of $\mathcal{T}^k$. Data packets are probabilistically spread over the neighbors according to a stochastic policy which depends on the values of the stochastic matrix $\mathcal{R}^k$. The exponential transformation of the values of the pheromone table is necessary in order to avoid to forward data along really bad paths. After exponentiation, only the next hop choices expected to be the really good ones are considered to route data packets. Exploration should not be carried out on data packets. On the contrary, they have to be routed exploiting the best paths that have been identified so far by the ants.

Link queues $\mathcal{L}^k$: are data structures independent from AntNet, since they are always present in a node if the node has been designed with buffering capabilities. The AntNet routing component at the node passively *observes* the dynamics of *data packets* in addition to the active generation and observation of the *simulated data packets*, that is, the ants. The status of the local link queues are a snapshot of what is locally going on at the precise time instant the routing decision must be taken, while the $\mathcal{T}^k$'s values provides what the ant agents have learned so far about routing paths. $\mathcal{T}^k$ locally holds information about the *long-term* experience accumulated by the collectivity of the ants, while the status of the local link queues provides a sort of *short-term* memory of the traffic situation. It will be shown that is very important for the performance of the algorithm to find a proper trade-off between these two aspects.

Statistical parametric model $\mathcal{M}^k$: is a vector of $N-1$ data structures (μ_d, σ_d^2, W_d), where μ_d and σ_d^2 represent respectively the sample mean and the variance of the traveling time to reach destination d from the current node, while W_d is the best traveling time to d over the window of the last w observations concerning destination d. All the statistics are based on the delays $T_{k\to d}$ experienced by the ants traveling from their source to destination nodes (and going back to destination). $\mathcal{M}^k$ represents the local view of the current traffic situation on the paths that are used to reach each destination d. In a sense, $\mathcal{M}^k$ is the *local parametric view* of the *global network traffic*.[9]

For each destination d in the network, the sample mean μ_d and its variance σ_d^2 are assumed to give a sufficient representation of the expected time-to-go and of its stability. The mean and the variance have been estimated using different sample statistics: arithmetic, exponential, and with moving window. Different estimation strategies have provided similar results, but the best results have always been observed using the exponential model:[10]

$$\begin{aligned} \mu_d &\leftarrow \mu_d + \eta(o_{k\to d} - \mu_d), \\ \sigma_d^2 &\leftarrow \sigma_d^2 + \eta\big((o_{k\to d} - \mu_d)^2 - \sigma_d^2\big), \end{aligned} \tag{7.2}$$

[9] In the following we will make use of superscripts to indicate the sub-part of $\mathcal{M}$ and $\mathcal{T}$ that refer to a particular destination: $\mathcal{M}_d^k$ and $\mathcal{T}_d^k$ will refer to the information specifically related to node d as destination contained in the $\mathcal{M}$ and $\mathcal{T}$ structures of node k.

[10] This model is the same model used by the Jacobson/Karels algorithm to estimate retransmission timeouts of the TCP on the Internet [351].

where $o_{k \to d}$ is the new observation, that is, the traveling time incurred by the reporting agent for traveling from k to destination d.[11]

In addition to the exponential mean and variance, also the best (i.e., the lowest) value W_d of the reported traveling times over an observation window of width w observations is stored. The value W_d represents an estimate of the minimum time-to-go to node d from the current node calculated according to a non-sliding window of w samples. That is, at $t = 0$, W_d is initialized to $\mu_d(0)$ and is then updated according to the next w samples. If the $w + 1$-th observation happens at time t_{w+1}, then W_d is reset to the current value of $\mu_d(t_{w+1})$ and the counter w is set back to 1. The values of η in 7.2 and of w can be set such that the number of effective samples for the exponential averages are directly related to those used to monitor the value of W_d. According to the expression for the number of effective samples as reported in Footnote 11, the value of w is set as:

$$w = 5(c/\eta), \quad c \in (0, 1]. \tag{7.3}$$

In this way, the relationship between the number of effective samples used for the exponential averages and those used for monitoring the best value is understood and under control. In the experiments reported in the next chapter, c has been set to 0.3. In this way W is updated inside a window slightly shorter than that used for the exponentials.

In principle, also the data packets, other than the ants, could have been used to accumulate statistics. However, in order to update the statistics for going from $k \to d$ using packet traveling times, one should: (i) use the acknowledgments sent at the transport layer for data packets going from $d \to k$, or (ii) assume that the network is cost-symmetric such that $T_{k \to d} = T_{d \to k}$, such that the trip times of packets moving in both directions can be exploited, or (iii) use either the ants as *carriers*, such that at d the packet statistics for $T_{k \to d}$ are accumulated, and then are brought back to k when an ant to k passes by d (a similar approach has been followed in [224]). Since in our network model we do not send acknowledgment packets and we do not quite unrealistically assume that the network is cost-symmetric, the options (i) and (ii) are automatically ruled out. On the other hand, strategy (iii) could have been implemented but we did not find it necessary, both considering the additional overhead introduced by ant carriers and the fact that actually data packets do not have a time stamp by default, such that this had to be added to each packet payload, incurring in slightly increased requirements of bandwidth and processing time per packet. Moreover, is our specific design choice to have an algorithm that do not interfere with data packets other than for what concerns their forwarding.

$\mathcal{T}$ and $\mathcal{M}$ can be seen as *local long-term memories* capturing different aspects of the global network dynamics. The models $\mathcal{M}$ maintain estimates of the *time distance*, and of its variability, to all the other nodes, while the pheromone table holds, for each possible destination, probabilistic estimates of the *relative goodness* of choosing one specific next hop to reach the destination. On the other hand, the status of the link queues $\mathcal{L}$ is a *short-term memory* of what is expected, in terms of waiting time, to reach a *neighbor node*.

In the terms of learning the routing policy, $\mathcal{T}$ and $\mathcal{M}$ can be seen as the components of an *actor-critic* [15] architecture. $\mathcal{M}$, which learns the model of the underlying traffic process, is the critic, which evaluates and reinforces the action policy of $\mathcal{T}$, the actor. This situation is quite different from those considered before of static combinatorial problems: here becomes necessary

[11] The factor η weights the number of most recent samples that will really affect the average. The weight of the t_i-th sample in the value of μ_d after j samples, with $j > i$, is: $\eta(1-\eta)^{j-i}$. For example, for $\eta = 0.1$ approximately only the latest 50 observations will really influence the estimate, while for $\eta = 0.05$, they will be the latest 100, and so on. Therefore, the number of effective observations is $\approx 5(1/\eta)$.

to learn not only a decision policy, but also a (local) model of the current problem instance, that is, of the current traffic patterns. These aspects are discussed more in detail in the following.

In addition to the above node data structures, each ant has also its *private memory* $\mathcal{H}$, where the personal history of the ant (e.g., visited nodes, waiting times, etc.) is maintained and carried along.

7.1.3 Description of the algorithm

7.1.3.1 Proactive ant generation

At regular intervals Δt from every network node s, a forward ant, $F_{s\to d}$ is *proactively* launched toward a destination node d with the objective of discovering a feasible, low-cost path from s to d, and at the same time, to investigate the load status of the network along the followed path. Forward ants share the same queues as data packets. In this way they experience the same traffic jams as data packets. In this sense, forward ants represent a faithful simulation of data packets. Being the forward ant an *experiment* aimed at collecting useful information, its characteristics can be assigned accordingly to the specific purposes of the experiment. It is in this sense that the destination node is assigned according to a probabilistic model biased toward the destinations more requested by data packets. That is, then the destination of a forward ant is chosen as d with a probability p_d,

$$p_d = \frac{f_{sd}}{\sum_{d'=1}^{N} f_{sd'}}, \tag{7.4}$$

where f_{sd} is the number of bits (or packets) bounded for d that so far have passed by s.

REMARK 7.5 (BIASED PROACTIVE SAMPLING): *Ant experiments are directed at proactively collecting data for the destinations of greater interest. The probabilistic component ensures that also destinations seldom requested will be scheduled as destinations for forward ants. If, by any reasons, a destination seldom requested in the past starts to be a hit, the system is somehow ready to deal with the new situation on the basis of the routing information accumulated in the past in relationship to that destination. When a destination starts to be requested more and more, an increasing number of ants will be headed toward it.*

Also other characteristics of the forward ant could be assigned on the basis of the specific purposes of the experiment the ant is associated to. In AntNet, all the generated forward ants have the same characteristics, they possibly differ only for the assigned source and destination nodes. On the contrary, in ACR forward ants are created with different characteristics also for what concerns their exploratory attitude, the way they communicate information, and so on.

The *ant generation rate*, $1/\Delta t$ determines the number of experiments carried out. A high number of experiments is necessary to reduce the variance in the estimates, but at the same time too many experiments might create a significant overhead in terms of routing traffic that could eventually have a negative impact on the overall network performance. It is very important to find a good trade-off between the need to keep collecting fresh data and reduce variance, and the need to avoid to congest the network with routing packets. In AntNet the ant generation rate is a fixed parameter of the algorithm, while ACR points out the need to adaptively change the generation rate according to the traffic characteristics.

On the assumption that the node identifiers are assumed to be known

In AntNet the identifiers (e.g., the IP addresses) of all the nodes $c_i,\ i = 1, \ldots, N$, participating to the network are assumed to be known. Then, at time $t = 0$ the routing tables, each containing

the N node identifiers, are instantiated and the algorithm can proceed by proactively collecting routing information concerning the N network nodes. In practice, this scenario reflects a situation of *topological stability* that possibly follows a "topological transitory" during which, for instance, a newly added node has advertised its presence to the network through some form of broadcasting/flooding. The realization of topological updates in an efficient way is really a matter of protocol implementation details that we have chosen to bypass since we are more interested in *traffic engineering* [9] for what concerns the better exploitation of network resources according to the traffic patterns. For instance, instead of pragmatically passing to each node from a configuration file the whole network description, we could have made the algorithm starting with empty routing tables, and then let the nodes probing their neighbors in order to know both the identifiers and characteristics (bandwidth and propagation delay) of the attached links, and have an initial phase of IP addresses broadcasting such that each could have automatically and independently built the routing table as a dynamic list structure. However, although more realistic, this would have just resulted in additional and quite uninteresting lines of software.

Some authors [274] have also argued that the AntNet strategy of keeping in the routing tables entries for all the network node identifiers might be unfeasible in large networks. However, since this is what precisely other Internet algorithms do (in particular OSPF, which maintains a full topological map), this argument can be ruled out once either a hierarchical network organization is assumed as in the Internet, or is assumed that router will become more than "switching boxes" with few kilobytes of memory as still happens today, but rather sort of specialized network computers with on-board possibly gigabytes of memory, as any cheap desktop can nowadays have.

The assumption of topological stability, as well as the fact that it is perfectly reasonable in wired IP networks to hold the list of all nodes on the same hierarchical level, result in the fact that AntNet is based on a purely proactive approach. In a sense, given these assumptions, there is no real need to make use of *reactive* strategies. On the other hand, it is rather natural to extend the AntNet's basic design with the inclusion of also reactive components. For instance, let the routing tables hold routing information only for those destinations that the node has so far heard about (i.e., the destinations of the data packets that have passed through the node). Therefore, when a new, previously unknown, destination is asked for by a local traffic session, the routing path has to be built *on-demand* from scratch. In this case it is clearly necessary to use also a *reactive* (or, on-demand) approach: before the session can start, agents must be sent in order to find a routing path for the new destination. This is the common situation in mobile ad hoc networks, for instance, since in those networks the normal status of operations consists in a continual topological reshaping of the network due to both mobility and nodes entering/leaving the network (see the description of AntHocNet in Subsection 7.3.2.2). Searching for a previously unknown destination has to necessarily rely on some form of broadcasting/flooding: all the nodes are tried out until either the searched destination is found or some nodes holding information about how to reach the destinations are found (e.g., AntHocNet, AODV [349, 103]). It is a sort of blind search, that can be possibly made more efficient by first searching, for instance, on some logical overlay network of nodes that, according to some heuristics, are expected to hold useful information about the searched destination. On the other hand, the same topology flooding of OSPF can be seen in these terms, with the signaling happening directing from the edge of the node that has newly entered the network. Again, this issue has not been explicitly considered in AntNet, since it can be seen as of minor importance in wired IP networks where some complete topological view of the network can be efficiently maintained and topology at the hierarchical levels of router/gateways does not change so often and quickly. The problem of setting up initial routing directions for all the network nodes is "solved" by allowing an initial unloaded phase during which ants are simply flooded into the network and build up shortest paths routing tables for all pairs of network nodes (see also Section 8.3). In rather general terms:

REMARK 7.6 (REACTIVE VS. PROACTIVE SAMPLING): *Reactive sampling can be seen as the process of* discovery on-demand *routing directions (usually for destinations for which no routing information is held at the node), while proactive sampling can be seen as the process of either discovering routing directions for destinations that might be addressed in the future and/or* maintaining *and* adapt *previously established paths.*

We will discuss again in general terms this issue about reactive/proactive behaviors when considering the cases of QoS (AntNet+SELA) and mobile ad hoc networks (AntHocNet).

7.1.3.2 Storing information during the forward phase

While traveling toward their destination nodes, the forward ants keep memory of their paths and of the traffic conditions encountered. The identifier of every visited node k and the step-by-step time elapsed since the launching time are saved in appropriate list structures contained in the ant's private memory $\mathcal{H}$. The list $V_{v_0 \to v_m} = [v_0, v_1, \ldots, v_m]$ maintains the ordered set of the nodes visited so far, where v_i is the identifier of the node visited at the i-th step of the ant journey. Analogously, the list $T'_{v_0 \to v_m} = [T_{v_0 \to v_1}, T_{v_1 \to v_2}, \ldots, T_{v_{m-1} \to v_m}]$ holds the values of the traveling times experienced while moving from one node to the other (i.e., the time elapsed from the moment the ant arrived at node v_i to the moment node v_{i+1} was reached).

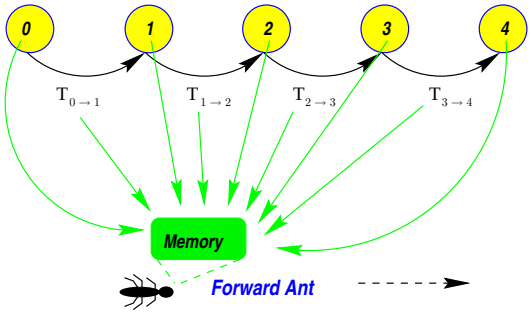

Figure 7.2: *Forward ants keep memory of each visited node and of the visiting time.*

7.1.3.3 Routing decision policy adopted by forward ants

At each intermediate node k, the forward ant $F_{s \to d}$ headed to its destination d must select the neighbor node $n \in \mathcal{N}_k$ to move to.

If $n \in V_{s \to k},\ \forall n \in \mathcal{N}_k$, that is, all the neighbors have already been visited by the forward ant, then the ant chooses the next hop by picking up at random one of the neighbors, without any preference but excluding the node from which the ant arrived in k. That is, in this case, if the current node k is being visited at the i-th step, the probability p_{nd} assigned to each neighbor n of being selected as next hop is:

$$\begin{cases} p_{nd} = \dfrac{1}{|\mathcal{N}_k| - 1} & \forall n \in \mathcal{N}_k \ \wedge \ (n \neq v_{i-1} \ \vee \ |\mathcal{N}_k| = 1) \\ p_{nd} = 0 & \text{otherwise} \end{cases} \tag{7.5}$$

On the other hand, in the most common case in which some of the neighbors have not been visited yet, the forward ant applies a *stochastic decision policy* π_ϵ, which, as usual, is parametrized by:

- *Local pheromone variables*: the values τ_{nd} of the pheromone's stochastic matrix $\mathcal{T}_k$ corresponding to the estimated goodness of choosing n as next hop for destination d.

- *Local heuristic variables*: the values l_n based on the status of the local link queues $\mathcal{L}^k$:

$$l_n = 1 - \frac{q_n}{\sum_{n'=1}^{|\mathcal{N}_k|} q_{n'}}. \tag{7.6}$$

  l_n is a [0,1] normalized value proportional to the length q_n, in terms of bits waiting to be sent, of the queue of the link connecting the node k to its neighbor n.

Moreover, decisions depend also on the contents of the *ant private memory* $\mathcal{H}(k)$, that contains the list $V_{s \to k}$ of the nodes visited so far, and which is used in order to build *feasible* solutions in the sense of *avoiding loops*. Therefore, taking into account all these components, the policy π_ϵ selects neighbor n as next hop node with a probability P_{nd} precisely defined as follows:

$$\begin{cases} p_{nd} = \dfrac{\tau_{nd} + \alpha l_n}{1 + \alpha(|\mathcal{N}_k| - 1)} & \forall n \in \mathcal{N}_k \ \wedge \ n \notin V_{s \to k} \\ p_{nd} = 0 & \text{otherwise} \end{cases} \tag{7.7}$$

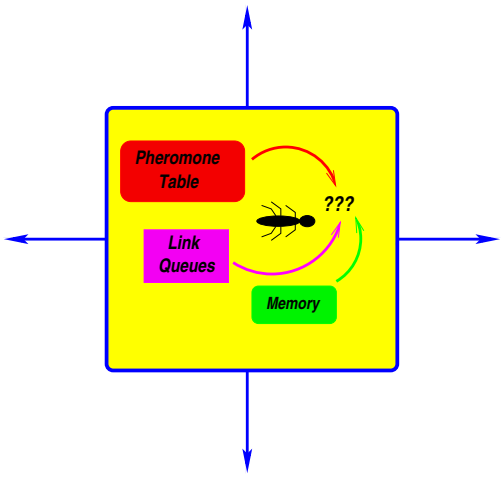

Figure 7.3: *The stochastic decision policy π_ϵ of the forward ants is parametrized by the entries in the pheromone table, the status of the local link queues (heuristic values), and depends on the memory of the already visited nodes (loops avoidance).*

The probability assigned to each neighbor is a measure of the *relative goodness*, with respect to all the other neighbors, of using such a neighbor as a next hop for d as final destination. The value of $\alpha \in [0, 1]$ weighs the relative importance of the heuristic correction with respect to the pheromone values stored in the pheromone matrix. Since both the values of the heuristic corrections and those of the pheromones are normalized to the same scale in [0,1] no additional scale factors are needed. Moreover, the use of normalized values allows for a computationally efficient calculation of the p values since the normalizing denominator can be computed once for all and in a straightforward way. In this case the *ant-routing table* entries takes the form $a_{nd} = \tau_{nd} + \alpha l_n$. This is an additive combination of the pheromone and heuristic values, similar to that adopted in the ANTS sub-class of ACO algorithms (see Subsection 5.1.2), in which the heuristic term is also adaptively computed and given an importance comparable to that of the pheromone term.

The l_n values reflect the instantaneous state of the node queues, and, assuming that the queues' consuming process is almost stationary or slowly varying, l_n gives a quantitative measure of the *expected waiting time* for a new packet added to the queue. As already pointed out,

this measure is a snapshot on the current local traffic situation. The values of the pheromone variables, on the other hand, are the outcome of a continual learning process carried out by the collectivity of the agents. Their values try to capture both the current and the recent past status of the whole network as seen by the local node. Pheromone is the *collective long-term memory* maintained at the local node, while the link queues are the expression of the *short-term* processes that are locally happening.

REMARK 7.7 (LONG-TERM MEMORY VS. SHORT-TERM LOCAL PREDICTION): *Assigning the probability values according to the weighted sum of Equation 7.7 tells that the routing decisions for the ants are taken on the basis of a chosen* trade-off, *the value of α, between estimates coming from a long-term process of collective learning, and estimates coming from a sort of* instantaneous *heuristic prediction based on a completely local view.*

A value of α close to 1 dumps the contribution of the ants collective learning, with the system closely following the local traffic fluctuations, and possibly resulting in large oscillations in performance. On the contrary, an α close to 0 makes the decision completely dependent on the long-term ant learning process, and it can result unable to quickly follow variations in the traffic patterns. In both cases the system is not expected to behave in a really satisfactory way. The number of ants, that is, the ant generation rate at each node, is expected to play a really critical role in both these cases. A large number of ants can greatly help to overcome the negative effects of an α close to 0, while the same large amount of ants can create an over-reactive behavior in the case of an α close to 1. In all the ran experiments it was observed that the good balancing between the values of τ and l is very important to get good performance. Clearly, depending on the characteristics of the network scenario at hand, the best value to assign to the weight α can vary, but from the experiments that we have carried out it seems that a robust and good choice is to assign α in the range between 0.2 and 0.5. Performance for this range of values is good and does not change greatly inside the range itself. For $\alpha < 0.2$ the effect of l is vanishing, while for $\alpha > 0.5$ the resulting routing tables oscillate and, in both cases, performance degrades appreciably.

7.1.3.4 Avoiding loops

The role of the *ant-private memory* $\mathcal{H}$ in Equation 7.7 is to avoid *loops*, when possible. A cycle for an ant agent is in practice a waste of time and resources. Although, according to the fact that both a stochastic policy and an adaptive updating of the routing tables are adopted, loops are expected to be short-lived (see also [72] for a general discussion on loops in ant-based routing systems). However, it is clear that loops should be avoided as much as possible, especially concerning data packets, since in this case the user will incur in much longer and highly undesired packet latencies.

When ACO is applied to problems of combinatorial optimization, the private memory of the ant is used as a practical tool to guarantee the step-by-step feasibility of the building solution. On the contrary, in the routing case feasibility basically means loop-free, that is, the ability to reach in finite (possibly short) time the target destination. Feasibility becomes a major issue in the case of route setup for QoS traffic, however AntNet is thought for best-effort traffic.

If a cycle is detected, that is, if an ant is forced to return to an already visited node, the nodes composing the cycle are taken out from the ant's internal memory, and all information about them is destroyed. If the ant moved on a cycle for a time interval which is greater than half of its age at the end of the cycle, the ant is destroyed. In fact, in this case the agent wasted a lot of time in the cycle, therefore, for what concerns the nodes visited before entering the cycle, it is carrying a possibly out-of-date picture of the the network state. In this case, it can be counterproductive to still use the agent to update the routing tables on those nodes.

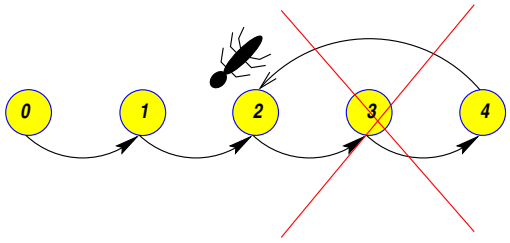

Figure 7.4: *Cycles are removed from the ant memory.*

Related to cycles is the issue *maximum time-to-live* (TTL) of an ant. If a forward ant does not reach its destination before of an assigned maximal value for its life time, the ant is destroyed. In all the experiments the maximum time-to-live has been set to 15 seconds, both for forward ants and data packets, similarly to the TTL values normally adopted in the Internet.

7.1.3.5 Forward ants change into backward ants and retrace the path

When the destination node d is reached, the forward agent $F_{s\to d}$ is virtually transformed into another agent $B_{d\to s}$ called *backward ant*, which inherits all its memory.

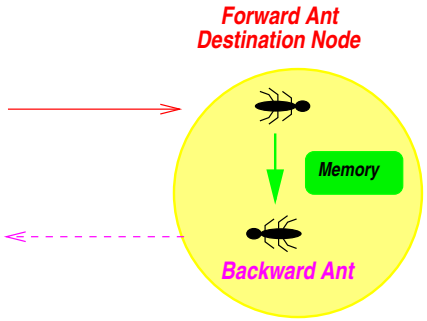

Figure 7.5: *Arrived at the destination node the* forward ant *is transformed into a* backward ant*, which inherits from the former all its memory.*

REMARK 7.8 (THE UPDATING PHASE): *When the destination node is reached, the random experiment realized by the forward ant is concluded. With the backward ant starts the* updating phase *that, according to the distributed nature of the network, requires* physically retracing *the path followed during the forward journey.*

The outcome of the experiment, that is, the "discovered" path, has to be *evaluated*, such that a measure of *goodness* can be assigned to it. In turn, this measure of goodness can be used to update the statistical estimates (the local models of the network traffic, and the pheromone and data-routing tables) maintained at the nodes along the discovered path. The estimates at each node are updated independently from those carried out at other nodes: there is neither bootstrapping nor global propagation of local estimates. The updates are executed in plain Monte Carlo fashion, in the sense previously discussed.[12]

[12] The AntNet strategy made of realizing a "path experiment" and then retracing and evaluating the steps of the experiment, in order to update some estimates, in a generic sense *beliefs*, associated to the situations observed during the experiment, is well captured by the following phrase of the great Danish philosopher Soren Kierkegaard: *Life can only be understood going backwards, but it must be lived going forwards.*

The backward ant takes the same path as the one followed by the corresponding forward ant, but in the opposite direction.[13] At each node k along the path the backward ant knows to which node it has to move to next by consulting the ordered list $V_{s\to k}$ of the nodes visited by the forward ant.

REMARK 7.9 (BACKWARD ANTS MOVE OVER HIGH-PRIORITY QUEUES): *Backward ants do not share the same link queues as data packets; they use* higher priority queues, *because their task is to quickly propagate to the nodes the information accumulated by the forward ants during their journey from s to d.*

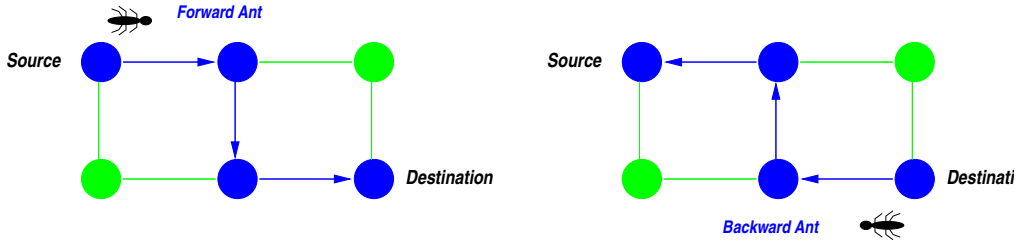

Figure 7.6: *Forward and backward ants visit the same sequence of nodes but in the opposite direction. The backward ant traces back the path followed by the forward ant.*

7.1.3.6 Updating of routing tables and statistical traffic models

Arriving at a node k from a neighbor node f, the backward ant updates all the data structures at the node. Using the information contained in the memory inherited from the forward ant, the backward ant $B_{d\to s}$ executes the following sequence of operations: (i) *update* of the local models $\mathcal{M}^k$ of the networks traffic for what concerns the traveling times from k to d, (ii) *evaluation* of the path $k \to d$ in terms of the value of the *traveling time* $T_{k\to d}$ experienced by the forward ant with respect to the expected traveling time according to the local model $\mathcal{M}^d_k$: smaller is $T_{k\to d}$ with respect to the previously observed traveling times, higher will be the score assigned to the path, (iii) use of the assigned score to locally *reinforce* the path followed by the forward ant, that is, to reinforce the choice of f as next hop node when the destination is d in both the pheromone and the data-routing tables $\mathcal{T}^k$ and $\mathcal{R}^k$.

UPDATING OF THE LOCAL MODELS OF THE NETWORK TRAFFIC PROCESSES

$\mathcal{M}^k$ is updated consulting the list $T'_{k\to d}$ and considering the value $T_{k\to d}$ of the traveling time experienced by the forward ant while traveling from k to d. After setting $o_{k\to d} = T_{k\to d}$, the Equations 7.2 are used to update the values for μ_d and σ^2_d. If $o_{k\to d} < W_d$, then $W_d = o_{k\to d}$.[14]

The values of the parameters of the statistical models $\mathcal{M}_k$ can show important variations, depending on the variable traffic conditions. The statistical model has to be able to capture this variability and to follow it in a robust way, without unnecessary oscillations. The robustness of the local model of the network-wide traffic plays an important role for the correct functioning of

[13] This assumption requires that all the links in the network are bi-directional. In modern networks this is a reasonable assumption.

[14] A correct use of traveling times requires the ability to actually calculate such times. The are two main possibilities: (i) if all the nodes have a "practically" synchronized clock (this is quite common nowadays), it will suffice to read the node's clock on arrival and calculate a trivial difference, (ii) if the clocks cannot be assumed as synchronized, then the time to hop from one node to another is the sum of the time spent at the node since the arrival and of the time necessary for the packet propagation along the link, which can be easily calculated with sufficient approximation once the link propagation characteristics are known.

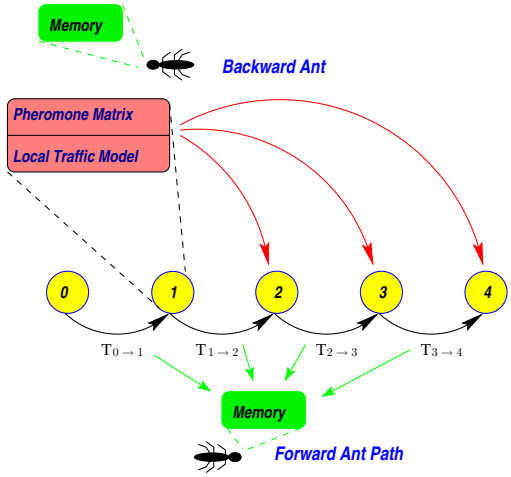

Figure 7.7: *At each visited node the backward ant $B_{4\to 0}$ makes use of the information contained in the memory inherited from the forward ant in order to: (i)* update *the local parametric models $\mathcal{M}$ for what concerns the traveling times to node 4, (ii)* evaluate *the path $k \to 4$, where $k \in \{0,1,2,3\}$ is the current node, on the basis of the values hold in $\mathcal{M}^k$, and (iii) use the assigned score to locally* reinforce *the path followed by the forward ant, that is, when the current node is for instance node 1, the choice of 2 as next hop for 4 as a destination is reinforced in $\mathcal{T}^k$ and, consequently, in $\mathcal{R}^k$.*

the algorithm since it provides the reference values to evaluate the followed path. The following example can help to clarify this point.

EXAMPLE 7.1: IMPORTANCE OF THE STATISTICAL MODELS $\mathcal{M}$ FOR PATH EVALUATION

Let $\mathcal{M}_d^k$ containing the current values: $\mu_d = 1\ sec$, $\sigma_d^2 = 0.01\ sec^2$, $W_d = 0.9\ sec$. Moreover let us assume that the input traffic is stationary so far. The question is: how good is a new reported traveling time equal, for example, to 1.5 seconds? According to the values in the parametric model that, assuming $\eta = 0.3$, become $\mu_d = 1.15$ and $\sigma_d^2 = 0.073$, the answer is that the new trip time is a rather bad one. Accordingly, the associated path, let us call it $\mathcal{P}_{1.5}$, is a bad one and should not be really used for data routing, but data should be routed instead to the next hops along the path(s) $\mathcal{P}_{0.9}$. If, after some short time, there is a sudden increase in the input traffic on the nodes along the paths $\mathcal{P}_{0.9}$, a new backward ant traveling along these paths will report now a trip time of, for instance, 2 seconds. Since these paths were perceived as the good ones, let us assume that actually not one but two ants come back along these paths, both reporting a trip time of 2 seconds. The model's values then become: $\mu_d = 1.583\ sec$ and $\sigma_d^2 = 0.354\ sec^2$. Now, a third ant coming back from $\mathcal{P}_{1.5}$ will actually find that its path has now become a good one, since, for instance, its distance from the average in standard deviation units has become $\zeta = (1.5 - 1.583)/\sqrt{0.354} = -0.15$, while before it was +5. Therefore, the path $\mathcal{P}_{1.5}$, if stationary for what concerns traveling time, should be now preferred to $\mathcal{P}_{0.9} \equiv \mathcal{P}_2$. This simple example wants to show that, due to the non-stationarity of the input traffic, it is always necessary to make relative comparisons of the traveling times. The end-to-end delay of 1.5 seconds of path $\mathcal{P}_{1.5}$ is firstly judged as bad but eventually it becomes a good one as a consequence of the increase of the traffic load on other parts of the network. The adaptive model $\mathcal{M}$ is precisely aimed at maintaining track of changes in the traveling times in order to score the paths in a meaningful way.

Clearly, an adaptive model able to track the traffic fluctuations with extreme robustness is rather hard to design, and likely also computationally expensive. We have chosen a parametric model with moving windows in order to optimize at the same time efficiency and robustness. While all the three updated parameters are important, W plays a more prominent role since it

provides an unbiased estimate of the optimal traveling performance obtainable at the current moment. On the contrary, for instance, the worst trip time is not similarly recorded, because, in principle, this value time is not bounded (in reality, is bounded by the maximum time-to-live associated to an ant, but this bound is not really of practical interest).

EVALUATION OF THE PATH AND GENERATION OF A REINFORCEMENT SIGNAL

After updating the local traffic model $\mathcal{M}_k^d$, the path that was followed by the forward ant from $k \rightarrow d$ must be evaluated. Evaluation is done on the basis of the experienced traveling time $T_{k \rightarrow d}$ only. The simple Example 7.1 of the previous paragraph has pointed out the critical role of a proper evaluation, as well as, the involved difficulties and the strong dependence on the robustness, of the traffic model. The purpose of the *evaluation phase* is the generation of a *reinforcement signal* r to be used to update the pheromone and data-routing tables.

The relationship between: (i) the learning of the characteristics of the input traffic processes ($\mathcal{M}$), and (ii) the learning of the routing policy ($\mathcal{T}$), is in the form of an *actor-critic* [15] architecture. Learning by the critic about the input networks-wide traffic processes is necessary in order to evaluate in a proper way the effects of the routing policy defined by $\mathcal{T}$, the actor. The outcome of an ant experiment is evaluated on the basis of the model $\mathcal{M}$, and then the current policy $\mathcal{T}$, which generated the outcome, is reinforced accordingly to this evaluation. Given the centrality,

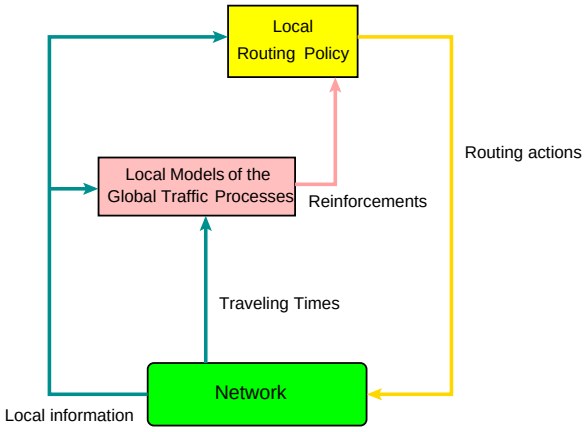

Figure 7.8: *The* actor-critic *scheme implemented in AntNet by the two learning components $\mathcal{M}$, the* critic, *locally learning the characteristics of the network-wide input traffic, and $\mathcal{T}$, the* actor, *learning the local routing policy. The traveling times T reported by the ants using the routing policy defined by the actor, are fed into the critic component and used to learn the main characteristics of the input traffic processes. In turn, the learned model is used by the critic to evaluate and reinforce the policy implemented by the actor. The signal "local information" summarize all the additional locally available information, which is a sort of imperfect "state" signal.*

as well as, the complexity of the issue related to the definition of an effective evaluation and definition of a proper reinforcement signal r given the intrinsic variability of the traffic patterns and the characteristics of spatial distribution, the next Subsection 7.1.4 is completely devoted to discuss this issue. Here, it is sufficient to say that r, according to the actor-critic scheme above, is a function of both $T_{k \rightarrow d}$ and $\mathcal{M}_d^k$: $r \equiv r(T_{k \rightarrow d},\ \mathcal{M}_d^k),\ r \in\ (0, 1]$. r is a dimensionless value which is used by the current node k as a positive reinforcement for the node f the backward ant $B_{d \rightarrow s}$ comes from. r is assigned taking into account the so far observed traveling times such that the smaller $T_{k \rightarrow d}$ is, the higher r is.

UPDATING OF THE PHEROMONE TABLE

Assumed that a score and, accordingly, a reinforcement value r, has been attributed to the ant path with associated traveling time $T_{k \to d}$, the question is now how to use this value to update the ant-routing and data-routing tables.

The pheromone table $\mathcal{T}^k$ is changed by incrementing the probability τ_{fd} (i.e., the probability of choosing neighbor f when destination is d) and decrementing, by normalization, the other probabilities τ_{nd}. The amount of the variation depends on the value of the assigned reinforcement r in the following way:

$$\tau_{fd} \leftarrow \tau_{fd} + r(1 - \tau_{fd}). \tag{7.8}$$

In this way, the probability τ_{fd} will be increased by a value proportional to the reinforcement received and to the previous value of the probability. That is, given the same reinforcement, small probability values are increased proportionally more than large probability values, favoring in this way a quick exploitation of new, and good discovered paths.

The probabilities τ_{nd} associated to the selection of the other neighbor nodes $n \in \mathcal{N}_k$ implicitly all receive a negative reinforcement by normalization. That is, their values are decreased in order to make all the probabilities for the same destination d still summing up to 1:

$$\tau_{nd} \leftarrow \tau_{nd} - r\tau_{nd}, \quad \forall n \in \mathcal{N}_k,\ n \neq f. \tag{7.9}$$

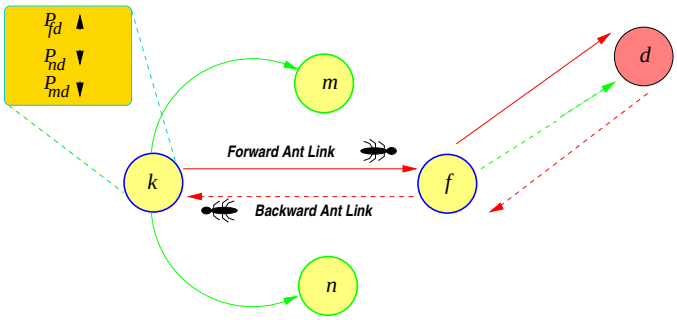

Figure 7.9: *Updating of the pheromone table at node k. The path using neighbor f to go to d is reinforced: the selection probability τ_{fd} is increased, while the probabilities τ_{md} and τ_{nd} associated to the other two neighbors, m and n, are decreased by normalization.*

REMARK 7.10 (PHEROMONE VALUES INCREASED BY BOTH EXPLICIT AND IMPLICIT REINFORCEMENTS): *According to Equation 7.8,* every discovered path receives a positive reinforcement *in its selection probability In this way, not only the (explicit) assigned value r plays a role, but also the* (implicit) *ant's* arrival rate *does. This strategy is based on trusting the paths that receive either high reinforcements, independently from the frequency of the reinforcements themselves, or low but frequent reinforcements.*

Under any conditions of traffic load, a path can receive high-valued reinforcements if and only if it is much better than the paths followed in the recent past, as indicated by the model estimates $\mathcal{M}$. On the other hand, during a sudden increase in traffic load, most of the paths have high traveling times with respect to the traveling times expected according to $\mathcal{M}$, which is somehow still "remembering" the previous situation of low congestion. Therefore, none of the paths will be able to receive an high reinforcement due to the current misalignment between the estimates of the local models and the current traffic load. In this case, even if good paths cannot be reinforced adequately by the single ant, the effect of the high number of ants that choose them

(likely because of the link queue factor in the decision rule) will result in cumulative high-valued reinforcements. Exploiting the frequency of agent arrivals due to shorter time paths is closely reminiscent of what happens in real ants and that allow them to discover shortest paths by using distributed pheromone trails (Section 2.1).

From Equations 7.8 and 7.9, it results that a single probability value τ_{nd} can in practice become equal to 1. Accordingly, all the other entries become equal to 0. Anyhow, this situation is not "harmful" because, according to Equations 7.5 and 7.7 a neighbor can still be chosen as next hop, due to either the situation of already all visited neighbors of Equation 7.5, or to a favorable status of the link queues given $\alpha > 0$ in the Equation 7.7. Once a neighbor has been selected as next hop, it will consequently receive a reinforcement $r > 0$, therefore, according to Equation 7.8, also its probability value in the ant-routing table will change from 0 to r. However, in order to keep a good level of exploration even, for instance, in the case of low traffic (such that the link queues do not get appreciably long), we have adopted the artifice of putting some limits on pheromone values. That is, a value τ_{max} is assigned such that if after updating pheromone becomes larger than τ_{max}, it is just set to τ_{max}. The corresponding τ_{min} clearly depends on the number of neighbors at each node. For instance, we have set $\tau_{max} = 0.01$. It can be easily recognized that this is the same strategy adopted in $\mathcal{MM}$AS, for example.

Also, the AntNet pheromone updating rule shares strong similarities with that characterizing the so-called *ACO hypercube framework* [44]. In fact, pheromone values are constrained in the interval $[0, 1]$ and the $\Delta\tau \equiv r$ is also in $[0, 1]$. However, it differs from the hypercube rule in the fact that at each increase corresponds also a decrease of the related alternatives. Most of the ACO implementations for static problems make use of pheromone variables whose values can have a possibly unbounded (or softly-bounded by defining min and max limits) increase, with pheromone evaporation at work in order to avoid stagnation. In AntNet, given the non-stationary nature of the problem at hand, such a strategy is not expected to work properly. In particular, the pheromone decrease frequency should strictly depend on the (unknown) variability in the traffic patterns to be effective and to allow to adapt to the changing conditions.

UPDATING OF THE DATA-ROUTING TABLE

The data-routing table $\mathcal{R}$ is updated after every update in the pheromone table. As explained at Page 206, where the characteristics of the data-routing table were discussed, data packets are routed according to a stochastic policy, whose parameters are the entries of the data-routing table. The probability of each possible routing decision is obtained from the corresponding entry $\mathcal{R}_{nd}$. Differently from the ant rule of Equation 7.7, no additional components are taken into account. The R's entries are supposed to already summarize all the necessary information about learned pheromone values and local queues. They are defined as the result of an exponential transformation of those of the pheromone table. The purpose of the transformation consists in favoring more the alternatives with high probability at the expenses of those with low probability:

$$\begin{aligned} \mathcal{R}^k_{nd} &= (\tau_{nd})^{\varepsilon}, \\ \mathcal{R}^k_{nd} &= \frac{\mathcal{R}^k_{nd}}{\sum_{i \in \mathcal{N}_k} \mathcal{R}^k_{id}}. \end{aligned} \tag{7.10}$$

In the experiments reported in the next chapter we have used $\varepsilon = 1.4$. Figure 7.10 shows the effect of such a transformation. We have also tried out other values for the exponent in the transformation function. Although, the value $\varepsilon = 1.4$ looked as a good compromise between the need to reduce the risk of forwarding data packets along bad directions, and the possibility of spreading the data over multiple paths.

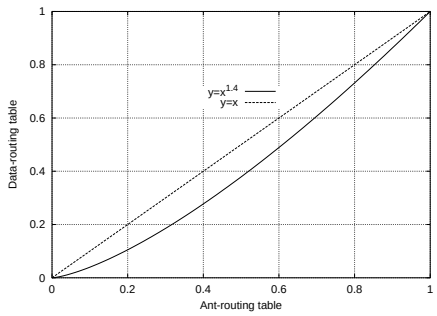

Figure 7.10: *Transformation of the entries of the pheromone table into those of the data-routing table for $\varepsilon = 1.4$ in Equation 7.10. The identical transformation is also evidenced in order to appreciate the difference.*

REMARK 7.11 (IMPORTANCE OF STOCHASTIC DATA ROUTING): *The use of stochastic routing is an important aspect of AntNet, it allows to take full advantage of the intrinsic multipath nature of the routing tables built by the ant agents. Stochastic routing allows to spread the data packets over multiple paths proportionally to their estimated quality, providing an effective load balancing. Moreover, it allows to deal in robust and efficient way with the issue of choosing the precise number of different paths that should be actually used. From the ran experiments, it has been observed an increase in performance up to 30%-40% when using stochastic instead of purely deterministic routing.*

7.1.3.7 Updates of all the sub-paths composing the forward path

In principle, at each node k, the same sequence of updates performed in relationship to the destination d, can be executed considering all the intermediate nodes between k and d as "destination" nodes. If $V_{k \to d}$ is the list of the nodes visited by the forward ant traveling from k to d, then, every node $k' \in V_{k \to d}, k' \neq d$, on the *sub-paths* followed by forward ant $F_{s \to d}$ after visiting the current node k, can be virtually seen as a "destination" from the k's point of view. The possibility of updating all the sub-paths composing a same path means that, if there are m nodes on the path, it is possible to update along the path a total of $m(m-1)/2$ sub-paths.

Unfortunately, the forward ant $F_{s \to d}$ was bounded for node d and not for any of the intermediate nodes. This means that all the ant routing decisions has been taken having d as a target. An example can help to show why, in some specific but rather common situations, is better to be cautious when considering intermediate nodes as destination nodes.

EXAMPLE 7.2: POTENTIAL PROBLEMS WHEN UPDATING INTERMEDIATE SUB-PATHS

Referring to Figure 7.11, let us consider a forward ant originating in A with destination B. Let us assume that nodes $1, 2, 3, 4$ are experiencing a sudden increase of the traffic directed toward node B. Being these nodes directly connected to B, most of this traffic is forwarded on the link directly connected to B. Due to the fact that the increase of traffic from $1, 2, 3, 4$ to B happened suddenly, node A is not yet completely aware of the new traffic situation. The forward ant is therefore routed to node 1, considered that the quickest known path from A to B was the two-hop path passing through 1. Unfortunately, once in 1, the length of the link queue $\mathcal{L}_B^1$ forces the ant to move to 2, also considered that the path through 2 should have a traveling time comparable to that through 1. Again, the congestion on the link directly connected to B moves the ant to node 3, and then further to node 4. Here, the two possible alternatives are: the three hops, congested path $\langle 10, 11, B \rangle$, and the path through C. The ant moves to C, which had a competitive traveling time to B, before the congestion at $1, 2, 3, 4$ due to the direct connection of C with 1 and 2. Once in C the ant, to avoid the cycles, must move to 5. At this point the path of the ant is constrained along the very long path $\langle 5, 6, 7, 8, 9, B \rangle$. Therefore, in the end, the ant's list $V_{A \to B}$ will

contain $[A, 1, 2, 3, 4, C, 5, 6, 7, 8, 9, B]$. *The considered situation is expected to happen quite often under non-stationarity in the input traffic processes.*
Let us consider the possible update actions of the backward ant in A. *In principle, the backward ant can update the estimates concerning all the sub-paths from* A *to any of the nodes in* $V_{A\to B} \setminus \{A\}$. *For instance, using the experienced value of* $T_{A\to C}$, *the backward ant could update in* A *the estimates* $\mathcal{M}_A^C$ *for the the traveling times to* C, *and, accordingly, the value* τ_{1C} *of the goodness of choosing* 1 *as next hop for* C *as destination. The value* $T_{A\to C}$ *corresponds to the long, jammed path* $\langle A, 1, 2, 3, 4, C\rangle$. *But this path is "wrong", in the sense that* if *the destination was* C, *the next hop decision in* A, *or either in* 1 *or* 2, *would have been different. Likely, the path followed would have been either* $\langle A, C\rangle$, *or* $\langle A, 1, C\rangle$, *or* $\langle A, 1, 2, C\rangle$. *In this sense, the use of the experienced value* $T_{A\to C}$ *to update in* A *the estimates concerning* C *is substantially incorrect.*

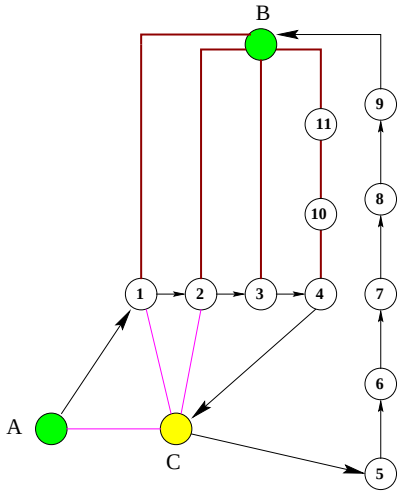

Figure 7.11: *Potential problems when updating all the sub-paths composing the path of a forward ant. The figure is explained in the text of Example 7.2.*

In the example, the followed path $\langle A, 1, 2, 3, 4, C\rangle$ corresponds to a *rare event*, under the current routing policy, when the ant is bounded for C. If a significant number of situations like the one just described happen (e.g., because B is frequently requested by packets passing by A), and all the sub-path are used to update the local models of the input traffic, the statistics of the rare events is completely distorted and, accordingly, all the statistical estimators result distorted.

REMARK 7.12 (SUB-PATH UPDATING IS SAFE UNDER STATIONARY TRAFFIC PATTERNS): *Using a path and all its sub-paths is a consistent procedure only in the case of stationarity in the traffic patterns. The issue concerning the exploitation of the sub-paths is strictly related to the validity of the Bellman's optimality principle (Definition 3.27). Under conditions of imperfect state information and/or quickly time-varying traffic loads, the conditions for the applicability of the principle to not hold anymore, and, accordingly, it might be not correct to use sub-path information.*

According to these reasonings, in AntNet the sub-paths composing the path followed by forward ants are *filtered out* before being selected for statistics updating. The filtering strategy is as follows: the traveling time $T_{k\to d'}$ associated to a sub-path $k \to d'$ is used for updates only if its value seems to be good, that is, if it is less than the sup of an estimated confidence interval $I(\mu_{d'}, \sigma^2_{d'})$ around $\mu_{d'}$. In fact, if the traveling time indicates that the sub-path is good with respect to what observed so far, then it can be conveniently used: a new good path has been discovered "for free". On the contrary, if the sub-path seems to be bad, it it better not to risk

to use it to update the statistical estimates. In practice, from the ran experiments it has been observed that only 10-20% of the sub-paths are actually rejected for updating.

7.1.3.8 A complete example and pseudo-code description

A complete c-like pseudo-code description of the actions of forward and backward ants is reported in Algorithm 7.1, while an example of the forward-backward behavior of AntNet ants is illustrated by means of Figure 7.12. The forward ant, $F_{1\to4}$, moves along the path $1 \to 2 \to 3 \to 4$ and, arrived at node 4, it is transformed in the backward ant $B_{4\to1}$ that will travel in the opposite direction. At each node $k,\ k = 3, 2, 1$, the backward ant uses the contents of the lists $V_{1\to4}(k)$ and $T'_{1\to4}(k)$ to update the values for $\mathcal{M}^k(\mu_4, \sigma_4^2, W_4)$, and, in case of good sub-paths, to update also the values for $\mathcal{M}^k(\mu_i, \sigma_i^2, W_i),\ i = k+1, \ldots, 3$. At the same time, the pheromone table $\mathcal{T}^k$ is updated by (a) incrementing the goodness τ_{j4}, $j = k+1$, of the node $k+1$ which is the node the ant $B_{4\to1}$ came from, for the cases of node $i = k+1, \ldots, 4$ as destination node, and (b) decrementing by normalization the value of the probabilities for the other neighbors (here not shown). The increment is a function of the traveling time experienced by the forward ant going from node k to destination node i. As it happens for $\mathcal{M}$, the pheromone table is also always updated for the case of node 4 as destination, while the other nodes $i' = k+1, \ldots, 3$ on the sub-paths are taken in consideration as destination nodes only if the traveling time associated to the corresponding sub-path of the forward ant is good in statistical sense.

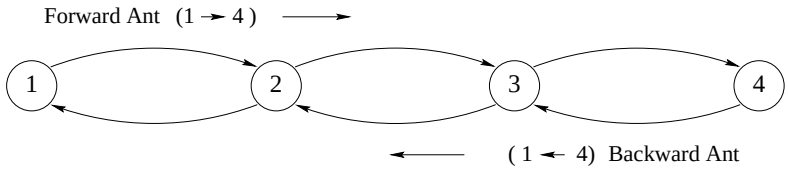

Figure 7.12: *A complete example, described in the text, of the forward-backward behavior in AntNet.*

7.1.4 A critical issue: how to measure the relative goodness of a path?

The traveling time (or *end-to-end delay*) $T_{k\to d}$ experienced by the forward ant is the metric used in AntNet to measure the goodness of the followed path $\mathcal{P}_{k\to d}$. $T_{k\to d}$ is a good indicator of the absolute quality of $\mathcal{P}_{k\to d}$ because is a sort of aggregate measure that depends on both physical characteristics (number of hops, transmission capacity of the used links, processing speed of the crossed nodes) and traffic conditions (the forward ants share the same queues as data packets).

EXAMPLE 7.3: ALTERNATIVES TO THE USE OF TRAVELING TIME FOR PATH EVALUATION

In principle, other metrics could have been used to score the path (this could be a future issue to explore). For instance, the number of hops *could have been also used. A routing strategy preferentially choosing the paths with minimal number of hops tends to minimize resources utilization in terms of nodes involved in routing data traffic for the same source-destination pair. On the other hand, paths with low number of hops are expected to be more robust to failures and easy to control/monitor.*

In AntHocNet we make use, for mobile ad hoc networks, of a composite metric taking into account both traveling time and number of hops: $J(\mathcal{P}_{k\to d}) = T_{k\to d} + T_H H_{k\to d}$, where T_H is the time for one hop under unloaded conditions, and $H_{k\to d}$ is the number of hops in the path. In this way, the number of hops is conveniently converted to a time. If the time T_H cannot be assumed as the same for all links it should be calculated link by link (and possibly, in this case, the ant trip time should not include the propagation time, that would be automatically included in the T_H values).

```
procedure AntNet_ForwardAnt(source_node, destination_node)
  k ← source_node;
  hops_fw ← 0;
  V[hops_fw] ← k;      /* V = LIST OF VISITED NODES */
  T'[hops_fw] ← 0;     /* T' = LIST OF NODE-TO-NODE TRAVELING TIMES */
  while (k ≠ destination_node)
    t_arrival ← get_current_time();
    n ← select_next_hope_node(V, destination_node, T^k, L^k);   /* L^k = LINK QUEUES */
    wait_on_data_link_queue(k, n);
    cross_the_link(k, n);
    T_{k→n} ← get_current_time() − t_arrival;
    k ← n;
    if (k ∈ V)   /* CHECK IF THE ANT IS IN A LOOP AND REMOVE IT */
      hops_cycle ←get_cycle_length(k, V);
      hops_fw ← hops_fw − hops_cycle;
     else
      hops_fw ← hops_fw + 1;
      V[hops_fw] ← k;
      T'[hops_fw] ← T_{k→n};
    end if
  end while
  become_a_backward_ant(V, T');
end procedure

procedure AntNet_BackwardAnt(V, T')   /* INHERITS THE MEMORY FROM THE FORWARD ANT */
  k ← destination_node;
  hops_bw ← hops_fw;
  T ← 0;
  while (k ≠ source_node)
    hops_bw ← hops_bw − 1;
    n ← V[hops_bw];
    wait_on_high_priority_link_queue(k, n);
    cross_the_link(k, n);
    k ← n;
    for (i ← hops_bw + 1; i ≤ hops_fw; i ← i + 1)   /* UPDATES FOR ALL SUB-PATHS */
      δ ← V[i];
      T_{k→δ} ← T + T'[i];   /* INCREMENTAL SUM OF THE TRAVELING TIMES EXPERIENCED BY FwAnt_{s→d} */
      T ← T_{k→δ};
      if (T_{k→δ} ≤ I_sup(μ_δ, σ_δ) ∨ δ ≡ d)   /* T_{k→δ} IS A GOOD TIME, OR IS THE DESTINATION NODE */
        M_δ^k ← update_traffic_model(k, δ, T_{k→δ}, M_δ^k);
        r ← get_reinforcement(k, δ, T_{k→δ}, M_δ^k);
        T_δ^k ← update_pheromone_table(T_δ^k, r);
        R_δ^k ← update_data_routing_table(R_δ^k, T_δ^k);
      end if
    end for
  end while
end procedure
```

Algorithm 7.1: *C-like pseudo-code description of the forward and backward ant actions in AntNet. The actions of the whole set of forward and backward ants active on the network happen in a totally concurrent and distributed way. Each ant is a fully autonomous agent.*

The different role between queuing time and propagation time is actually well captured by Wedde et al. [443]: in their bee-inspired algorithm they assign the cost of a link according to a formula that separates the contribution due to propagation time from that due to queuing time. The rationale behind this choice consists in the fact that, when the network is experiencing a heavy load, queuing delay plays the primary role in defining the cost of a link, while in case of low load, is the propagation delay that plays a major role.

The main problem with the use of end-to-end delays consists in the fact that their absolute value T cannot be scored with respect to any precise *reference value*. That is, it is not possible to know exactly how good or how bad is the experienced time because the "optimal" traveling times, conditionally to the current traffic patterns, are unknown. In the jargon of machine learning: it is not possible to learn the routing policy through a process of supervised learning. A set of pairs of the type: $(\textit{network traffic condition}, T_{k \to d})$ for all $k, d \in \{1, 2, \ldots, N\}$ and for most of the possible network traffic situations, is not available under any realistic assumption. Moreover, as shown with Example 7.1, a same value of a $T_{k \to d}$ can be judged good or bad according to the changing traffic load.

Therefore, each value of T can only be associated to a *reinforcement*, advisory, signal, not to a precise, known, error measure. This gives rise to the same credit assignment problem encountered in field of reinforcement learning. This is the reason why in the previous sections we have used the term *reinforcement* to indicate $\Delta\tau$, the value r which is used to increase the pheromone variables, that is, the goodness of a path according to the experienced traveling time. This is also the reason why an actor-critic architecture was used for the assignment of the reinforcement values. Actor-critic architectures have been developed in the field of reinforcement learning, and have shown as particularly effective in the cases in which the explicit learning of a stochastic policy turns out to be useful, as it happens in the routing case, and, more in general, in non-Markov cases.

It is evident that it is quite hard to define robust reinforcement values. At the same time, it is also evident that the overall performance of the algorithm can critically depend on the way these values are defined.

The value of r should be assigned according to the following facts: (i) paths have to receive an increment in their selection probability *proportional* to their goodness, (ii) goodness is a *relative measure*, depends on the traffic conditions, and can be estimated by means of the models $\mathcal{M}$, (iii) models must be *robust* with respect to small traffic fluctuations. Uncontrolled oscillations in the routing tables are one of the main problems in adaptive routing [441]. It is customary to define an appropriate trade-off between stability and adaptivity in order to obtain good performance.

The following two subsections discusses two major ways of assigning the values of r in the perspective of the facts (i-iii).

7.1.4.1 Constant reinforcements

The simplest strategy is to set the value of r as a constant:

$$r = C, \quad C \in (0, 1], \tag{7.11}$$

that is, independently from the experienced trip time: every path followed by a forward ant is rewarded in the same way. In this case, what is at work is the *implicit* reinforcement mechanism due to the differentiation in the ant arrival rates discussed in Remark 7.10. Ants traveling along faster paths will arrive at a higher rate than other ants, hence their paths will both receive a higher cumulative reward and have the capacity to attract new generated ants.

The obvious problem with this approach is the fact that, although the single ant following a longer path arrives with some delay, nevertheless it has the same effect on the routing tables as the single ant which followed a shorter path. The *frequency of ant generation* in this case plays a

critical role to allow the effective discrimination between good and bad paths. If the frequency is low with respect the traffic variations the use of constant reinforcements is not expected to give god results.

In the experiments that we have ran, using the a frequency of more than 3 ants per second at each node, the algorithm showed moderately good performance. These results suggest that the implicit component of the algorithm based on the ant arrival rate plays a significant role. However, to compete with state-of-the-art algorithms, the available information about path costs has to be used (see also the discussion in Subsection 4.3.3 about the use of the implicit component in both distributed and non-distributed problems).

7.1.4.2 Adaptive reinforcements

In its general form r is a function of the ant's trip time T, and of the parameters of the local statistical model $\mathcal{M}$, that is, $r = r(T, \mathcal{M})$. We have tested several possible combinations of the values of T and $\mathcal{M}$. In the following the discussion is restricted to the functional form that has given the best experimental results and that has been used in all the experiments reported in the next chapter:

$$r = c_1 \left(\frac{W}{T} \right) + c_2 \left(\frac{I_{sup} - I_{inf}}{(I_{sup} - I_{inf}) + (T - I_{inf})} \right), \tag{7.12}$$

where, as usual, W is the best traveling time experienced by the ants traveling toward the destination d under consideration over the last observation window of size w samples. On the other hand, I_{sup} and I_{inf} are estimates of the limits of an *approximate confidence interval* for μ:

$$\begin{aligned} I_{inf} &= W \\ I_{sup} &= \mu + z(\sigma/\sqrt{w}), \quad \text{with} z = 1/\sqrt{(1-\gamma)}, \end{aligned} \tag{7.13}$$

where γ is the confidence coefficient. The expression in Equation 7.13 is obtained by using the Tchebycheff inequality that allows the definition of a confidence interval for a random variable following a whatever distribution [345]. Usually, for specific probability densities the Tchebycheff bound is too high, but here it is used because: (i) no specific assumptions on the characteristics of the distribution of μ can be easily made, (ii) only a raw estimate of the confidence interval is actually requested. There is some level of arbitrariness in the computation of the confidence interval, because it is defined in an asymmetric way and both μ and σ are not arithmetic estimates. The asymmetry of the interval is due to the fact that, in principle, the trip time is not superiorly bounded (in reality, it is bounded by the maximum time-to-live associated to an ant, but this bound is not of practical interest), while it is inferiorly bounded (by the traveling time corresponding to the path which is the shortest in conditions of absence of traffic).

The first term in Equation 7.12 evaluates the how good is the currently experienced traveling time T with respect to the best traveling time observed over the current observation window. This term is *corrected* by the second one, that evaluates how far the value T is from I_{inf} in relation to the extension of the confidence interval, that is, considering the stability in the latest trip times. The coefficients c_1 and c_2 weight the importance of each term. The first term is the most important one, while the second term plays the role of a correction. In the current implementation of the algorithm $c_1 = 0.7$ and $c_2 = 0.3$. It has experimentally observed that c_2 should not be too large (0.35 is a reasonable upper limit), otherwise performance starts to degrade appreciably, also considering the approximate nature of the terms it weights in 7.13. The behavior of the algorithm is quite stable for c_2 values in the range 0.15 to 0.35, but setting c_2 below 0.15 slightly degrades performance. The algorithm seems to be very robust to changes

in γ, which defines the confidence level. The best results have been obtained for values of the confidence coefficient γ in the range 0.6÷0.8.[15]

The denominator of the second term in Equation 7.12 can become equal to zero, in the rather pathological cases in which $I_{sup} = I_{inf} = T$ and $T = 2I_{inf} - I_{sup}$. In these cases the term weighted by c_2 is simply not taken into account.

In order to prevent the pheromone values going to 1, or to make too large jumps, r is actually saturated at the value 0.9. Finally, the value of r is *squashed* by means of a function $s(x)$:

$$s(x) = \left(1 + exp\left(\frac{a}{x|\mathcal{N}_k|}\right)\right)^{-1}, \qquad x \in (0, 1], \quad a \in \mathbb{R}^+, \tag{7.14}$$

$$r = \frac{s(r)}{s(1)}. \tag{7.15}$$

By squashing the value of r, the upper scale of the r values is expanded, such that the sensitivity of the system in the case of good (high) values of r is increased. On the contrary, the lower scale is compressed, bad (near to 0) r values are saturated around 0. In such a way more emphasis is put on good results. The coefficient $a/|\mathcal{N}_k|$ determines a parametric dependence of

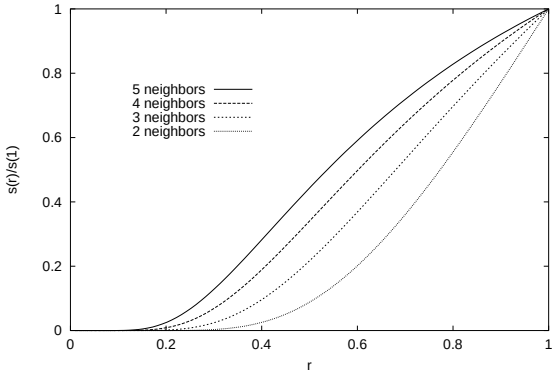

Figure 7.13: *Squash functions with a different $a/|\mathcal{N}_k|$.*

the squashed reinforcement value on the number $|\mathcal{N}_k|$ of neighbors of the node k: the greater the number of neighbors, the higher the reinforcement (see Figure 7.13). The rationale behind this choice is the need to have similar effects independently of the number of neighbor nodes. The number of neighbors has, in fact, an effect at the moment of the normalization of the probabilistic entries of the routing table. A more uniform distribution of the probability values is expected, in general, with an higher number of neighbors, due to the fact that several alternative paths might result similar. To contrast this tendency, the coefficient $a/|\mathcal{N}_k|$ tries to slightly amplify the possible differences.

REMARK 7.13 (ROBUSTNESS TO PARAMETER SETTING): *In spite of the fact that many parameters are involved, the algorithm has experimentally proved to be very robust to parameter settings. All the different experiments presented in the next chapter have been realized using the same set of values for the*

[15] For larger confidence levels, the confidence interval becomes in some sense too wide, given the characteristics of the Tchebycheff bound. In order to understand this fact, let us admit that the trip times within each time window are normally distributed. In this case, an expression for the confidence interval similar to the Tchebycheff one, but more precise and with a slightly different meaning for the term z, can be written. In particular, setting up a confidence level of 0.95 implies $z \approx 2$. The same value of z for the more general, non Gaussian, case expressed by the Tchebycheff inequality, implies a confidence level of the 65%, which is also the same used in the experiments reported in Chapter 8.

parameters, in spite of the significant differences among the different problem scenarios. Clearly, the performance could have been improved by mean of a fine tuning of the parameters, choosing different values for the different problem instances. This tuning process has purposely not been carried out. The target was, in fact, the design of an algorithm able to show a robust behavior under a variety of completely different characteristics for traffic and networks. Such a robustness cannot be clearly obtained if an algorithm is too sensitive to the values assigned to its internal parameters.

7.2 AntNet-FA: improving AntNet using faster ants

In AntNet, forward ants make use of the same queues that are used by data packets. In this way they behave like data packets and experience the same traveling time that a data packet would experience. In this sense, forward ants faithfully *simulate* data packets. The problem with this approach is that, in case of congestion along the path that is being followed, it will take a significantly long time to the forward ant to reach its destination. This fact will have two major consequences: the value of the reinforcement will be quite small, and the ant will be delayed in reinforcing its path. On the contrary, ants which have followed less congested paths will update earlier and with higher reinforcement values the routing tables along their paths, strongly biasing in this way the choices of subsequent ants and data packets towards these paths. What is wrong in this picture? Let us consider the following scenario: a forward ant has been moving so far on a very good path, but at the current node it meets a sudden and temporary traffic-jam situation. The forward ant is then forced to wait for some appreciably long time. When the ant can finally leave the jammed node, the cause of the sudden traffic-jam (e.g., a short-lived bursty session) might be over, but the ant has been hopelessly slowed down and the whole path will not receive the reward it might actually deserve. In an even worse scenario, a path gets congested "behind" the ant. The ant arrives quickly but its picture of the path in terms of traveling time is not anymore conformal to the reality. This can happen with a probability which is somehow increasing with the increase of the number of hops in the path.

Therefore, the strategy of making the forward ants wait in the same, low priority, queues as data packets, can be such that the previously acquired view of the traffic status becomes completely out-of-date at the time the routing tables are updated by backward ants. Moreover, the effect of the implicit path evaluation plays in a sense a quite important role with respect to the explicit evaluation. To avoid these problems we modified AntNet's design and defined *AntNet-FA* [124, 113], a new algorithm based on the following property:

DEFINITION 7.1 (MAIN CHARACTERISTICS OF ANTNET-FA): *Forward ants make use of* high-priority queues *as backward ants do, while backward ants update the routing tables at the visited nodes using local* estimates *of the ant traveling time, and not anymore the value of the time directly experienced by the forward ant.*

According to these modifications, forward ants do not simulate anymore data packets in a mechanistic way. The traveling time they experience does not realistically reflect a possible traveling time for a data packet. In order to have at hand a value of T which is a realistic estimate of the traveling time that a data packet following that path would experience, a model for the depletion dynamics of link queues is used:

DEFINITION 7.2 (MODEL $\mathcal{D}$ FOR THE DEPLETION DYNAMIC OF LINK QUEUES): *The depletion dynamics of the link queues is modeled in the terms of a uniform and constant process $\mathcal{D}$ which depends only on the link bandwidth b_l. That is, both the sender process and the consuming process at the other side of the link do not slow down the depletion of the queue. According to these assumptions, the transmission of the packets waiting on the link queue $\mathcal{L}_l^k$ to the connected neighbor l is completed after a time interval*

defined by:

$$\mathcal{D}_l^k(q_l;\ b_l, d_l) = d_l + \frac{q_l}{b_l}, \tag{7.16}$$

where q_l is the total number of bits contained in the packets waiting in the queue $\mathcal{L}_l^k$, while b_l and d_l are respectively the link bandwidth (bits/sec) and propagation delay (sec).

According to this depletion model, the estimate of the time $T_{k\to l}$ that it would take to a data packet of the same size s_a of the forward ant to reach the neighbor l from which the backward ant is coming from, is computed as:

$$T_{k\to l} = \mathcal{D}_l^k(q_l + s_a;\ b_l, d_l) \tag{7.17}$$

The value of $T_{k\to m}$ for a node m on the ant path V is computed, as the sum of the time for for each hop:

$$T_{k\to m} = \sum_{i=\{v_m, v_{m-1}, \ldots, v_{k+1}\}} T_{i-1\to i}\ , \qquad \{v_k, v_{k+1}, \ldots, v_{m-1}, v_m\} = V_{k\to m} \tag{7.18}$$

Compared to the previous model, where the ants were slowly "walking" over the available, often jammed, "roads", here the forward ants can be pictorially thought as *flying* over the data queues to quickly reach the target destination. Because of this visual metaphor, the new algorithm is called *AntNet-FA*, where the acronym FA stands for *flying ants*!

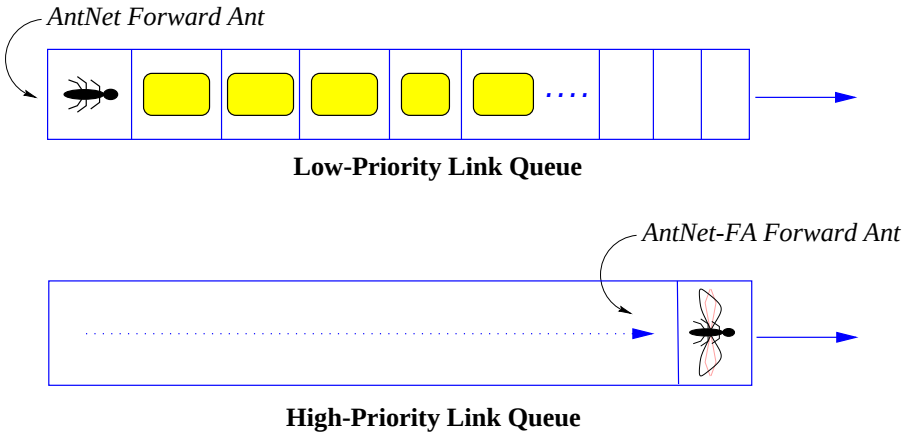

Figure 7.14: *Forward ants in AntNet vs. forward (flying) ants in AntNet-FA*

REMARK 7.14 (ANTNET-FA VS. ANTNET): *As it will be shown from the experimental results, AntNet-FA outperforms AntNet. The difference in performance becomes more and more evident with the increase of the number of nodes in the network, that is, with the increase in the average hop length of the paths.*

An additional advantage when using AntNet-FA consists in the fact that forward ants are *lighter* agents with respect to the AntNet agents, since they do not need to carry in their memory information about the experienced hopping delays. This fact might result particularly effective when either the number of nodes in the network grows significantly or the link bandwidth is quite limited. A further positive aspect of AntNet-FA consists in the fact that, since the time interval between the moment the ant is launched and it comes back is possibly very short (it depends only on the number of hops of the path and not really on the congestion on it), AntNet-FA can be used in a *connection-oriented* architecture to effectively setup virtual circuits *on-demand*.

This is not really feasible in AntNet since setup times would be not a priori bounded.[16] On the other hand, a sort of negative aspect consists in the fact that the effectiveness of the mechanism of the implicit reinforcement due to the ant arrival rate is much reduced.

Clearly, the reliability of the link depletion model $\mathcal{L}_l^k$ is important for the proper functioning of AntNet-FA. The model adopted here is at the same time very simple and light from computational and memory point of view. According to the excellent experimental results obtained with such a simple mode, the definition of a more robust but necessarily more complex model does not seem as really necessary, at least for the considered case of wired networks.[17]

7.3 Ant Colony Routing (ACR): ant-like and learning agents for an autonomic network routing framework

The routing algorithms described so far have a flat organization and uniform structure: all the ants have the same characteristics and are at the same hierarchical level, while nodes are just seen as the repository of the data structures used by the ants. Moreover, ants are only generated in a proactive way following a periodic scheduling regulated by a fixed frequency. While these characteristics might match quite well the considered case of wired best-effort networks, it is clear that effective implementations of ACO algorithms in the case of more complex and highly dynamic network scenarios, in which several classes of events must be dealt with, require higher levels of adaptivity and heterogeneity. This is for instance the case of networks provisioning QoS, and networks with frequent topological modifications and limited and constrained bandwidth (i.e., mobile ad hoc networks).

These facts are taken into account in the *Ant Colony Routing (ACR)* framework, which extends and generalizes the ideas that have guided the design of AntNet and AntNet-FA according to the ACO' specifications.

ACR defines a high-level distributed control architecture that specializes the general ACO's ideas to the case of network routing and at the same time provides a generalization of these same ideas in the direction of integrating explicit learning and adaptive components into the design of ACO algorithms. ACR is a collection of ideas aimed at defining a general framework of reference for the design of fully distributed and adaptive systems for network control tasks. In the same way ACO has been defined as an ant-inspired meta-heuristic for generic combinatorial problems, ACR can be seen as the equivalent ACO-inspired meta-architecture for network routing problems (and, more in general, for distributed control problems).

ACR answers to the question: "Which are the ingredients of a fully adaptive network routing (control) algorithm based on the use of ant-like agents in the sense indicated by ACO?" ACR, inherits all the essential characteristics of AntNet-FA (and of ACO in general), but at the same time introduces new basic types of agents, defines their hierarchical relationships, and points out the general characteristics and strategies that are expected to be part of a distributed control architecture that makes use of the ACO's ant-like agents as well as learning agents. Instances of ant-based algorithms for specific network scenarios are expected to include a subset of the general ACR components, as well as to instantiate them according to the specific characteristics of the scenario at hand.

[16] Precisely according to this possibility of using AntNet-FA in a connection-oriented (and/or QoS) architecture, in its original definition given in [124] AntNet-FA is actually called AntNet-CO, while AntNet is called AntNet-CL, where CO and CL stand respectively for connection-oriented and connection-less.

[17] For instance, AntHocNet adopts a similar scheme for the case of mobile ad hoc networks. However, the depletion model is more complex and adaptive, since it has to take into account the transmission collision happening at the MAC layer due to the fact that for these networks there is only one shared wireless channel. Therefore, the model is based on an adaptive estimate of the time to access the channel, which depends, in turn, on the number of neighbors and on their transmission requests.

```
procedure AntNet-FA_ForwardAnt(source_node, destination_node)
  k ← source_node;
  hops_fw ← 0;
  V[hops_fw] ← k;      /* V = LIST OF VISITED NODES */
  while (k ≠ destination_node)
    n ← select_next_hope_node(k, destination_node, T^k, L^k);      /* L^k = LINK QUEUES */
    wait_on_high_priority_link_queue(k, n);
    cross_the_link(k, n);
    k ← n;
    if (k ∈ V)    /* CHECK IF THE ANT IS IN A LOOP AND REMOVE IT */
      hops_cycle ←get_cycle_length(k, V);
      hops_fw ← hops_fw − hops_cycle;
    else
      hops_fw ← hops_fw + 1;
      V[hops_fw] ← k;
    end if
  end while
  become_a_backward_ant(V);
end procedure

procedure AntNet-FA_BackwardAnt(V)    /* INHERITS THE MEMORY FROM THE FORWARD ANT */
  k ← destination_node;
  hops_bw ← hops_fw;
  T ← 0;
  while (k ≠ source_node)
    hops_bw ← hops_bw − 1;
    n ← V[hops_bw];
    wait_on_high_priority_link_queue(k, n);
    cross_the_link(k, n);
    T'[hops_bw + 1] ← d_k + (q_k + s_a)/b_k;      /* TIME TO TRAVEL FROM n TO k ESTIMATED FROM D_n^k */
    k ← n;
    for (i ← hops_bw + 1; i ≤ hops_fw; i ← i + 1)    /* UPDATES FOR ALL SUB-PATHS */
      δ ← V[i];
      T_{k→δ} ← T + T'[i];
      if (T_{k→δ} ≤ I_sup(μ_δ, σ_δ) ∨ δ ≡ d)    /* T_{k→δ} IS A GOOD TIME, OR IS THE DESTINATION NODE */
        M_δ^k ← update_traffic_model(k, δ, T_{k→δ}, M_δ^k);
        r ← get_reinforcement(k, δ, T_{k→δ}, M_δ^k);
        T_δ^k ← update_ant_routing_table(T_δ^k, r);
        R_δ^k ← update_data_routing_table(R_δ^k, T_δ^k);
      end if
      T ← T_{k→δ};
    end for
  end while
end procedure
```

Algorithm 7.2: *C-like pseudo-code description of the forward and backward ants in AntNet-FA. The actions of the whole set of forward and backward ants active on the network happen in a totally concurrent and distributed way. Every ant is autonomous, acting independently from all the other ants. This pseudo-code should be compared to that of Algorithm 7.1.*

The ACR framework introduces a *hierarchical organization* into the previous schemes. The mobile ant-like agents are now seen as ancillary to and under the direct control of *node agents* that are the true controllers of the network. Their tasks consists in the *adaptive learning of pheromone tables*, such that these tables can be in turn used by the local *stochastic control policy*. The generation of the ant agents is expected to be *adaptive* and to follow both *proactive* and *reactive* strategies. Moreover, the ant agents do not need anymore to have all the same characteristics. On the contrary, *diversity* at the level of both the value of their parameters and of their actions can play and important role to cope with the intrinsic non-stationarity and stochasticity of network environments.

REMARK 7.15 (GENERAL CHARACTERISTICS OF ACR): *ACR defines the routing system as a distributed society of both* static *and* mobile agents*. The static agents, called* node managers*, are connected to the nodes and are involved in a continual process of* adaptive learning *of* pheromone tables*, that is, of arrays of variables holding statistical estimates of the goodness of the different control actions locally available. The control actions are expected to be issued on the basis of the application of* stochastic decision policies *relying on the local pheromone values. The mobile, ant-like agents, play the role of either* active perceptions *or* effectors *for the static agents, and are generated proactively and on-demand. Node managers are expected to* self-tune *their internal parameters in order to adaptively regulate the generation and the characteristics of these ancillary agents. In this way they are involved in two levels of learning activities. The active perceptions carry out* exploratory tasks across the network *and gather the collected non-local information back to the node managers. The effectors carry out ad hoc tasks, and base their actions on pre-compiled deterministic plans (opposite to the active perceptions, that make use of stochastic decisions and adapt to local conditions).*

Using the language of the ant metaphor, passing from AntNet to ACR equals to moving from a colony of ants to a society of multiple ant colonies*, with each colony being an autonomic element devoted to manage the activities of a single network node in social agreement with all the other colonies.*

The ACR's *society of heterogeneous agents* well matches forthcoming scenarios, in which networks will be likely populated by node agents and mobile agents. With these lasts carrying their own code and specifications, moving across the networks, acting and interacting with other mobile or node agents. They will be able to adapt themselves to new situations, possibly learn from past experience, replicate if necessary or remove themselves if not useful anymore, cooperate and/or compete with other agents, and so on.[18] This picture will be likely closer and closer to reality with the networks becoming more and more *active* (e.g., [420, 438]). That is, such that packets will be able to carry their own execution or description code and all the network nodes will be able to perform, *as normal status of operations*, customized computations on the packets passing through them. Nowadays networks are more like a collection of high speed switching boxes, but in the future those boxes will be replaced by "network computers" and the whole network will likely appear as a *huge multiprocessor system*.

The organization envisaged by ACR is in accordance with the recent vision of *autonomic computing* [250], that is, of computing systems that can manage themselves given high-level objectives from the administrators. ACR defines the generalities of a multi-agent society based on the integration of the ACO's philosophy with ideas from the domain of reinforcement learning, with the aim of providing a meta-architecture of reference for the design and implementation of fully *autonomic routing systems*. In fact, the node managers are expected to proactively monitor, experiment with, and tune their own parameters in order to learn to issue effective decisions in

[18] Agent technology is receiving an ever increasing attention from telecommunication system engineers, researchers and practitioners. Agent modeling account in a straightforward way for the modularity of network components and, more importantly, they overcome the typical client-server model of communication since they can carry their own execution code and they can be used as autonomous component of a global distributed and fault-tolerant system (among a number of ever increasing works, see for example [220, 259, 206, 424, 265, 432]).

response to the different possible events and traffic scenarios. Each node manager is equipped with a repertoire of basic strategies for control and monitoring actions. Adaptively and autonomously the node manager optimizes the parameters of these strategies acting in accordance to social agreements with the other node managers. Each node manager is a fully autonomic element active in self-tuning and optimization by learning. On the other hand, the whole system of node managers give raise to a fully autonomic routing system aimed at providing a traffic-adaptive routing policy which is optimized at the network level.

7.3.1 The architecture

In the ACR framework the network is under the control of a system of distributed and adaptive agents θ_k, one for each node k of the network. Each controller θ_k, also called a *node manager*, is static (i.e., non-mobile) and its internal status is defined by the local values of *pheromone tables* $\mathcal{T}^{\theta_k}$ (and also of *heuristic arrays*), as well as by other additional data structures ad hoc for the problem at hand. Each entry in the pheromone array is associated to a different control action locally available to the controller, and represents a statistical estimate of the goodness (e.g., utility, profit, cost-to-go, etc.) of taking that action. The controller adopts a *stochastic decision policy* which is parametrized by the pheromone and heuristic variables. The target of each controller is to locally, and in some sense independently, learn a decision policy in terms of pheromone variables such that the distributed society of controllers can jointly learn a decision policy optimizing some *global performance*. Each controller is expected to learn good pheromone values by *observing* the network environment, as well as the effect of its decisions on it, and making use of these observations to *adaptively* change the pheromone values as well as other parameters regulating its monitoring and control actions. Observations can be both local and non-local. Ant-like agents are responsible for carrying out non-local observations and bringing back useful information to the node managers. At each time the node manager must decide which kind of "action" (local observation, remote perception, data routing, etc.) looks more appropriate according to current status, requirements, and estimated costs/benefits.[19] The local decision must be issued taking into account the fact that the set of all node managers act concurrently and without any form of global coordination. That is, the node managers must act socially and possibly cooperate in order to obtain positive synergy.

More specifically, the control architecture defined by the ACR framework is designed in the terms of a hierarchical society composed of the following classes of agents:

Node managers: adaptive learning agents statically bounded to the nodes (one for each node). They are all situated at the same hierarchical level. Each node manager autonomously control the node activities (e.g., data routing and performance monitoring) on the basis of stochastic decision policies whose parameters, in the form of locally maintained pheromone variables, are the main object of learning.

Active perceptions: mobile ant-like agents that are explicitly generated by the node managers to collect non-local information (e.g., discovery of new routing paths). They have characteristics of local adaptivity, might learn at individual level, and make use of stochastic decision policies.

Effectors: mobile ant-like agents generated by the node managers to carry out pre-compiled and highly specialized tasks (e.g., reserve and free network resources). They are expected to adopt deterministic policies.

[19] See for instance [219] for general discussions on decision systems that have to repeatedly decide between control and observation actions.

The dynamics of the whole network control system is driven by the node managers, while both active perceptions and effector agents are generated in order to support node managers activities. Generation dynamics can be based on both *proactive* and *on-demand* strategies in order to deal effectively with the characteristics of the network at hand.

The terms "perceptions" and "effectors" suggest the interpretation of the node managers as *static robotic agents* acting in the physically constrained network environment. On the other hand, using the language of the ant metaphor, each node manager can be seen in the terms of a single colony of ant-like agents. Learning processes happen as usual at the level of the colony but are now concentrated on a node, while the ant agents, the active perceptions, are adaptively generated in order to collect specific information or to reserve/free resources for the colony. The ensemble of all the colonies constitutes a *society of colonies*, that must find the right level of cooperation in order to obtain effective synergistic behaviors. Under this new point of view, a greater emphasis is put on the activities at the nodes, which become the main actors of the whole system, with learning happening at two levels, both at the level of the single nodes (the colonies) and at the level of the ensemble of all the nodes (the colonies' society). In the following we will use interchangeably the terms node manager and colony manager. Also the terms ant-like agent, perception ant (effector ant), and active perception (effector agent), will be used interchangeably.

7.3.1.1 Node managers

The first task of a node manager consists in gathering data for building and continually updating pheromone variables. Node managers can *passively* observe the local dynamics of traffic flows and packet queues. However, this information might be in general insufficient to effectively accomplish the node activities because of its intrinsic incompleteness. Therefore, node managers need also to expand their "sensory field" with the adaptive generation of *active perceptions agents* $\xi^{\theta_k}(t)$, that is, ant-like mobile agents that leave the node at time t, carry out some network exploration activities, and gather back to the colony manager the information it requires after some possibly short time delay. The term "active perception" is referred to both the facts that a non-local perceptual act is explicitly issued and that each of these perceptual acts are actually generated with a possibly different set of parameters in order to precisely get information about a specific area of the network by using specific parameters concerning the path-following strategy. That is, in general, $\xi^{\theta_k}(t_1) \neq \xi^{\theta_k}(t_2)$, for $t_1 \neq t_2$. This means that a node manager has a high degree of control over the type of information that must be collected by its remote sensors, the ant-like agents.

The design of node managers involves three main general aspects. That is, the definition of the strategies for: (i) the scheduling of the active perceptions, (ii) the definition of the internal characteristics for the generated perceptions, and (iii) the use of the gathered information in order to learn effective decision policies and tune other internal parameters regulating for instance the proactive scheduling of active perceptions. It is clear that a wide range of possible strategies exists, depending on the specific characteristics of the problem at hand. Nevertheless, some general strategies can be readily identified. Each of the three subsections that follow considers one of the aspects (i-iii) and discusses in very general terms possible problems and solutions.

Scheduling of active perceptions

The set $\Theta = \{\theta_1, \theta_2, \ldots, \theta_N\}$ of all node managers virtually constitutes an *agent society*, in the sense that node managers must behave socially since all of them equally contribute to the global network performance. That is, generation of the perceptions must happen in order to safeguard the overall social welfare: each node must find the right tradeoff between gathering large amounts of information and not creating congestion that in turn would either have a negative impact on data routing or not allow the other node managers to gather the information that they

might need. It is apparent that the issue of the *scheduling of the ant-like agents* is central in ACR, as it is in all ACO algorithms since they are all based on repeated agent generation for information sampling.

REMARK 7.16 (REACTIVE AND PROACTIVE INFORMATION GATHERING): *In ACR the ant-like mobile agents can be generated by the node managers according to both reactive (on-demand) and proactive strategies. In particular, the proactive strategies are not necessarily based on a fixed frequency scheme but on the contrary are expected to be adaptive in order to find a good tradeoff between information gathering and congestion induced by control packets.*

Some general strategies for both the reactive and proactive generation of ant-like agents can be identified. The list that follows is aimed at clarifying in which general situations ant-like agents are "expected" to be generated and the general modalities of these generations. That is, we point out some common problems/situations and we suggest some general solutions. More concrete examples of mixing on-demand and proactive generation are provided by the descriptions of AntNet+SELA and AntHocNet.

On-demand generation of active perceptions. Four classes of events can automatically trigger the generation of new perception ants:

1. *Setup of a new application:* In the case of a best-effort connection-less network, at the start-up of a new application $A_{s \to d}$ a burst of $m \propto |\mathcal{N}(s)|$ new perception agents can be generated toward d by θ_s to collect up-to-date information about the paths d. In practice, active perceptions can be broadcast to the local set of neighbors, and then move toward d using pheromone information.

   In the case of connection-oriented and/or QoS networks, these *setup ants* play a more critical role. In fact, they have to actually find (and possibly reserve) the path(s) to allocate the application flow. A possible behavior of the perception agents in this case is described more in detail in the following referring to the solution envisaged in AntNet+SELA.

2. *Major change in the traffic patterns:* If the information associated to some newly acquired data seems to strongly disagree with previously learned information, new perceptions can be fruitfully generated in order to confirm or not the changes. For instance, if the traveling time $T_{s \to d}$ reported by an ant agent indicates a value much larger than the one estimated up to that moment going through the same next hop, then it might be worth to get a clearer understanding of the traffic situation along the path. In fact, the unexpected value could have been caused either by "wrong" choices made by the perception agent due to a high level of stochasticity in its decision policy, or by some new congestion along the path. If the involved next hop was among the best ones so far for d, then before either keeping trusting it or reducing its goodness, it can be worth to generate further perception agents with destination d through some of the other outgoing links in order to gather more extensive information.

   This behavior can be effectively seen in the terms of a local feedback loop between *monitoring* of the network performance and issuing of control actions according to the fact that the monitored performance is either poor or has significantly changed.

3. *Request for or notification from a new destination node:* According to the discussion in Subsection 7.1.3.1 and in particular in the Remark 7.6 of the same subsection, it is clear that in some cases a destination which is not yet present in the node routing table might be requested (or, equivalently, a new node entering the network might advertise its presence). In these cases perception agents need to be generated to

gather information about the new destination. This might be a rather common situation in mobile ad hoc networks. For instance, in AntHocNet, so-called reactive setup ants search for a new destination by following pheromone information when this is present at the current node, or being radio broadcast when no information is available.

4. *Topological failure:* After the failure of a local link, or, equivalently, the disappearing of a neighbor node, it might be necessary to generate perception agents in order to gather fresh information about those destinations that were reached through that link/node. This situation is quite common in networks with frequent topological alterations, like the mobile ad hoc ones. However, since ACR instances make use of multipath routing (because of the use of a stochastic decisions based on pheromone values), several equally good alternatives are expected to be always available. Such that it might be not strictly necessary to "recover" from the failure event since good "backup" paths are likely made available. Again, in AntHocNet, ants are generated only if the broken link was the choice of election to reach some of the used network destinations, and/or no other equally good alternatives seem to be locally available. When alternatives are available, some (effector) ant agents are however radio broadcast in order to notify neighbors about major changes in the routing table due to the broken link.

Proactive generation of active perceptions. A background flow ω_k of active perception agents is continually and independently generated by each node manager θ_k to proactively keep an updated view of the overall network status, and for the purpose of *maintenance* of the paths used to route current traffic sessions. That is, proactive perceptions serve to be ready for future traffic requests and to maintain and/or improve the quality of the current ones. For what concerns routing in particular, proactive perceptions allow to maintain for each destination a bundle of paths with an associated measure of quality in the form of pheromone value. Each path can be used either for multipath data spreading or as alternate path in case of failures or sudden congestion.

The frequency of the proactive background flow is in general expected to be *adaptive* in ACR instances, whereas it was heuristically assigned as a constant value in AntNet and AntNet-FA. While the experimental results reported in Figure 8.17 in Subsection 8.3.6 suggest that AntNet is quite robust with respect to the choice of the background frequency, they also show that, with an appropriate tuning that depends on the overall network scenario, performance can be appreciably improved. In general, answering the question "At which rate control/routing agents should be generated ?" is at the same time extremely difficult and of fundamental importance for any adaptive and social scheme. As pointed out before, it is necessary to find a good tradeoff between frequency in information gathering and generated congestion.

In principle, more control packets mean more precise and up-to-date information but also control-induced congestion. However, control overhead should not be measured on an absolute scale (as it is often done) but rather in relationship to the network performance that it allows to obtain. A control algorithm that makes use of more control packets has not to be seen in a negative way if those extra packets allow to obtain much better performance, and at the same time performance scales well with network size and transmission characteristics.[20] The use of adaptive generation schemes precisely address this problem,

[20] For instance, the generation of 3-4 control agents per second (more or less the same frequency used in the AntNet experiments) at each node of a backbone network with transmission links of more than 1 Gbits/sec cannot not be considered as a significant overhead. On the other hand, the same overhead might be not negligible in the case of networks with link bandwidths less than of 1 Mbits/sec or with shared wireless links.

in principle allowing to optimize over the time the generation of control packets versus the obtained performance.

Focusing on routing, a few major aspects have to be taken into account in the adaptive definition of ω_k: (i) the local transmission bandwidth, (ii) the minimal transmission bandwidth across the network, (iii) the input traffic profile, (iv) the relative load generated by routing traffic with respect to data traffic, (v) the current congestion profile. The value of the local transmission capacity, as well as that of the minimal transmission capacity over the network is a useful starting point for the definition of initial values and upper limits for ω_k. On the other hand, all the other aspects need to be estimated online through local and possibly also non-local measures.

In Algorithm 7.3 we report as an example a rather simple and generic rule-based scheme that could be used the adaptive setting of the local ω_k in a routing task. The procedure is based on measures (over an assigned time window) of the status of the local buffers and the behavior of the local throughput for both data and routing packets. That is, throughput and buffer lengths are seen as good local indicators of the relative impact of routing packets on data traffic. When some limit values for these indicators are reached (e.g., buffers too full, data throughput considerably slowed down by routing throughput, etc.) significant changes in ω_k are triggered. For non-limit values, or, more in general, for all those situations that are hard to understand in the sense of getting a clear picture of the causes, a conservative strategy is adopted: ω_k is either left unchanged or is slightly decreased. The general idea consists in adaptively changing the local generation frequency of ant-like agents according to the estimated impact that they have on data traffic. These local measures can be integrated with non-local ones carried by the mobile agents. For instance, active perceptions can also carry a measure of the maximal level of congestion encountered along the followed path. If several ants brings the indication of congested paths, then is clear that the local ω_k should be promptly decreased.

Algorithm 7.3 makes use of a certain number of constants that defines relative percentages. The values assigned to these percentages correspond to the amount of risk accepted a priori: for example, C_{th} expresses the maximum ratio between routing and data throughput that can be tolerated. The procedure of Algorithm 7.3 has been partially tested in AntNet. Results seem promising but further tuning and analysis are required. The main purpose of reporting it here consists in showing what we concretely think it might a good direction to follow for the adaptive definition of ω_k. Unfortunately, none of the number of AntNet implementations/modifications (see Section 7.4) have considered yet this fundamental issue. A much simpler but still adaptive scheme is adopted for instance in AntHocNet: proactive ant-like agents are generated according to the frequency of data packets. Every $1/\omega_k$ packets generated by application A_k a proactive ant is generated bounded for A_k's destination. In this way, the algorithm designer has only to decide which will be the *fraction* $1/\omega_k$ of the overall traffic which will due to control packets. The problem with this approach is that for bursty traffic sessions a number of routing packets are generated in a short time, and this can determine some unnecessary congestion.

Diversity in the internal characteristics of the active perceptions

While in AntNet and AntNet-FA all the ant-like agents are created with the same characteristics (apart for the destination), in ACR node managers are expected to generate each active perception with a set of parameters ad hoc for the task the mobile agent has been created for. For a routing task, considering an instance of ACR in which active perceptions are very similar to the AntNet-FA's ants, parameters that need to be set are, for example, the weight α, (Equation 7.7)

procedure ACR_LocalFrequencyOfProactiveAgentGeneration(ω)
  if $(routing\_throughput \geq C_{th} \cdot data\_throughput)$ LOCAL ROUTING THROUGHPUT SEEMS TOO HIGH...

    if $(waiting\_data\_bits \geq C^{-}_{buf} \cdot total\_size\_of\_local\_buffers)$ ... AND TOO MANY DATA PACKETS...
      $\Delta \leftarrow \dfrac{C^{-}_{buf} \cdot total\_size\_of\_local\_buffers}{waiting\_data\_bits}$; ... DECREASE THE FREQUENCY

    else if $(waiting\_routing\_bits > C_w \cdot waiting\_data\_bits)$ ... TOO MANY ROUTING PACKETS ...
      $\Delta \leftarrow \dfrac{C_w \cdot waiting\_data\_bits}{waiting\_routing\_bits}$; ... DECREASE THE FREQUENCY

    else if $\big((waiting\_data\_bits < C^{+}_{buf} \cdot total\_size\_of\_local\_buffer) \wedge$
      $(waiting\_data\_bits > C^{+}_{w} \cdot waiting\_routing\_bits) \vee$
      $(waiting\_routing\_bits < C^{+}_{wr} \cdot total\_size\_of\_local\_buffers)\big)$ ... QUEUES ALMOST EMPTY...
      $\Delta \leftarrow 1 + \dfrac{waiting\_data\_bits}{C^{+}_{buf} \cdot total\_size\_of\_local\_buffer}$; ... INCREASE THE FREQUENCY

    else ... THE SITUATION IS HARD TO EVALUATE...
      $\Delta \leftarrow 1$; ... CONSERVATIVE STRATEGY: DO NOT CHANGE
    end if

  else if $((waiting\_data\_bits \geq C^{-}_{buf} \cdot total\_size\_of\_local\_buffers) \wedge$
    $(waiting\_routing\_bits > C^{-}_{w} \cdot waiting\_data\_bits))$ ... ROUTING QUEUES OK BUT BUFFERS ARE FULL...
    $\Delta \leftarrow \dfrac{C^{-}_{w} \cdot waiting\_data\_bits}{waiting\_routing\_bits}$; ... DECREASE FREQUENCY

  else ... BOTH THROUGHPUT AND BUFFERS ARE OK...
    $\Delta \leftarrow 1 + \epsilon$; ... INCREASE FREQUENCY
  end if
  $\omega \leftarrow \omega \cdot \Delta$;
end procedure

Algorithm 7.3: *Pseudo-code for the example of adaptive setting of the frequency for the proactive generation of active perceptions by a node manager.* $C_{th}, C^{-}_{buf}, C^{+}_{buf}, C^{-}_{w}, C^{+}_{w}, C^{+}_{wr}, \epsilon$ *are assigned constants. For instance, a reasonable assignment of values might be as follows:* $C_{th} = 0.2,\ C^{-}_{buf} = 0.1,\ C^{+}_{buf} = 0.001,\ C^{-}_{w} = 0.2,\ C^{+}_{w} = 1.2,\ C^{+}_{wr} = 0.0001,\ \epsilon = 0.001$.

that defines the tradeoff at decision time between pheromone and current queue status, and the the coefficients c_1 and c_2 (formula 7.12) that at evaluation time regulate the weight of the window best with respect to that of the exponential averages. More in general, ACR perceptions are expected to make use of a parameter ε that defines the level of stochasticity in the routing decisions (see also, Section 3.3 and formula 7.10). According to this value, routing decisions are taken following a random proportional scheme or strategies more greedy toward the best next hop(s). For instance, at the setup time of a new application in a QoS network, on-demand perceptions searching for a QoS-feasible path are expected to behave more greedily than proactive perceptions exploring the network (see also the description of AntNet+SELA).

The probabilistic selection of (some of) the agent characteristics determines in the agent population a high level of *diversity*, which is in general seen as an effective feature in a multi-agent system operating in non-stationary environments, since it is expected to provide robustness, and favor global adaptability and task distribution.[21] The assignment of parameter values according to some sampling procedure avoids the assignment of critical crisp values to the algorithm parameters. On the contrary, it might be necessary to define only the parametric characteristics of statistical distributions (e.g., Gaussian) from which the parameter values are sampled from during the algorithm execution. In principle, also the characteristics of centrality and disper-

[21] At least this is what is commonly observed in Nature's systems (e.g., see [168, 3]). Some general studies on artificial multi-agent systems [10, 240, 239] further support this view.

sion of the sampling distributions can be in turn adapted online according to the monitored performance.

Learning strategies

Node managers can make use of any appropriate strategy to learn effective decision policies concerning data routing, monitoring, agent generation, parameters setting, etc. The pheromone variables are the main object of learning, since they play the role of parameters of the *stochastic decision policies*. Generally speaking, stochastic decision policies appears as more appropriate than deterministic ones to deal with the intrinsic characteristics of non-stationarity and distribution (i.e., hidden global state) of network environments. Moreover, in the case of routing tasks, the adoption for data routing of a stochastic policy based on pheromone values likely results in spreading data over *multiple paths* and automatically providing *load balancing*, that we see as positive features.

In the case of AntNet and AntNet-FA the adopted learning strategies are quite essential. An example of a more complex example learning architecture and strategy is given in AntNet+SELA, in which the node managers are *stochastic estimator learning automata* (SELA) [430, 330] and make use of link-state tables to provide guaranteed QoS routing in ATM networks. Stochastic learning automata, have been used in early times [331, 334] to provide fully distributed adaptive routing. Their main characteristics consists in the fact that they learn by *induction*: no data are exchanged among the controllers. They only monitor local traffic and try to get an understanding of the effectiveness of their routing choices. In AntNet+SELA the static inductive learning component is enhanced by using the ants as active perceptions in order to gather also non-local information to keep up-to-date the link-state routing table in the perspective of rapidly allocating resources for multipath QoS routing when necessary.

7.3.1.2 Active perceptions and effectors

Active perceptions are mobile agents explicitly generated by node managers for the purpose of non-local exploration and monitoring. Most of the general characteristics that can be attributed to active perception agents have been already described in the previous subsections. Here we complete the general picture pointing out some additional aspects.

- The model of interaction between perceptions and node managers envisaged by ACR is that typical of agent and *object-oriented programming*. Node managers and active perceptions are situated at a different hierarchical level. The active perception is subordinate to the node manager, that creates it for the purpose of collecting useful non-local information. Therefore, the hierarchically lower active perception should not directly modify the internal state of a node manager. That is, perceptions are not expected to directly modify the node manager's data structures (e.g., the pheromone table). Perceptions can only communicate to node managers the information they have collected along the path, while is the node manager's responsibility and decision to use or not this information to update its private data structures. There are at least three reasons to follow this behavior. First, by design the component for "intelligent" processing of sensory data is supposed to be the node manager and not its perception. Second, for *security reasons*: what if a competing and "malicious" network provider generates intruder perceptions carrying wrong information and evil objectives? ACR does not directly address this class of problems, but shielding a potential enemy from gaining access to the main node data structures is clearly an issue of critical importance. Third, a direct modification of the node data structures would be against the principles of information hiding at the basis of all the modern (object-oriented)

programming. The perception does not need to know about which kind of models, structures, data are used by the node managers. All it does need to know is the protocol to communicate its data to the node manager. In this way, data representations and learning algorithms internal to the nodes can either change very often or even being different from node to node. The only important thing from the point of view of the perceptions is that the local communication interface does not change.

- Active perceptions can have the peculiar characteristic of *replicating* (or, *proliferating*) when there is more than one single good alternative. For instance, let $F_{s \to d}$ be a "forward" perception moving from s to d. At the current node k, an AntNet ant would just pick-up one single link proportionally to its probability and move through it. Acting in this way, the ant discards other potentially promising links. On the other hand, if after the calculations of formula 7.7 there is no one single best link but a set $\{n_1, n_2, \ldots, n_b\}, b > 1$, of more or less equally good links, then the perception might replicate in b copies. Each copy can proceed on a different link $n_i, i = 1, \ldots, b$ to explore all these equivalent alternatives. Clearly, to avoid an uncontrolled proliferation of perception agents, some limits must be heuristically assigned on the allowed number of replicas.

  For instance, in AntHocNet, when no pheromone information for the ant destination is present at the current node, the ant is broadcast. This equals to considering all the links as equally good, such that the ant is transmitted to each possible neighbor. Clearly, if multiple broadcastings happen, the network can easily get flooded (and congested) by routing packets. Therefore, some heuristics are applied in order to kill the forward ant (e.g., ants that have proliferated from the same original ant and that arrive at a same node with no pheromone are destroyed if they have been already broadcast a certain number of times, or if their traveling time and number of hops do not score favorably with those of previous passed by ants).

- Active perceptions can be of *different type*, according to the different task they will be involved in. Clearly, different tasks usually require also different characteristic (e.g., level of stochasticity). While in AntNet and AntNet-FA all the ants are involved in the same task such that they all have the same type and characteristics, in both AntNet+SELA and AntHocNet there are several types of active perceptions, and the different types have different internal characteristics.

- There is not so much more to say about the *effector agents*. We have already introduced them when talking about AntHocNet's ants that notify pheromone updates after a link failure. Also in AntNet+SELA effector agents are used: to take care of allocation and deallocation of QoS resources. In general effectors are "blind" *executor agents*, used to carry out tasks whose specifications have been completely defined at the level of the generating node manager.

7.3.2 Two additional examples of Ant Colony Routing algorithms

Both AntNet and AntNet-FA can be seen as instances of the more general ideas of the ACR framework. On the other hand, in both AntNet and AntNet-FA all the ants are generated according to the same fixed proactive scheme, have the same characteristics and behavior, and the notion of node manager has not been made explicit, such that there is little "intelligence" at the nodes. In a sense, AntNet and AntNet-FA results in a rather natural way from the general ACO's guidelines and from the adaptation of the main ideas of previous ACO implementations for static combinatorial problems. Even though these characteristics might be a quite good match for the considered cases of wired best-effort networks, it is apparent that smarter node

managers, as well as increased levels of heterogeneity and adaptivity, might be useful if not necessary components to deal effectively with network scenarios that are either highly dynamic (e.g., mobile ad hoc networks) or include some forms of QoS provisioning. Therefore, in order to show how the ACR's ideas find their natural application in such complex and highly dynamic scenarios (or, equivalently, how the basic AntNet and AntNet-FA design can be extended and adapted to deal with the increased complexity), we briefly describe in the two next subsections two additional routing algorithms: *AntNet+SELA* [130] and *AntHocNet* [126, 154, 127].

AntNet+SELA is a intended for guaranteed QoS routing in ATM networks, while AntHocNet is for routing in mobile ad hoc networks. AntNet+SELA and AntHocNet will be not described in all their details. In fact, in order to properly discuss all the components of these algorithms, an in-depth introduction to both QoS and mobile ad hoc networks issues is required. However, this would likely require an additional chapter, that would make this thesis unnecessarily too long. Moreover, AntNet+SELA has never been fully tested, such that it is more a model than a real implementation, while AntHocNet is still under intensive development, and only preliminary, even if extremely encouraging, results are at the moment available (they will be shortly reported in the next chapter).[22]

7.3.2.1 AntNet+SELA: QoS routing in ATM networks

The transmission capacity of current network technology allows to support multiple classes of network services associated to different QoS requirements. QoS routing is the first, essential step toward achieving QoS guarantees. It serve to identify one or more paths that meet the QoS requirements of current traffic sessions while possibly providing at the same time efficient utilization of the network resources in order to satisfy the QoS requests of also future traffic sessions. When a new user application arrives and requires some specific network resources, the local connection admission control (CAC) component makes use of the available routing information to evaluate at which degree the QoS requirements can be satisfied and to decide if the session can be accepted or not. Several different general models (e.g., IntServ, DiffServ, MPLS, ATM) have been proposed so far to deal with the several issues involved in this general picture: reservation vs. non-reservation of the resources, classes of type of provided services, deterministic vs. statistical guarantees, mechanisms for evaluation (and allocation) of the available resources, levels of negotiation between the user and the CAC component, rerouting vs. non-rerouting of the applications, use of multiple paths vs. single path, etc. (there is an extensive literature on QoS, good and quite comprehensive discussions can be found for example in [398, 285, 440]).

AntNet+SELA [130] focuses on the case of providing QoS in ATM networks for generic variable bit rate (VBR) traffic. For this class of networks virtual circuits can be established either per flow or per destination basis such that physical reservation of resources is possible. Therefore, statistically guaranteed quality of service can be provided.

In AntNet+SELA the node managers, that are directly responsible for both routing and admission decisions, are designed after the *stochastic estimator learning automata* (SELA) [430] used in the SELA-routing distributed system [8]. They make use of active perceptions to collect non-local information according to both on-demand and proactive generation schemes. The active perceptions are designed after the AntNet-FA ants. Also effectors agents are used, to tear down paths and to gather specific information about traffic profiles of running applications.

[22] More conclusive and comprehensive experiments about AntHocNet are expected to result from Ducatelle's doctoral work, as explained in Footnote 4.

Node managers in SELA-routing

In order to understand the overall behavior of the algorithm, is first necessary to briefly explain the characteristics of the SELA-routing system, in which each node manager is a stochastic learning automaton. In general terms, a stochastic learning automaton is an finite-state machine that interacts with a stochastic environment trying to learn the optimal action the environment offers. The automaton chooses an action according to an output function and a vector of probabilities scoring the goodness of every possible action. After executing the action, it receives a reward/feedback signal from the environment and uses the signal to update a statistical model (e.g., a moving average of the received rewards and of the last time an action has been selected) of the expected feedback associated to every possible action. The actions are then sorted according to the new estimates and the vector of probabilities is updated increasing the probability of the action with the new best estimated reward and lowering the probabilities for all the other actions. In the specific case of SELA-routing, the node managers make use of a *link-state* database and *offline* find the k-minimum-hop paths $\mathcal{P}_1, \ldots, \mathcal{P}_k$ for each destination. These k paths are the set of actions available to the automaton. The behavior of the algorithm is then as follows:

Arrival of a new application: Whenever a new application asks for a QoS connection, the application must communicate to the local node manager: (i) its traffic profile in terms of peak cell rate, mean idle and burst periods, (ii) its QoS requests in terms of bandwidth, delay, delay jitter, and loss rate.

Estimate of link utilization: The application's characteristics are input to a fluid-flow approximation model to estimate the expected utilization u_i for each link i on the k possible paths.

Environment feedback: For each possible action (path with n hops) the feedback from the environment is computed as: $a_j = M - \sum_{i=1,u_i \in \mathcal{P}_j}^{n} u_i$, where M is a constant representing the maximum number of hops that can be admitted in a path, $u_i \in [0,1]$ and $u_i = 1$ if u_i is greater than a threshold u_{trunk} defined according to some trunk-reservation strategy.

Path selection: Order the actions according to the estimated feedback and select the action a_j with the best estimated feedback: $a_j > a_i,\ \forall i, i \neq j$ (ties are resolved at random). If a_j is a minimum-hop path and meets the QoS requirements then the path is accepted. Otherwise, if a_j is not a minimum-hop path but meets the QoS requirements and $\sum_{i=1,u_i \in \mathcal{P}_j}^{n} u_i < u_{trunk}$, then the path is accepted. If none of these two sets of conditions are met, the next path in the a's sequence is considered until an acceptable path is found and the application can start. If none of the paths meets the requirements, the application is rejected.

AntNet+SELA: description of the algorithm

In AntNet+SELA the node managers make use of ant-like agents in order to proactively update their link-state database, take routing decision using fresh information about the candidate paths, and split and reroute the applications if useful/necessary. In particular, node managers make use of two sets of active perceptions. The perceptions in one set behave exactly like AntNet-FA ants, and are aimed at proactively building and maintaining pheromone tables for the perceptions (ants) in the second set, which are reactively generated at the setup of a new application. These perceptions make use of the pheromone tables either to probe the paths suggested for routing by the node manager or to discover other and possibly better paths. Node managers make also use of effector agents, to gather information about the congestion status over the allocated paths in order to update the parameters of fluid-flow model used by node managers and/or to possibly reroute them. More in specific, the overall behavior of AntNet+SELA is as follows:

- Perception agents (hereafter called *proactive ants*) are generated from every node manager according to a proactive scheme. They behave like AntNet-FA ants, searching for paths and updating the pheromone tables that are going to be used for routing also by other perceptions. That is, the routing information used by the perception agents is different from that used to route data, which is contained in the link-state tables of the node managers. In this way, network exploration and routing of data traffic follow different dynamics.
- At the arrival of a new application, the node manager launches two groups of perceptions:
  - Perceptions in the first group (*path-probing setup ants*) probe online the the k paths suggested by the node manager path selection phase.
  - Perceptions in the other group (*path-discovering setup ants*) make use of the pheromone tables built by proactive ants to discover new QoS-feasible paths for the application.
- Ants in both groups temporary reserve resources for the application while they move along the paths.
- Path-discovering setup ants are created with different internal parameters, such that they can be more or less greedy with respect to the current status of the link queues (parameter α in Equation 7.7). Moreover, they always choose the link with the highest probability (computed on the basis of ant-routing table).
- In the case that there is only a little difference between the best and the second best link, the setup ant *forks* and both links are followed (however, to avoid uncontrolled multiplication of ants, an ant is allowed the fork only once).
- If a followed path does not meet anymore the QoS requests, an effector ant is generated to retrace back the setup ant path and free the allocated resources.
- Every setup ant which is able to arrive at destination following a QoS-feasible path comes back and reports the information to the node manager. This information is also used to revise the selection probabilities for the k paths.
- After the first setup ant is back (earlier than a maximum timeout delay), the application can start sending packets.
- If more setup ants come back, the node manager decides about the opportunity to split or not the application over *multiple paths* (e.g., if path superposition is very low it might be quite effective to use multiple paths).
- Once the application is active, periodically some effectors agents (*monitor ants*) are proactively sent over the allocated path(s) to collect information about the application traffic profile and about the links' usage. This information is used to update online the parameters of the fluid-flow approximation model used by the node managers to calculate the environment feedback associated to the possible paths.
- The information reported by the monitor agents is also used to monitor the traffic over established paths for the purpose of *maintenance*. If the network load becomes heavily unbalanced and/or new resources are made available, the application can be gracefully rerouted (or split).
- When the application ends its activities, effector agents are sent over the paths used by the application in order to free the allocated resources.
- The system can be used to manage at the same time QoS traffic and *best-effort* traffic (via the routing tables built by the proactive monitor ants).

AntNet+SELA contains several additional components with respect to both AntNet and AntNet-FA: learning agents as node managers, reactive and effector agents, diversity in the agent population, allocation and maintenance of virtual circuits, etc. However, all the three algorithms are clearly designed according to the same philosophy, which is that that we have tried to capture in the informal definition of ACR.

AntNet+SELA is a good example of how several aspect can and must coexist in the same ant-based routing system in order to cope effectively with the overall complexity of routing tasks. Moreover, it is apparent the modularity of the approach: a different learning architecture can be used for the node managers in place of learning automata without requiring major modifications in the structure and behaviors of the ant-like agents (whose task remains that of exploring and gathering information).

No pseudo-code description is given for AntNet+SELA (as well as for AntHocNet). In fact, these algorithms, are designed to deal with a number of different events (e.g., generation, forwarding, and processing of monitor, path-probing and proactive ant perceptions, as well as of various effector agents), such that, for instance, the pseudo-code description of an AntNet+SELA's node manager would result in a finite state machine with several states and state transitions. The pseudo-code of AntHocNet would result even more complex since AntHocNet has been implemented using a realistic packet-level simulator, such that the protocol has to deal in practice with all the possible events that characterize a fully realistic protocol. These facts would probably make the pseudo-code quite hard to follow and so quite useless for the purpose of helping to get a clearer idea of the algorithm behavior.

7.3.2.2 AntHocNet: routing in mobile ad hoc networks

In recent years there has been an increasing interest in Mobile Ad Hoc Networks (MANETs). In this kind of networks, all nodes are mobile and can enter and leave the network at any time. They communicate with each other via wireless connections. All nodes are equal and there is neither centralized control nor fixed infrastructure to rely on (e.g., ground antennas). There are no designated routers: all nodes can serve as routers for each other, and data packets are forwarded from node to node in a multi-hop fashion. Providing reliable data transport in MANETs is quite difficult, and a lot of research is being devoted to this. Especially the routing problem is very hard to solve, mainly due to the constantly changing topology, the lack of central control or overview, and the low bandwidth of the shared wireless channel. In recent years a number of routing algorithms have been proposed (e.g., see [371, 61, 399]), but even current state-of-the-art protocols are quite unreliable in terms of data delivery and delay.

The main challenge of MANETs for ant-based routing schemes consists in finding the right balance between the rates of agent generation and the associated overhead. In fact, from one side, repeated path sampling is at the very core of ACR algorithms: more agents means that an increased and more up-to-date amount of routing information is gathered, possibly resulting in better routing. On the other side, an uncontrolled generation of routing packets can negatively affect whole sets of nodes at once due to the fact that the radio channel is a shared resource among all the nodes, such that multiple radio collisions can happen at the MAC layer with consequent degradation of performance (this is especially true if the channel has low bandwidth).

In a MANET nodes can enter and leave the network at any time, as well as nodes can become unreachable becomes of mobility and limitations in the radio range. Therefore, in general it is not reasonable to keep at the nodes either a complete topological description of the network or a set of distances/costs to all the other nodes (even at the same hierarchical level) as it can be done, for instance, in the most of the cases of wired networks. These situations call for building and maintaining routing tables on the basis of *reactive* strategies possibly supported by *proactive*

actions in order to continually *refresh* the routing information that might quickly become out-of-date because of the intrinsic dynamism of the network.

This is the strategy followed in *AntHocNet* [126, 128, 154, 127], which is a reactive-proactive multipath algorithm for routing in MANETs. The structure of AntHocNet is quite similar to that of AntNet-FA with the addition of some components specific to MANETs that results in the presence of several types of ant-like agents. In particular, the design of AntHocNet features: reactive agents to setup paths toward a previously unknown destination, per-session proactive gathering of information, agents for the explicit management of link failure situations (because of mobility and limited radio range the established radio link between two nodes can easily break). Node managers are not really learning agents as it was the case for AntNet+SELA, but rather finite state machines responding more or less reactively to external events. This is partially due to the fact that in such highly dynamic environments it might be of questionable utility to rely on approaches strongly based on detecting and learning environment's regularities. In the general case, some level of learning and proactivness is expected to be of some usefulness, but at the same time the core strategy should be a reactive one. This has been our design philosophy in this case.

The algorithm's behavior is explained in the following subsections. Each subsection discusses the algorithm actions in relationship to a different subtask: (i) setup of routing information, (ii) maintenance of established routes and exploration of new ones, (iii) data routing, (iv) management of link failures.

Path setup

When a new data session to a destination d is started, the node manager at source node s reactively sends out a *setup ant* for the purpose of searching for routes between s and d (unless, of course the source already has routing information about the destination). Setup ants, as all the other agents, always make use of higher priority queues with respect to data packets. Each node which receives the setup ant either radio *broadcast* or *unicast* it according to the fact that it has or not routing information about d in the routing table. The node to unicast the ants to is selected according to the pheromone information. Broadcasting determines a sort of *proliferation* of the original ant since each neighbor will receive a copy of it. This proliferation, if uncontrolled, can be easily lead to an unwanted congestion. Therefore, setup ants are *filtered* according to the quality of the path followed so far in order to limit the generated overhead. When a node receives several ants of the same generation (i.e., deriving from the same forward ant generated in s), it will compare the path traveled by the ant to that of the previously received ants of the same generation: only if its number of hops and travel time are both within a certain factor of that of the best ant of the generation, it will forward the ant. Ants can get also killed on the way if their number of hops exceeds a predefined time-to-live value, which is set according to the network size.

Upon arrival at the destination d, the forward ant is converted into a backward ant which travels back to the source, retracing the forward path. At each node i, it sets up a path towards the original destination d creating or updating routing table entries $\mathcal{T}_{nd}^{i}$ for the neighbor n it is coming from. The entry will contain a pheromone value which represents an average of the inverse of the cost, in terms of both *estimated time* and *number of hops*, to travel to d through n from i.

Data routing

The path setup phase creates a number of paths between source and destination, indicated in the datagram routing tables of the nodes. Data can then be forwarded between nodes accord-

ing to a stochastic policy parametrized by the data-routing tables, which are obtained from the pheromone tables after a power-law transformation equivalent to that used in AntNet-FA.

The probabilistic routing strategy leads to data load spreading with consequent *automatic load balancing*. This might be quite important in MANETs, because the bandwidth of the wireless channel is very limited. Of course, in order to spread data properly, adapting to the repeated modifications in both the traffic and mobility patterns, it is customary to keep monitoring the quality (and the existence) of the different paths. To this end the node managers generate proactive maintenance ants (perceptions) and pheromone diffusion ants (effectors).

Path maintenance and exploration

While a data session is running, the node manager sends out *proactive maintenance ants* according to the data sending rate (one ant every n data packets). They follow the pheromone values similarly to data but have a small probability at each node of being broadcast. In this way they serve two purposes. If an ant reaches the destination without a single broadcast it simply samples an existing path. It gathers *up-to-date quality estimates* of this path, and updates the pheromone values along the path from source to destination. If on the other hand the ant got broadcast at any point, it will leave the currently known "pheromone-constrained" paths, and *explore* new paths. However, we limit the total number of broadcasts of a proactive ant to a small number (e.g., two) in order to avoid excessive agent proliferation. The effect of this mechanism is that the search for new paths is concentrated around the current paths, so that we are looking for *path improvements* and *variations*.

In order to guide the search more efficiently, node managers make use also of effector agents termed *pheromone diffusion ants*: short messages resembling hello messages broadcast every t seconds (e.g., $t = 1\ sec$). If a node receives a pheromone diffusion agent from a new node n, it will add n as a new destination in its routing table. After that it expects to receive an agent from n every t seconds. After missing a certain number of expected messages (2 in our case), n will be removed. Using these messages, nodes know about their immediate neighbors and have pheromone information about them in their routing table. Pheromone diffusion ants also serve another purpose: they allow to detect broken links. This allow nodes to clean up stale pheromone entries from their routing tables. In the following we plan to make these agents carrying more information (e.g., the list of neighbors).

Link failures

Node managers can detect link failures (e.g., a neighbor has moved far away) when unicast transmissions (of data packets or ants) fail, or when expected pheromone diffusion agents were not received. When a link fails, a node might loose a route to one or more destinations.

If the node has other next hop alternatives to the same destination, or if the lost destination was not used regularly by data, this loss is not so important, and the node manager will just update its routing table and broadcast an effector agent, termed *failure notification ant*. The agent carries a list of the destinations it lost a path to, and the new best estimated end-to-end delay and number of hops to this destination (if the node still has entries for the destination). All its neighbors receive the notification and update their pheromone table using the new estimates. If they in turn lost their best or their only path to a destination due to the failure, they will in turn generate and broadcast a failure ant, until all nodes along the different paths are notified of the new situation.

If the lost destination was regularly used for data traffic, and it was the node's only alternative for this destination, the loss is important and the node should try to locally *repair the path*. This is the strategy followed in AntHocNet, with the restriction that a node only repairs the path

if the link loss was discovered with a failed data packet transmission. After the link failure, the node manager broadcasts a *route repair ant* that travels to the involved destination very alike a setup ant: it follows available routing information when it can, and is broadcast otherwise (with a limit of the maximum number of broadcasts). The node manager waits for a certain time and if no backward route repair ant is received, it concludes that it was not possible to find an alternative path to the destination which is then removed from the routing table, and a failure notification ant is generated to advertise the new situation.

7.4 Related work on ant-inspired algorithms for routing

An exhaustive related work analysis should include general work on both multipath and adaptive routing, as well as work on (multi-)agent systems in telecommunication and ant/nature-inspired work on routing. The general discussions on routing approaches given in the previous chapter, as well as the discussions in the next chapter on the algorithms used for performance comparison, partly cover general related work on adaptive multipath algorithms. On the other hand, the use of agent and multi-agent technologies in the telecommunication domain (and not only in this domain) is attracting a growing interest, and there is already a large number of applications and scientific studies. However, a proper discussion of these works would result rather long while at the same time take us quite far away from the logical path that has been followed so far. Therefore, we choose to not to account here for related work on multi-agent systems. Good overviews and/or particularly significant applications can be found for instance in [444, 400, 220, 259, 206, 320, 424, 265, 432].

According to these facts, the only related work which is discussed in the following of this section concerns ant-inspired algorithms for routing in telecommunication networks. That is, algorithms that have been inspired by some behavior of either ants or ant colonies. As a matter of fact, most of the ant-inspired algorithms algorithms take inspiration from the the same foraging behavior at the roots of ACO. Therefore, they can be seen as conceptually, if not historically, belonging to the ACO framework. Moreover, several among these algorithms are either modifications of or have been directly inspired by AntNet. A fact that indirectly confirms the general goodness and appealing of the approaches to routing that have been proposed in this thesis.

It follows a commented list (chronologically ordered) of ACO- and ant-inspired related work on routing. The algorithms are divided in two groups, those for wired networks (including both data and telephone networks, and best-effort an QoS routing), and those for wireless and mobile ad hoc networks. The review is not comprehensive. In fact, it is quite hard to keep track of all the newly proposed implementations implementations (this is particularly true for the case of mobile ad hoc networks). Moreover, the list does not take into account approaches that bear only little resemblance with ACO and/or that have to be intended more as proof-of-concept than practical implementations and study of optimized algorithms for routing. The algorithms for mobile ad hoc networks are only shortly discussed, since the focus in the thesis is more on wired networks. Moreover, as a matter of fact, in spite of their claim of being inspired by ACO and/or AntNet, the majority of these algorithms for MANETs loose much of the proactive sampling and exploratory behavior of the original ACO approach in their attempt to limit the overhead caused by the ant agents, such that they often show a behavior which is very close to that of the already mentioned AODV (while, on the other hand, AntHocNet retains most the of original ACO's characteristics).

Wired networks

• Schoonderwoerd, Holland, Bruten and Rothkrantz (1996, 1997) [382, 381] were the firsts to consider routing as a possible application domain for algorithms inspired by the behavior of

ant colonies. Their approach, called *ABC*, has been intended for routing in telephone networks, and differs from AntNet in many respects that are discussed here with some detail to acknowledge the impact of ABC during the first phase of AntNet's development. The major differences between the two algorithms are a direct consequence of the different network model that has been considered. ABC's authors considered a network with the following characteristics: (see Figure 7.15): (i) connection links can potentially carry an infinite number of full-duplex, fixed bandwidth channels, and (ii) transmission nodes are crossbar switches with limited connectivity (that is, there is no necessity for queue management in the nodes). In such a model, bottlenecks are put on the nodes, and the congestion degree of a network can be expressed in terms of connections still available at each switch. As a result, the network is *cost-symmetric*: the congestion status over available paths is fully bi-directional. A path $\mathcal{P}_{s \to d}$ connecting nodes s and d exhibits the same level of congestion in both directions because the congestion depends only on the state of the nodes in the path. Moreover, dealing with telephone networks, each call occupies exactly one physical channel across the path. Therefore, calls are not multiplexed over the links, but they can be accepted or refused, depending on the possibility of reserving a physical circuit connecting the caller and the receiver. All these modeling assumptions make the problem of Schoonderwoerd *et al.* rather different from the more general cost-asymmetric routing problems for data networks considered by AntNet algorithms. This difference is reflected in several important implementation differences between ABC and AntNet. The most important one consisting in the fact that in ABC ants update pheromone trails after each hop, without waiting for the completion of an experiment, as done in AntNet, and the ants do not need to go back to their source.[23] In ABC ants move over a control network isomorphic to the one were calls are really established. In the used model the system evolves synchronously according to a discrete clock. At each node an ant ages of ΔT virtual steps computed as a fixed function of the current node spare capacity, that is, the number of still available channels. If s is the source and d the destination node of a traveling ant, after crossing the control link connecting node i with node j, the routing table on node j is updated by using the ant age T. The probability for subsequent ants to choose node i when their destination node is s, is increased proportionally to the current value of T according to an updating/normalizing rule similar to rules 7.9 and 7.8. In this way, during the ant motion, the routing table entries that are modified are those concerning the ant source node, that is, modifications happen in the direction opposite to that of the ant motion. This strategy is sound only in the case of an (at least) approximately cost-symmetric network, in which the congestion status is direction-independent.

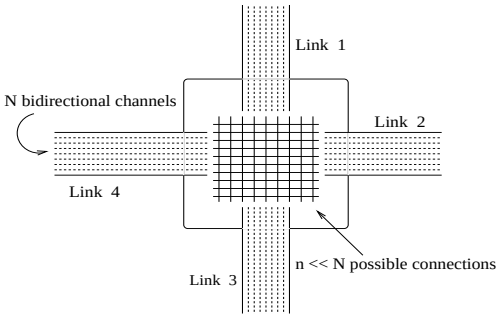

Figure 7.15: *Network node in the telecommunication network model of Schoonderwoerd et al. [382, 381].*

Other important differences can be found in the fact that: (i) ABC does not use local traffic models to score the ant traveling time: T values are used as they are, without considering their relativity with respect to different network states (see Subsection 7.1.4), (ii) neither local queue information (see Equation 7.7) nor ant-private memory are used to improve the ant decision policies and to balance learned pheromone information and current local congestion. Moreover,

[23] This choice, justified by the cost-symmetry assumption, is reminiscent of the pheromone trail updating strategy implemented in *ant-density*, one of the Dorigo's *et al.* first algorithms inspired by the foraging behavior of ant colonies [150, 135, 91], and makes ABC behavior in a sense closer to those of real ants than AntNet (see Chapter 2).

ABC's ants do not recover from cycles and do not use the information contained in all the ant sub-paths (see Subsection 7.1.3.7).
ABC has been tested on a model of the British Telecom (BT) telephone network in UK (30 nodes) using a sequential discrete time simulator and has been compared, in terms of percentage of accepted calls, to an agent-based algorithm previously developed by BT researchers [7]. Results were encouraging: ABC always performed significantly better than its competitor on a variety of different traffic situations.

• Subramanian, Druschel and Chen (1997) [411] have proposed an ant-based algorithm for packet-switched networks. Their algorithm is a straightforward extension of Schoonderwoerd *et al.* system by the addition of so-called *uniform ants*, a very simple (but of quite questionable efficacy) exploration mechanism: ants just wonder around without a precise destination and select their next hop according to a uniform random rule. While regular ants update the routing tables forward, in the direction of their destination, uniform ants update in the reverse direction, that is, toward the nodes they come from. Both the two types of ants update the routing tables according to a not well specified path cost, which is the sum of the costs associated to each one of the links belonging to the path followed so far. The mechanism of the uniform ant is intended to avoid a rapid sub-optimal convergence of the algorithm and help to find new paths in case of link/node failure (the algorithm is explicitly aimed at scenarios involving frequent topological modifications). A major limitation of the approach consists in the fact that, although the algorithm they propose is based on the same cost-symmetry hypothesis as ABC, they apply it to packet-switched networks where this requirement is seldom met. On the other hand, the pheromone updating of the uniform ants, since it happens in the reverse direction, does not require the assumption of cost-symmetry, but it is however prone to efficiency problems since the followed paths from one node to another are expected to be highly sub-optimal due to the uniform random selection rule.

• Bonabeau, Henaux, Guérin, Snyers, Kuntz and Theraulaz (1998) [50] have improved ABC by the introduction of a mechanism based on dynamic programming ideas. Pheromone values along an ant path are updated not only with respect to the ant's origin node as in ABC, but also with respect to all the other intermediate nodes between the origin and the ant current node. A similar mechanism, as discussed in depth in Subsection 7.1.3.7, was earlier implemented in AntNet since its first draft version [116]. The difference between the AntNet's and Bonabeau's *et al.* updating strategy lies in the fact that AntNet does not necessarily updates all the sub-paths of a path, but only those which appear to carry reliable or competitive information. The fact that a straight application of the Bellman's principle is not free from problems has been discussed in Subsection 7.1.3.7.

• Van der Put and Rothkrantz (1998, 1999) [426, 427] designed *ABC-backward*, an extension of the ABC algorithm based on the AntNet strategy. Accordingly, ABC-backward is applicable to cost-asymmetric networks, but, ultimately, the algorithm results to be identical to AntNet with some, rather questionable simplifications directly inherited from the ABC settings. The authors use the same forward-backward mechanism used in AntNet: Forward ants, while moving from the source to the destination node, collect information on the status of the network, and backward ants use this information to update the routing tables of the visited nodes during their journey back from the destination to the source node. In ABC-backward, backward ants update the routing tables using an updating formula identical to that used in ABC, except for the fact that the ants' age is replaced by the trip times experienced by the ants in their forward journey. The authors have shown experimentally that ABC-backward has a better performance than ABC on both cost-symmetric (because backward ants can avoid depositing information on cycles) and cost-asymmetric networks. ABC-backward has been applied to a fax distribution problem proposed by the Dutch largest telephone company (KPN Telecom).

• White, Pagurek and Oppacher (1998) [446, 445] use an ACO algorithm for unicast and mul-

ticast routing in connection-oriented networks. The algorithm, called *ASGA*, follows a scheme very similar to that of Ant System (see Section 4.2) which was designed for TSPs. On the other hand, the TSP is a minimum cost path problem with the addition of the Hamiltonian constraint. Therefore, it can be seen in the terms of a constrained routing problem. In this perspective, the authors added some additional components to Ant System to account for the fact that they were dealing with routing and not anymore with TSP. In particular, they have used the same forward-backward mechanism of AntNet, combined with a mechanisms very similar to that envisaged in ACR for supporting the setup phase of a QoS or connection-oriented application and for maintaining it. The major difference with AntNet lies in the fact that the authors apply the same selection and updating formulae used in Ant System. In practice, Equation 7.7 is replaced by one in which the terms are multiplied instead of being summed, and the l term is replaced by a not clearly specified link cost. Equations 7.9 and 7.8 are replaced by exponential averages. From the source node of each incoming connection, a group of ants is launched to search for a path. At the beginning of the trip each ant k sets an internal path cost variable C_k to 0, and after each link crossing the internal path cost is incremented by the current link cost l_{ij}: $C_k \leftarrow C_k + l_{ij}$. When arrived at destination, the ant moves backward to its source node and at each node uses a simple additive rule, similar to that of AntNet, to compute the equivalent of the AntNet's T value: T becomes equal to C_k, which is the sum of all the previously encountered costs. When all the spooled ants come back at the source node, a simple local daemon algorithm decides whether a path should be allocated for the session, based on the percentage of ants that followed a same path. Moreover, during all the connection lifetime, the local daemon launches and coordinates exploring ants to re-route the connection paths in case of network congestion or failures. A genetic algorithm [202, 226] is used online to evolve two critical parameters (the equivalent of α in Equation 7.7) that regulate the behavior of the transition rule formula (the name ASGA precisely comes from this mechanism: ant-system plus genetic algorithm). Some preliminary results were obtained testing the algorithm on several networks and using several link cost functions. Results are promising: the algorithm is able to compute shortest paths and the genetic adaptation of the rule parameters seems to improve considerably the algorithm's performance.
Actually the same authors have published several other short conference papers on ant, swarm, multi-agent systems for network management and control. Most of them mix the above mechanisms and concepts with other similar flavors, therefore they are not further mentioned here.

• Heusse, Snyers, Guérin and Kuntz (1998) [224] developed a new algorithm for general cost-asymmetric networks, called *Co-operative Asymmetric Forward* (CAF). CAF, even if still grounded on the ACO framework, introduces some new ideas to exploit the advantages of an online step-by-step pheromone updating scheme, as that used in ABC, in addition to the forward-backward model of AntNet. In ABC the step-by-step updating strategy was made possible by the assumptions of cost-symmetry, in CAF this assumption is relaxed, but it is still possible to use step-by-step updates. In fact, each data packet, after going from node i to node j, releases on node j the information c_{ij} about the sum of the waiting and crossing times experienced from node i. This information can be used as an estimate of the traveling time to go from i to j. An ant $A_{s \to d}$ hopping from j to i reads the c_{ij} information in j and moves it to i, where it is used to update the estimate for the time to travel from i to j, and, accordingly, for the total traveling time $T_{i \to d'}$ from i to all the nodes s' visited by the ant during its journey from s. In this way the routing tables in the opposite direction of the ant motion can be updated online step-by-step. Clearly, in order to work properly, the system requires that an "updating ant" would arrive in coincidence or with a negligible time shift with respect to the moment the "information ant" deposited the information c_{ij}. The algorithm's authors tested CAF under some static and dynamic conditions, using the average number of packets waiting in the queues and the average packet delay as performance measures. They compared CAF to an algorithm very similar to an earlier

version of AntNet. Results were encouraging and under all the test situations CAF performed better than its competitors.

• The CAF mechanism is quite general and could be straightforwardly incorporated in AntNet and AntNet-FA. Actually, Doi and Yamamura (2000, 2002) [133, 134] make use of a similar strategy in their *BntNetL* algorithm. BntNetL has been designed as a supposed improvement over AntNet-FA. Supposed because the authors misunderstood some of the behaviors of AntNet-FA and then tried to devise some additional heuristics to "correct" them. For instance, they have asserted that AntNet's selection rule could get "locked" if one entry in the routing table would reach the limit value of 1, but actually this is not the case, because AntNet-FA makes use of the selection rule of Equation 7.7 which weights both the entries in the routing table and the current status of the link queues. BntNetL has been compared to AntNet-FA on a restricted set of simulated situations. BntNetL and AntNet-FA (actually, a version of AntNet-FA containing some incorrectly interpreted parts), showed more or less similar performance.

• On a similar stream of misleading interpretations of AntNet is the work of Oida and Kataoka (1999) [337]. These authors, for some reasons, decided to work on improving the very first draft version of AntNet [116], in which the status of the data link queues was not used, such that the above mentioned "lock" mechanism could actually happen. To avoid this fact, Oida and Kataoka have added some simple adaptive mechanisms to the routing table updating rule. Their algorithms, *DCY-AntNet* and *NFB-Ants*, once compared to the early version of AntNet [116] performed much better under the challenging considered situations. Actually, the mechanisms proposed by Oida and Kataoka could be also of some usefulness in AntNet-FA and ACR, but their efficacy should be anyhow tested more carefully.

• The same Oida, but this time in collaboration with Sekido (1999, 2000) [338, 339], also added some functionalities to the basic behavior of AntNet to make it suitable to work in a QoS environment with constraints on the bandwidth and the number of hops. In particular, in their algorithm *Agent-based Routing System* (ARS), they make the forward ants moving in a "virtually constrained network" in order to support several classes of bandwidth requirements (e.g., similarly to the DiffServ model [440]). The probability of a forward ant of choosing a link is made depending not only on the values in the routing table but also on the level of the already reserved bandwidth, and for each class of service in terms of bandwidth there is a different colony of ants. All the ants of a colony only use those links whose the unreserved bandwidth resources are not less than the value of the bandwidth constraint assigned to the colony. If almost all bandwidth had already been reserved then the probability is accordingly made very low. Similarly, if the number of executed hops is already too large and/or none of the outgoing links has still much free bandwidth, then the forward ant terminates its journey. Interestingly, these heuristics can be readily seen as the direct transposition of the basic AntNet's mechanisms in a QoS context. In fact, the heuristic taking into account the available bandwidth to assign the link probabilities is equivalent to the AntNet's heuristic that takes into account the current number of bits waiting on the link queue. While the mechanisms for the ant self-termination had already been implemented in AntNet by using the TTL parameter to remove those ants that are trying to build a likely very bad path.

• Using the same concepts (stigmergy, ants, pheromone, multi-agency) underlying ACO but not being explicitly inspired by ACO, Fennet and Hassas (2000) [166, 167] developed a model of a multi-agent system for multiple criteria balancing on a network of processors. It has been tested on a some rather simple simulated situations. In their system ants move among the processing units, perceiving local artificial pheromone fields that are emitted by the units and that encode some useful information about the criteria to optimize. While the description that the authors give of the actions and of the tasks is quite vague and not not well defined, it is interesting to mention here that they use a terminology that reminds that of ACR: a distinction is made

between static and mobile agents interacting together, while the mobile agents are also seen as sensory and effector agents for user applications.

• Baran and Sosa (2000) [13] have proposed several supposed improvements over AntNet: (i) routing tables are initialized more efficiently by taking into account knowledge about the neighbors, such that ants searching for destinations coinciding with one of the neighbors are not actually generated, (ii) link and node failures are explicitly taken into account: after a link failure, the related pheromone entries are explicitly set to zero, (iii) with some small probability ants can also issue a random uniform selection among the available alternatives in order to supposedly reduce the (in)famous locking problem that has been repeatedly and incorrectly attributed to AntNet, (iv) ants make greedy deterministic decisions instead of random proportional ones (with the clear risk of incurring in very long lived loops and at the same time cutting off necessary exploration),(v) the number of ants living in the network is arbitrarily limited to four times the number of links (it is however not clear why this number and also how the number of active ants can be actually determined given the fully distributed characteristics of the problem).

• Michalareas and Sacks (2001) [314] have studied the performance of an AntNet version that makes use of deterministic greedy choices and of no link queue heuristic with respect to that of an OSPF-like algorithm. The authors have considered three small topologies (tree, ring and grid-like) and uniform FTP traffic using TCP Tahoe. According to the their results, at steady state both the algorithms show equivalent performance. The same authors have implemented a hybrid between their AntNet and ABC for the management of the routing in multi-constrained QoS networks [315, 314]. They have considered an IP network offering soft-QoS service with two constraints: on the end-to-end delay and on the available bandwidth. The proposed algorithm makes use of two types of ant agents, one for each QoS constraint. The ants dealing with the delay constraint are the same as in their previous version of AntNet, since AntNet's ants precisely try to minimize end-to-end delays. On the other hand, the bandwidth constraint is dealt with ants that are artificially delayed at the nodes, as in ABC, proportionally to the occupied link bandwidth as measured by a local exponential average of the link utilization. In this way, their virtual delay bring a measure of the available bandwidth along the path. Experiments conducted on the same traffic types and networks of the previously mentioned work show that the AntNet-like algorithm performs similarly to an OSPF-like one, but scales much better with the increasing of the load.

• In [378] (2001), Sandalidis, Mavromoustakis, and Stavroulakis have studied the performance of Schoonderwoerd et al.'s ABC using a different network and considering some additional variables to monitor ants behavior. Their study confirms the earlier results for ABC. In a more recent work [379] (2004), the same authors have developed a new version of ABC which makes use of the notions of probabilistic routing of the phone calls and of anti-pheromone. When an ant arrives at a node with an age larger than that currently recorded on the node, pheromone is decreased instead of being increased. The performance of the new algorithm is compared to that of ABC for a topology of 25 nodes and have shown a certain degree of improvement over it.

• Jain (2002) [234] has compared AntNet to a link-state protocol using the well known network simulator ns-2. The author has implemented a version of AntNet very similar to that of Michalareas and Sacks and has made use of greedy deterministic forwarding for data packets (the link with the highest probability is deterministically chosen), such that the algorithm turned into a single-path one. Experiments were run on a small grid network and on the same networks that have been used in the next chapter for AntNet experiments, but using different traffic patterns. The experiments show that under light traffic conditions the two considered algorithm behave similarly, but on the other hand the single-path AntNet algorithm can adapt to new situations much quicker and much better.

• Kassabalidis, El-Sharkawi, Marks II, Arabshahi, and Gray (2002) [244] have proposed a modification of AntNet based on the organization of the network in clusters and on the use of

multiple colonies. The network is first partitioned into clusters of nodes which are defined by running a centralized k-means algorithm using as a metric the geographic location of the nodes (e.g., clusters can be easily seen in the terms of Autonomous Systems in the Internet). Once the clustering is realized, intra- and inter-clustering route discovery and maintenance is realized by two different types of ants, and separate routing tables are hold at the nodes for intra- and inter-clustering routing. In this way, the number of ants that have to be sent is in principle reduced, since every node needs to hold a route only to each node in the same cluster it belongs to and to each other cluster, and not to all the other nodes in the network (see also the discussion in Subsection 7.1.3.1 about the assumption of adopting a flat topological organization in AntNet). The authors propose also other minor modifications that are more simplifications of the AntNet's mechanisms more than real improvements. In particular, again, the authors have misunderstood AntNet's behavior, and affirm that in AntNet data packets are routed deterministically according to the link with the highest probability (while a whole section is specifically devoted to this issue in the main AntNet paper [119, Pages 352–353]). Their AntNet-derived algorithm, *Adaptive-SDR*, was compared for two test networks of 16 and 49 nodes respectively to their single-path implementation of AntNet and to OSPF and RIP. The ns-2 simulator has been used. While AntNet, OSPF, and RIP show similar performance, Adaptive-SDR shows much better results for both throughput and average delay.

The same authors have also realized a short review on so-called swarm intelligence (in practice, ACO) algorithms for routing in networks [245]. Moreover, in [243] they have also discussed the conditions for the applicability of AntNet-like algorithms to wireless networks, like satellite and sensor networks. In particular, they have pointed out the importance of energy and radio propagation issues, and have proposed some possible ways of incorporating these aspects in the mechanisms used to search and score a path.

• Sim and Sun (2003) [390] have made a quite detailed review paper on ACO approaches for routing and load balancing, focusing in particular on discussing the issue of the different methods (for both static and dynamic problems) devised to avoid early convergence in the pheromone tables (also previously indicated as stagnation or locked decisions). In the same paper the authors propose an ACO approach, named *MACO*, based on multiple colonies for load balancing tasks in connection-oriented networks. The idea is to use multiple colonies and pheromone repulsion among colonies in order to find good disjoint path to allocate the traffic sessions: at setup time of a traffic session multiple (two in the example reported in the paper) colonies are generated and concurrently search for feasible paths. The issue of how many colonies should be generated in not considered. Differently from what is claimed in the paper, AntNet and AntNet-FA, with their stochastic data spreading, comes with a built-in way of providing load balancing. However, the use of pheromone repulsion in order to favor the exploration of disjoint paths appears as promising and it has been already used also by Navarro Varela and Sinclair (1999) [333] to solve (static) problems of virtual wavelength path routing and wavelength allocations. For this class of problems the challenge consists in allocating a minimum number of wavelengths for each link by evenly distributing the wavelengths over the different links, while at the same time minimizing the number of hops for each path. Pheromone repulsion is used precisely to favor the even distribution of the wavelengths.

• Tadrus and Bai (2003) [416] have implemented the *QColony* algorithm for QoS routing. QColony is based on AntNet but contains a number of new features specific for QoS, such that it is more correct to see it as an interesting example of ACR. In QColony each node contains several QoS pheromone tables, with each table used to route ants associated to a different QoS requirement. Each QoS table is associated to a range of continuous bandwidth constraints (i.e., each QoS table is related to one class of service in terms of provided range of bandwidth). QoS sessions ask for a class of service in terms of acceptable range of bandwidth and maximum number of hops. The QoS tables are built by the ants according to several proactive and on-demand

schemes, and are used by the ants to search for feasible QoS paths at the setup time of a new session. The system comprises several classes of ant agents, each class has a different priority and deals with a different task: search for best-effort paths, search for a QoS-feasible path at setup time, retry of the search in case of failure of the first search attempt, allocation/deallocation of resources, search for alternative paths to be used by the QoS sessions in case of link/node failures. All the ants update the QoS tables, but with a strength which is proportional to their priority and age. QColony shares several similarities with AntNet+SELA. In fact, both: make use of different agents for the different tasks, search for a QoS-feasible path at setup time by using ants, send per-session ants (so-called *soldier ants* in the case of QColony) in order to provide the QoS session with a bundle of paths to deal with possible failure situations, try to optimize also the number of hops of the used paths. However, important differences also exist between the two algorithms. In particular, one of the claims of QColony consists precisely in the fact that purely local information is used at the nodes, while in the case of AntNet+SELA the node manager issues its decisions on the basis of an OSPF-like topological description of the whole network. The QColony's design is quite interesting, since it is another crystalline example of how different types of agents and on-demand/proactive generation strategies are jointly necessary in order to effectively deal with complex routing tasks. The performance of QColony has been compared to that of ARS and to the selective flooding algorithm described in [79]. For the considered scenarios (three networks up to 35 nodes, and 10 ranges of bandwidth) QColony has shown performance comparable to the competitors for small networks and mild traffic, and much better performance for the larger networks and heavy traffic loads.

Other work in the domain of QoS is that of Subing and Zemin (2001) [410] and Carrillo et al. (2003) [74], that suggest algorithms similar to QColony in the sense of using ants at setup time and multiple QoS tables, but the structure of their algorithms is simpler and has less components than that of QColony. The same authors of [74] have also made a preliminary theoretical study on the general scalability of AntNet, showing the expected good level of scalability of the approach [74].

• Other implementations of both the ACO and AntNet paradigms are the works of: Gallego-Schmid (1999) [181], who implemented a minor modification of AntNet in the general perspective of using it for network management, and Zhong (2002) [452], which is the first who has considered the issue of security when using AntNet, an issue that has been also pointed out when ACR has been discussed, emphasizing the fact that ant agents should not be allowed to directly modify the routing tables. The use of key certificates and ant identifiers are proposed as a way to overcome to possible armful situations like: (i) generation of bogus backward ants in order to promote/avoid a specific route, (ii) dropping ant agents in order to not allow them to further update routing tables and/or find a path through the current node, (iii) tampering the backward ants information in order to generate wrong routing paths.

Wireless and mobile ad hoc networks

• Matsuo and Mori (2001) [302] have proposed *Accelerated Ants Routing*, which is an extension to MANETs of the ideas of Subramanian et al. [411]. They add the rule that uniform ants do not return to an already visited node, and make these ants to hold the history about the n last visited nodes. In this way, routing information can be updated not only toward the source, but all the intermediate nodes. The algorithm heavily rely on the uniform ants, and no on-demand actions are taken. It has been compared for a small 10 nodes network to Q-routing, dual Q-routing [264], and to the original algorithm making use of uniform ants, showing better performance than the competitors. In a later paper [178] the algorithm was tested in a 56 node network, and compared also to AntNet, again showing better performance and faster convergence.

• Câmara and Loureiro (2001) [69, 68] describe a location-based algorithm which makes use of ant agents to collect and disseminate routing information in MANETs. Every node keeps in a routing table location information of all the other nodes, and routing paths are calculated with a shortest path algorithm. To keep the tables up-to-date, information is exchanged locally among neighbors, and globally by sending ants to nodes further away. In practice, ants implement an efficient form of flooding. The algorithm is compared to another popular location-based algorithm, LAR [255] and shows less overhead and comparable performance.

• Sigel, Denby, and Heárat-Mascle (2002) [389] have applied a straightforward modification (and simplification) of AntNet to the problem of adaptive routing in the LEO satellite network. Some experimental results from simulations are discussed taking into account some possibly realistic traffic patterns in terms of both voice and data telecommunication. The algorithm is compared to more a classical routing algorithms such as SPF (see next chapter), as well as to an "ideal" realization of it aimed at providing a kind of performance upper bound. Overall, the performance of the proposed algorithm is near-optimal and much better than that of SPF, especially for high non-bursty traffics.

• Günes et al. (2002) [211, 210] introduced *Ant-Colony-Based Routing Algorithm (ARA)*, which does not differ much from other popular MANET algorithms like AODV. In fact, it works in a purely on-demand way, with both the forward and backward ants setting up the paths about the node they are coming from. Also data packets do the same, reducing in this way the node for sending more ants. The reported performance was slightly better than that of AODV but worse than that of DSR [236] in highly dynamic environments.

• Marwaha et al. (2002) [301] have proposed *Ant-AODV*, an hybrid algorithm combining ants with the basic AODV behavior. A certain number of ants keeps going around the network in a more or less random manner, keeping track of the last n visited nodes and when they arrive at a node they update its routing table. The behavior of the algorithm is like AODV, but the presence of the ants is supposed to boost AODV's performance. In fact, they proactively refresh routing tables, increasing the chance that either the node will have a route available or one of its neighbors has. Moreover, ants can discover better paths than those currently in use by AODV and the paths can be rerouted. According to the reported simulation studies, Ant-AODV performs usually better than the simple ant-based algorithm or AODV separately.

• In [14] Baras and Mehta (2003) propose two algorithms for MANETs. The first is very similar to AntNet: it is in fact purely proactive, trying to maintain pheromone entries for all the destinations. It differs from AntNet because of data packets that are routed deterministically and ants taking also random uniform decisions for the purpose of unbiased exploration. The algorithm does not work very well, due to the large amount of routing overhead, and the inefficient route discovery. The second version of the algorithm, called *Probabilistic Emergent Routing Algorithm* (PERA), is much closer to AODV. In fact, the algorithm becomes purely reactive: ants are only sent out when a route is needed. Also, they are now broadcast instead of being unicast to a certain destination. In this way, the forward ants are in practice identical to route request messages in AODV and DSR. The only difference stays in the fact that multiple routes are set up, but actually only the one with the highest probability is actually used by data, with the other being available for quick recovery from link failures. The performance of the algorithm is comparable to that of AODV.

• In [221], Heissenbüttel and Braun (2003) describe an ant-based algorithm for large scale MANETs. The algorithm is quite complicate and makes use of geographical partitioning of the node area. It shares some resemblance with the Terminode project [40, 39].

7.5 Summary

In this chapter four novel ACO algorithms for adaptive routing in telecommunication networks, AntNet, AntNet-FA, AntHocNet, and AntNet+SELA, have been described and throughly discussed. These algorithms cover a wide range of possible network scenarios. Experimental result are be reported in the next chapter, and only for the first three of them.

In addition to these algorithms, in this chapter we also introduced ACR, a high-level distributed control framework that specializes the general ACO's ideas to the domain of network routing and at the same time provides a generalization of these same ideas in the direction of integrating explicit learning components into the design of ACO algorithms.

AntNet, which was chronologically the first of these algorithms to be developed, is a traffic-adaptive multipath routing algorithm for best-effort datagram networks. It makes use of a strategy based on:(i) ant-like mobile agents repeatedly and concurrently sampling paths between assigned source and destination nodes, (ii) stochastic routing of data according to the local pheromone values, with automatic load balancing and multipath routing effects, (iii) information from both pheromone values (resulting from collective ant learning) and current status of the local link queues (current congestion at the node) to take balanced ant routing decisions, (iv) adaptive statistical models to track significant changes in the estimate end-to-end delays of the sampled paths. The basic characteristics of the routing delivered by AntNet (traffic-adaptive, and multipath), and the strategies adopted to obtain these behaviors (active sampling by mobile agents, stochastic routing, Monte Carlo learning, use of local queues information) are in some sense conserved also in all the other algorithms. They can be seen as their true fingerprints. Moreover, we discussed in this and in the previous chapter why these characteristics and strategies have to be seen as innovative with respect to those of popular routing algorithms.

AntNet-FA is an improvement over AntNet in which both backward and forward ants make use of high priority queues. AntNet-FA allows prompter reactions and always makes use of information more up-to-date with respect to AntNet. AntNet-FA is expected to always perform equally or better than AntNet. This will be confirmed by the experimental results reported in the next chapter.

ACR introduces a hierarchical organization into the previous schemes, with node managers that are fully autonomic learning agents, and mobile ant-like agents that are under the direct control of the node managers and serve for the purpose of non-local discovering and monitoring of useful information. Even if all the ACO routing algorithms described in this thesis can be seen as instances of the ACR framework, the purpose of defining ACR is more ambitious than a pure generalization of the ACO design guidelines for the specific case of network problems. ACR defines the general architecture of a multi-agent society based on the integration of the ACO's philosophy with ideas from the domain of reinforcement learning, with the aim of providing a framework of reference for the design and implementation of fully autonomic routing systems in the same spirit as described in more general terms in [250] (fully distributed systems able to globally self-govern through self-tuning, self-optimization, self-management, ..., of the single autonomic unit that socially interacts with all the other units). ACR points out the critical issues of the scheduling of the ant-like agents, the definition of their characteristics, and the ability to deal effectively with a range of different possible event (such that some diversity is necessary into the population of the ant-like agents).

In order to show how these ACR issues can be (partly) dealt with, two more novel routing algorithms have been described: AntNet+SELA, for QoS routing in ATM networks, and AntHocNet, for best-effort routing in mobile ad hoc networks. The purpose of introducing these two additional algorithms was twofold: from one hand they address more complex routing problems than those considered by AntNet and AntNet-FA, such that that they are practical examples of

how to deal with the issues of ant scheduling, diversity, response to several different high-level events, and so on, raised up in ACR. So, they can be seen as practical instances of ACR. On the other hand, with AntNet+SELA and AntHocNet in addition to AntNet and AntNet-FA, we have covered the majority of the routing scenarios of practical interest.

In AntNet+SELA the node managers, that are directly responsible for both routing and admission decisions, are designed after the *stochastic estimator learning automata* (SELA) [430] used in the SELA-routing distributed system [8]. They make use of several types of agents: (i) mobile ant-like (perception) agents to collect non-local information, and that are generated according to both on-demand (at session setup time) and proactive generation schemes, and (ii) mobile effectors agents used to tear down paths and to gather specific information about traffic profiles of running applications. The different perception agents are designed after the AntNet-FA ants, however, they are generated with different parameters in order to have different behaviors according to the different task they are involved in.

AntHocNet is a reactive-proactive multipath algorithm for routing in MANETs. Node managers are not really learning agents as it was the case for AntNet+SELA, but rather finite state machines responding more or less reactively to external events and generating the appropriate action and type of ant-like agent. In particular, the design of AntHocNet features: reactive agents to setup paths toward a previously unknown destination, per-session proactive gathering of information, and agents for the explicit management of link failure situations (repair and notification to the neighbors). The fact that a strong reactive component is included in the algorithm is partially due to the fact that in such highly dynamic environments it might be of questionable utility to rely on approaches strongly based on detecting and learning environment's regularities. Some level of learning and proactivity is expected to be of some usefulness, but at the same time the core strategy is expected to be a reactive one.

The chapter has also extensively reviewed related work in the domain of ant-inspired algorithms for routing tasks. The review has pointed out the main different solutions proposed so far, as well as has served to remark the interest that AntNet and, more in general, ACO ideas, have aroused in the routing community.

CHAPTER 8

Experimental results for ACO routing algorithms

This chapter is devoted to the presentation of extensive experimental results concerning AntNet, AntNet-FA, and AntHocNet. The majority of the results concern AntNet and AntNet-FA and refer to the case of *best-effort routing in wired IP networks*. As discussed in Subsection 7.1.1, failure events are not taken into account, the focus being on the study of traffic-adaptiveness and on the effective use of multiple paths. Some preliminary results concerning AntHocNet and routing mobile ad hoc networks are also briefly reported. However, AntHocNet is still under development and testing, such that these results must be considered as preliminary ones.

All the results refer to *simulations*. The algorithms have not been tested yet on real networks, even if some explicit interest in this sense has been shown by some important network companies. Actually, a large part of the research in the field of networks is based on simulation studies, due to the difficulties related to the use of effective mathematical tools to study the properties of routing algorithms under realistic assumptions (e.g., see the extensive discussions in [325]). The simulator that has been implemented and used for the experiments has the characteristics discussed in Subsection 7.1.1.

No mathematical proofs concerning the specific properties of the algorithms are provided. In fact, a sound mathematical treatment for the non-stationary case would require the knowledge of the dynamics of the input traffic processes, but this is precisely what is a priori not known. If such knowledge is available, an optimal routing approach (Subsection 6.4.1) should be definitely adopted (at least in the wired case, while the situation is way more complex in the wireless and mobile case). Moreover, such dynamics should be representable in terms of low-variance stationary stochastic processes to be amenable to effective mathematical analysis. On the other hand, it is not immediately obvious which type of convergence or stability is appropriate to be studied under conditions of non-stationarity.

Concerning convergence under conditions of traffic stationarity, the general proofs of convergence provided for the more general ACO case by Stützle and Dorigo [403] and Gutjahr [214, 215, 216] can intuitively guarantee that a basic form of an AntNet-like algorithm, will converge to an optimal solution. Where the optimality must be intended in some sense in between the optimal shortest path solution and the optimal routing solution. In fact, the optimization strategy followed by AntNet and AntNet-FA can be situated in between the criteria used by these two approaches. However, ACR algorithms are intended for traffic-adaptive routing. The case of static or quasi-static traffic patterns is in a sense not of interest. Generic ACR algorithms are not expected to be really competitive with more classical approaches for the static case.

The experimental results will show that under all the considered situations AntNet and AntNet-FA clearly outperform all the considered five competitors, that include several traffic-adaptive state-of-the-art algorithms. Tests have been run for a number of different traffic patterns and for five different topologies (two real-world ones, two artificially designed ones, and one set of randomly generated ones containing up to 150 nodes). End-to-end packet delay and throughput have been considered as measures of performance, as is common practice.

Also the results for AntHocNet show very good performance under a variety of scenarios and up to networks with 2000 nodes. Instead of varying the traffic load, as we have done in the wired case, we have rather changed the conditions affecting the topological characteristics of the network (i.e., the number of nodes, their average density, and their mobility). In spite of the fact that some scenarios were not really suited for a reactive-proactive multipath approach, AntHocNet shows performance (in terms of packet delivery ratio and end-to-end delays) always comparable or better than AODV [349], the popular state-of-the-art algorithm that we have used for comparison. In this case, we ran simulation tests using *Qualnet* [354], a commercial network simulator providing packet-level resolution and realistic implementation of all the network layers, as well as of the radio propagation physics and of the most popular protocols, including AODV.

At the end of the chapter the rationale behind the excellent performance that has been observed is discussed, focusing in particular on AntNet and AntNet-FA and on their differences with respect to classical link-state and distance-vector algorithms.

Organization of the chapter

Section 8.1 is completely devoted to the detailed description of the experimental settings. Its three subsections describe, respectively, the considered network topologies, traffic patterns, and performance metrics.

Section 8.2 describes the set of six state-of-the-art routing algorithms that have been used to evaluate by comparison the performance of AntNet and AntNet-FA. Subsection 8.2.1 discusses the way parameters have been set for all the considered algorithms.

Section 8.3 and its subsections report all the experimental results obtained by simulation. Subsections from 8.3.1 to 8.3.5 report the performance of all the considered algorithms for five different types of network (starting from a simple hand-designed 9-nodes network and ending with 150-node randomly generated networks). Subsection 8.3.6 shows the amount of traffic overhead due to routing packets for all the implemented algorithms, while Subsection 8.3.7 shows the behavior of AntNet as a function of the ant generation rate at the nodes. Subsection 8.3.8 discusses the importance of using adaptive reinforcements in AntNet by showing the performance with and without path evaluation.

Section 8.4 first describes the experimental settings used to test the performance of AntHocNet, which are clearly quite different from those adopted in the wired and no-mobility, case of AntNet and AntNet-FA, and then reports the result from simulation studies. The performance of AntHocNet are compared to those of AODV in function of the number of nodes, of their density, and their mobility. We tested situation from 50 up to 2000 nodes.

The chapter ends with Section 8.5, which contains a discussion of the reasons behind the fact that the implemented ACO algorithms for routing problems outperform all the other considered algorithms.

8.1 Experimental settings

The functioning of a telecommunication network is governed by many components, which may interact in a very complex way. The characteristics of the network itself, those of the input traffic, as well as the metrics used for performance evaluation, are all components which critically affect the behavior of the routing algorithm. In order to cover a wide range of possible and realistic situations, different settings for each of these components have been considered. The following

two subsections show which type of networks and traffic patterns have been used to run the experiments, while the last subsection discusses the performance metrics.

8.1.1 Topology and physical properties of the networks

In the ran experiments, the following networks have been considered: a small hand designed network, two networks modeled on the characteristics of two different real-world networks, one network with somehow regular grid-like topology, and two classes of randomly generated networks with a rather high number of nodes. Their characteristics are described in the following of this subsection. For each network a triple of numbers (μ, σ, N) is given, indicating respectively the mean shortest path distance in terms of hops between all pairs of nodes, the variance of this mean value, and the total number of nodes. These three numbers are intended to provide a measure concerning the degree of connectivity and balancing of the network. It can be in general said that the difficulty of the routing problem, for the same input traffic, increases with the value of these numbers.

- *SimpleNet* (1.9, 0.7, 8) is a small network designed *ad-hoc* to closely study how the different algorithms manage to distribute the load on the three different possible paths. SimpleNet is composed of 8 nodes and 9 bi-directional links, each with a bandwidth of 10 Mbit/s and propagation delay of 1 msec. The topology is shown in Figure 8.1.

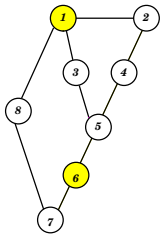

Figure 8.1: SimpleNet. *Numbers within circles are node identifiers. Shaded nodes have a special interpretation described later on. Each edge in the graph represents a pair of directed links. Link bandwidth is 10 Mbit/sec, propagation delay is 1 msec.*

- *NSFNET* (2.2, 0.8, 14) is the old USA T1 backbone (1987). NSFNET is a WAN composed of 14 nodes and 21 bi-directional links with a bandwidth of 1.5 Mbit/s. Its topology is shown in Figure 8.2. Propagation delays range from 4 to 20 msec. NSFNET is a well balanced network.

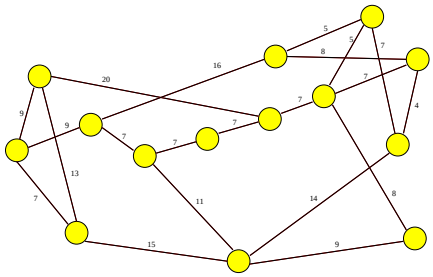

Figure 8.2: NSFNET. *Each edge in the graph represents a pair of directed links. Link bandwidth is 1.5 Mbit/sec, propagation delays range from 4 to 20 msec and are indicated by the numbers reported close to the links.*

- *NTTnet* (6.5, 3.8, 57) is modeled on the former NTT (Nippon Telephone and Telegraph company) fiber-optic corporate backbone. NTTnet is a 57 nodes, 162 bi-directional links network. Link bandwidth is of 6 Mbit/sec, while propagation delays range from around 1 to 5 msec. The topology is shown in Figure 8.3. NTTnet is not a well balanced network.

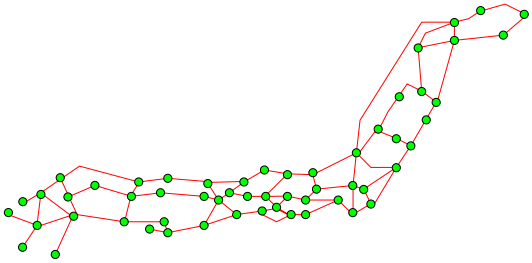

Figure 8.3: NTTnet. *Each edge in the graph represents a pair of directed links. Link bandwidth is 6 Mbit/sec, propagation delays range from 1 to 5 msec.*

- *6x6Net* (6.3, 3.2, 36) is a 36 nodes network with a regular topology and a sort of bottleneck path separating the two equal halves of the network. This network has been introduced by Boyan and Littman [56] in their work on Q-Routing. In a sense, this is a "pathological" network, considered its regularity and the bottleneck path. All the links have bandwidth of 10 Mbit/s and propagation delay of 1 msec.

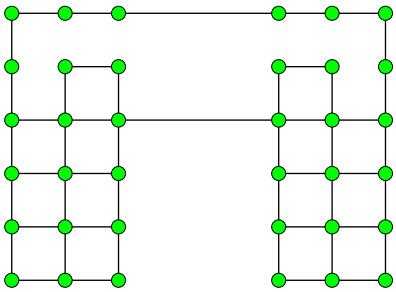

Figure 8.4: 6x6Net. *Each edge in the graph represents a pair of directed links. For all the links the bandwidth is equal to 10 Mbit/sec and the propagation delay is equal to 1 msec.*

- *Random Networks* (4.7, 1.8, 100) and (5.5, 2.1, 150) are two sets of randomly generated networks of respectively 100 and 150 nodes. The level of connectivity of each node has been forced to range between 2 and 5. The reported values for the mean shortest path distances and their variances are averages over the 10 randomly generated networks that have been used for the experiments. Every bi-directional link has the same bandwidth of 1 Mbit/sec, while the propagation delays have been generated in a uniform random way over the interval [0.01, 0.001].

For all the networks the probability of node or link failure is equal to 0. Node buffers are of 1 Gbit, and the maximum time-to-live (TTL) for both data packets and routing packets is set to 15 sec.

8.1.2 Traffic patterns

Traffic is defined in terms of open sessions between pairs of different nodes. Traffic patterns over the whole network depend on the characteristics of each session and on their geographical and temporal distribution.

Each single session is characterized by the number of transmitted packets and by the distribution of their sizes and inter-arrival times. Sessions over the network are characterized by their geographical distribution and by their inter-arrival time. The geographical distribution is controlled by the probability assigned to each node to be selected as a start- or end-point of a session.

Arrival of new sessions at the nodes

The following three different types of basic processes have been used in the experiments to regulate the *arrival* of new sessions at the nodes:

- *Poisson* (P): for each node an independent Poisson process regulates the arrival of new sessions, i.e., sessions' inter-arrival times are negative exponentially distributed.
- *Fixed* (F): at the beginning of the simulation an assigned number of sessions is set up at each node, and they keep sending data for all the remainder of the simulation.
- *Temporary Hot Spots* (TMPHS): in this case some nodes act like temporary hot spots, that is, they generate a heavy load of traffic but only for a short time interval.

Geographical distribution of traffic sessions

The *geographical distribution* of the active sessions all over the network is defined according to one of the following three basic patterns:

- *Uniform* (U): the characteristics of the process that at each node is regulating the arrival of new sessions are the same for all the nodes in the network.
- *Random* (R): the characteristics of the process that at each node is regulating the arrival of new sessions are set up according to the same randomized procedure for all the network nodes.
- *Hot Spots* (HS): some nodes act as hot spots, concentrating a high rate of input/output traffic. In this case, a fixed number of sessions are opened from the hot spots toward all the other nodes.

Complex traffic patterns have been obtained by combining in various ways the above basic patterns for temporal and spatial distribution (e.g., F-CBR, UP, UP-HS). For example, UP traffic means that for each node an identical Poisson process is regulating the arrival of new sessions, while RP means that the Poisson process is different for each node and its characteristics are drawn from a random distribution. UP-HS means that a Hot Spots traffic process is superimposed to a UP traffic, and so on.

Packet stream of traffic sessions

Concerning the characteristics of the bit stream generated by each session, two basic types have been considered:

- *Constant Bit Rate* (CBR): the per-session bit rate is kept constant. Examples of applications of CBR streams are the voice signal in a telephone network, which is converted into a stream of bits with a constant rate of 64 Kbit/sec, and the MPEG1 compression standard, which converts a video signal into a stream of 1.5 Mbit/sec.
- *Generic Variable Bit Rate* (GVBR): the per-session generated bit rate is time varying. The term GVBR is a broad generalization of the VBR term normally used to designate a bit stream with a variable bit rate but with known average characteristics and expected/admitted fluctuations.[1] Here, a GVBR session generates packets whose sizes and inter-arrival times are variable and follow a negative exponential distribution. The information about these characteristics is never directly used by the implemented routing algorithms.

The values used in the experiments to shape traffic patterns are values considered somehow "reasonable" once the current network usage and computing power are taken into account. The mean of the distribution of packet sizes has been set to 4096 bits in all the experiments.

8.1.3 Performance metrics

Results refer to measures of *throughput, packet delays and network resources utilization*. Results for throughput are reported as average values without an associated measure of variance. The inter-trial variability is in fact usually very low, just a few percent of the average value. Network resource utilization is only considered with respect to the routing packets and is reported for a single specific sample instance. Numerical results on packet delays are reported either by displaying the whole empirical distribution or by using the statistic of the 90-th percentile. The empirical distribution shows the complete picture of what happened during the observation interval, but it might be difficult to compare two empirical distributions. On the contrary, the 90-th percentile allows to compactly compare the algorithms on the basis of the upper value of delay they been have able to keep the 90% of the correctly delivered packets. In other works on routing, the mean packet delay is often used as a sufficient statistics for packet delays. However, the mean value is of doubtful significance. In fact, packet delays can spread over a wide range of values. This is an intrinsic characteristics of data networks: end-to-end delays can range from very low values for sessions open between adjacent nodes and connected by fast links, to much higher values in the case of sessions involving nodes very far apart and possibly connected by several slow links. According to this, in general, the empirical distribution of packet delays cannot be meaningfully parametrized in terms of the mean and variance. This is why here the display of the whole empirical distribution and/or the statistics of the 90-th percentile have been used.

Most of the results will show little difference in throughput among the algorithms, while more marked differences will concern end-to-end delays. This is also a consequence of the fact that node buffers are quite large. Therefore, packets can accumulate long delays while suffering little throughput loss due to dropping because of lack of buffer space (however, the 15 seconds TTL put a limit on the maximum delay that a packet can experience). Some experiments conducted with smaller buffers have confirmed the obtained results with differences more marked for what concerns throughput.

[1] The knowledge about the characteristics of the incoming CBR or VBR bit streams is of fundamental importance in networks able to deliver QoS. It is on the basis of this knowledge that the network can accept/refuse the session requests, and, in case of acceptance, allocate/reserve necessary resources.

8.2 Routing algorithms used for comparison

The performance of AntNet and AntNet-FA is compared to that of the following routing algorithms which are taken as representative of the state-of-the-art in the fields of telecommunication and machine learning for both the cases of static and adaptive algorithms.

OSPF (static, link-state): the algorithm considered here is a simplified implementation of *OSPF* [328], the most used Interior Gateway Protocol on the Internet (see Chapter 6). According to the assumptions made for the communication network model as described in Subsection 7.1.1 (no failure situations or topological modifications), the routing protocol here called OSPF does not mirror the real OSPF protocol in its details. It only retains some basic features of OSPF. Link costs are *statically* assigned on the basis of the link physical characteristics. Routing tables are set as the result of the shortest (minimum end-to-end delay) path computation for a sample data packet of 512 bytes. This way of assigning link costs penalizes the implemented version of OSPF with respect to the one used on real networks, where costs are set up by network administrators, who can use additional heuristic and the on-field knowledge they have about local traffic workloads. Since no topological alterations are considered, the periodically flooded link state advertising messages have always the same contents. As a result, the routing tables do not change over time and the algorithm is here labeled as "static", while the real OSPF is a dynamic algorithm in the sense specified in Subsection 6.2.2 (i.e., topology-adaptive).

SPF (adaptive, link-state): this algorithm is a sort of prototype of *link-state algorithms with adaptive link costs*. A similar algorithm was implemented in the second version of ARPANET [304] and in its successive revisions [252] (see Section 6.5). The implementation considered here makes use of the same flooding algorithm used in the real implementation, while link costs are assigned over a discrete scale of 20 values by using the ARPANET *hop-normalized-delay metric*[2] of Khanna and Zinky [252] and the the *statistical window average method* described in [386]. Link costs are computed as weighted averages between short- and long-term real-valued statistics reflecting a cost measure (e.g., utilization, queuing and/or transmission delay, etc.) over time intervals of assigned length. The resulting cost values are then rescaled and saturated by a linear function in order to obtain a value in the range $\{1, 2, 3, \ldots, 20\}$. We have tried also other discrete and real-valued metrics in addition to the discretized hop-normalized-delay, but none of them was able to provide better performance and stability than the hop-normalized-delay one. The reason might be that using a discrete scale reduces the sensitivity of the algorithm but at the same time reduces also undesirable oscillations.

BF (adaptive, distance-vector): is an implementation of the *asynchronous distributed Bellman-Ford* algorithm [26] making use of adaptive cost metrics. The basic structure of the algorithm is the same as described in Subsection 6.4.2.1, while link costs are calculated as in the SPF case, that is, according to [386]. As discussed in Subsection 6.4.2.1, several enhanced versions of the basic Bellman-Ford algorithm can be found in the literature (e.g., the Merlin-Segall [310] and the Extended Bellman-Ford [82]). However, these algorithms mainly focus on the problem of avoiding the *counting to infinity*, or, more in general, the slow convergence recovering from a link failure. These algorithms try to optimize the time necessary to spread the information about the link failure such that the risk of routing inconsistencies is much reduced. On the other hand, concerning dynamic situations other than link

[2] The transmitting node monitors the average packet delay $\overline{d}$ (queuing plus transmission time) and the average packet transmission time $\overline{t}$ over observation windows of assigned time length. From these measures, and assuming an *M/M/1* queue model [26], a cost measure for the link utilization is calculated as $1 - \overline{t}/\overline{d}$.

failures, their behavior is in general equivalent to that of the basic adaptive distributed Bellman-Ford considered here.

Q-R (adaptive, distance-vector): this algorithm is a faithful implementation of the *Q-Routing* algorithm proposed by Boyan and Littman [56]. The algorithm makes use of the *Q-learning* [442] updating rule within a scheme which is an online version of the asynchronous distributed Bellman-Ford algorithm. Q-learning is a popular reinforcement learning algorithm designed for solving Markov decision process without relying on the use of the environment model. Q-Routing learns online the values $Q_k(d,n)$, which are estimates of the time to reach node d from node k via the neighbor node n. The algorithm operates as follows. Upon sending a packet P from node k to neighbor node n with destination d, a back packet P_{back} is immediately generated from n to k. P_{back} carries: (i) the information about the current estimate

$$Q_n^*(d) = \min_{n' \in \mathcal{N}(n)} Q_n(d,n')$$

held at node n about the best expected time-to-go for destination d, and (ii) the sum $T_{k \to n}^P$ of the queuing and transmission time experienced by P to hop from k to n. The sum

$$\hat{Q}_k(d,n) = Q_n^*(d) + T_{k \to n}^P$$

is used to update the k's Q-value for destination d and neighbor n:

$$\Delta Q_k(d,n) = \eta(\hat{Q}_k(d,n) - Q_k(d,n)), \quad \eta \in (0,1] \, .$$

Data packets are routed toward the neighbor currently associated to the Q^* estimate. In the BF scheme running averages for link costs are calculated locally and then broadcast at somehow regular intervals to the neighbors, that, in turn use them to directly update their time-to-go estimates. In the Q-routing scheme, local estimates are sent from one node to another after each packet hop, and time-to-go estimates are updated not by direct value replacement but rather using exponential averaging.

PQ-R (adaptive, distance-vector): this is the *Predictive Q-Routing* algorithm [84], which is an extended and revised version of Q-Routing designed to possibly deal in a better way with traffic variability. In Q-routing, the best link (i.e., the one with the lowest $Q_k(d,n)$) is always deterministically chosen by data packets. Therefore, a neighbor n which has has been associated to a high value of $Q_k(d,n)$, for example because of a temporary high load condition, will never be used again until all the other neighbors will be associated to a worse, that is, higher, Q-value. To overcome this potential problem, PQ-R try to learn a model, called the *recovery rate*, of the variation rate of the local link queues, and makes use of it to probe also those links that, although do no have the lowest $Q_k(d,n)$, shows a high recovery rate. The idea is to give a chance to those links that seem to have recovered from that was only a temporary congestion.

Daemon (adaptive, centralized): this algorithm is defined to provide a measure of an empirical upper bound on the achievable performance. It is a sort of *ideal* algorithm, in the sense that in general it cannot be implemented in practice since is at the same time centralized and makes use of a "daemon" able to instantaneously read the state of all the queues in the network such that shortest path calculations can be done for each packet hop according to the up-to-date overall network status. A more precise upper bound could have been defined by assuming full knowledge on the input traffic processes and then calculating the optimal routing solution. However, such solution would have required to choose traffic patterns amenable to mathematical treatment, and would have ruled out the study of temporary hot spots situations. On the other hand, the Daemon algorithm proposed here can

be applied to any kind of traffic patterns and does not require any additional knowledge. The algorithm works as follows.

Link costs are defined in terms of the depletion time of the corresponding queue (similarly to the strategy adopted in AntNet-FA), and the daemon knows at any time the cost of all the links in the network. In order to assign the next hop node, before each packet hop the algorithm re-calculates the shortest path solution according to the current costs of all network links. The next hop node is then chosen as the one along the current minimum cost path. In a sense, Daemon behaves as an SPF making use of instantaneous and continuous information flooding.

Links costs express the time necessary for a new packet of size s_p to cross the link l given the current queues:

$$C_l = d_l + \frac{s_p}{b_l} + (1-\alpha)\frac{q_l}{b_l} + \alpha\,\frac{\bar{q}_l}{b_l},$$

where d_l is the transmission delay for link l, b_l is its bandwidth, q_l is the current size (in bits) of the queue of link l, and $\bar{q}_l$ is an exponential average of the size of the same link queue. By means of the weight factor α, a weighted average between q_l and $\bar{q}_l$ is in practice calculated in order to take into account both the current and the previous level of congestion along the link (the value of α has been set to 0.4 in all the ran experiments). Clearly, for each recalculation of the shortest paths, the value of s_p is set to zero for all links but the ones connected to the node where the current packet to be routed is located at.

8.2.1 Parameter values

All the implemented algorithms depend on their own set of parameters. For all the algorithms the size of their routing packets and the related elaboration time must be properly set. Settings for these specific parameters are shown in Table 8.1.

Table 8.1: *Characteristics of routing packets for the implemented algorithms, except for the Daemon algorithm, which does not generate routing packets. N_h is the incremental number of hops made by the forward ant, $|\mathcal{N}_n|$ is the number of neighbors of the generic node n, and N is the total number of network nodes.*

	AntNet	AntNet-FA	OSPF, SPF	BF	Q-R, PQ-R
Packet size (byte)	24 + 8N_h	24 + 4N_h	64 + 8$\lvert\mathcal{N}_n\rvert$	24 + 12N	12
Packet processing time (msec)	3	3	6	2	3

These parameters have been assigned on the basis of values used in previous simulation works [5] and/or on the basis of heuristic evaluations taking into consideration information encoding schemes and currently available computing power. The size for AntNet forward ants has been determined as the same size of a BF packet plus 8 bytes for each hop to store the information about the node address and the traveling time. In the case of AntNet-FA only 4 additional bytes are necessary for each hop.

Except for AntNet, the parameters specific to each algorithm have been assigned by using the best settings available in the literature, and/or through a tuning process in order to obtain possibly better results. The length of the time interval between consecutive broadcasting of routing information and the length of the time window to average the link costs are both set to the value of 0.8 or 3 seconds, depending on the experiment, for SPF and BF. The same values have been set to 30 seconds for OSPF. Link costs inside each time window are assigned as the weighted sum between the arithmetic average computed over the window and an exponential average with decay factor equal to 0.9. The obtained values are mapped on a scale of integer values ranging

from 1 and 20, through a linear transformation with slope set to 20. The maximum variation which is admitted from one time step to another is set to 1 (i.e., when costs are updated the difference between the previous and the new cost can be either -1, 0, or 1). For Q-R and PQ-R the transmission of routing information is totally data-driven. The used learning and adaptation rates have been set to the same values reported in the original papers [56, 84].

Concerning AntNet and AntNet-FA, the algorithms appear very robust to internal parameter setting. Actually, the same set of values have been used for all the different experiments that have been ran, without really going through a process of fine tuning experiment by experiment. Most of the parameter values have been previously reported in the text at the moment the parameter was discussed. Therefore, they are not repeated here. The ant generation interval at each node has been set to 0.3 seconds; Subsection 8.3.6 discusses the robustness of AntNet with respect to this important parameter. Regarding the parameters of the statistical model $\mathcal{M}$: the value of η, weighting the number of the samples considered in the model (see Equation 7.2), has been set to 0.005, the c factor for the expression of w (see Page 207) has been set to 0.3, and the confidence level factor z to 1.70, implying a confidence level of approximately 65%.

8.3 Results for AntNet and AntNet-FA

For each of the networks considered in Subsection 8.1.1, performance has been studied for various traffic patterns configurations. Most of the experiments refer to the following two classes of situations: (i) increasing traffic loads, from low congestion to near-saturation, (ii) low traffic loads temporarily modified by the addition of near-saturation input traffic. Most of the results concern only AntNet and not AntNet-FA. A direct comparison between the two algorithms is given on the two sets of larger, randomly generated networks.

In spite of the variety of the considered situations, both in terms of networks and traffic patterns, it cannot be claimed that an exhaustive experimental analysis has been carried out. The universe of the possible situations of some practical interest is inevitably too large. According to this fact, in the following the performance of algorithm A is said to be better of the performance of algorithm B only if there is a clear difference between their performance. "Clear" is somehow intended in the sense that it is not necessary to run any statistical test to assert the difference in performance, since it is immediately *evident* that a significant difference it exists. In this sense, in the following algorithms are ranked only when they provide really different performance, otherwise, considered the large number of different traffic situations that are *not* included in the test suite, it is not reasonable to predict (or assert) any difference in the expected performance. This approach could be formally translated into a statistical test procedure in which the acceptation threshold is set to a very high value. In the following, since the differences in performance between AntNet (and AntNet-FA) and the other algorithms are usually quite striking, the use of statistical tests has been judged as unnecessary in practice.

All reported data are *averaged over 10 trials*. Each trial corresponds to *1000 seconds of activity in a real network*. One thousand seconds should represent a time interval long enough to expire all transients and to get enough statistical data to evaluate the behavior of the routing algorithm. Before being fed with data traffic, the algorithms run for 500 seconds in conditions of absence of data traffic. In this way, each algorithm can initialize the routing tables according to the physical and topological characteristics of the network. The choice of 500 seconds is completely arbitrary. Usually a much shorter time is necessary for the algorithms to converge. However, since this phase usually lasted only few seconds in real time, it was unnecessary to define any more optimized initialization procedure.

The values of the parameters specifying the characteristics of the *traffic generation processes* are given in the figure captions, with the following meaning: MSIA is the mean of the inter-

arrival time distribution for new sessions for what concerns the the Poisson (P) case, MPIA stands for the mean of the distribution of the inter-arrival time of data packets. In the CBR case, MPIA indicates the fixed packet production rate. HS is the number of nodes acting as hot-spots, and MPIA-HS is the equivalent of MPIA for the hot-spot sessions. In the following, when not otherwise explicitly stated, the shape of the bit streams in each session is assumed to be of GVBR type.

This is a summary of the observed results:

- Under input traffic corresponding to a *low load* with respect to the available network resources, all the tested algorithms show similar performance. In this case, according to the above reasonings, it is very hard to assess whether an algorithm is significantly better than another or not. According to this fact, results for very low traffic loads are not reported.
- Under *high, near saturation, loads*, all the tested algorithms are more or less able to deliver the input throughput. That is, in most of the cases, all the generated traffic is routed without incurring in major data losses. On the contrary, the resulting distributions for what concerns the packet delays show remarkable differences among the different algorithms. In some sense, almost all the input traffic is delivered, but the paths followed by the packets are significantly different among the different algorithms, such that final end-to-end delays show large variations.
- Significant differences are also evident in the case of *temporary saturation loads*. Saturation is the situation in which heavy packet losses and/or high values for packet delays are observed. Therefore, saturation can be accepted only as a temporary situation. If it is not, structural changes to the network characteristics, like adding new and faster connection lines, should be in order. The traffic load bringing a network in saturation can be in principle estimated according to the physical characteristics of the network itself. However, saturation usually appears before the physical limit is reached. In this case, is the routing algorithm which is responsible to bring the network in saturation. Therefore, for the same physical network, saturation will happen in correspondence to different traffic loads according to the different routing algorithm which is in use. The reference saturation load considered in the reported experiments has been that observed for AntNet.

8.3.1 SimpleNet

Experiments with SimpleNet have been designed to closely study how the different algorithms manage to distribute the load on the different available paths. In these experiments all the traffic, of F-CBR type, is directed from node 1 to node 6 (see Figure 8.1), and the traffic load has been set to a value higher than the bandwidth of a single link, so that it cannot be routed efficiently over a single path.

Results regarding throughput (Figure 8.5a) evidence a marked difference among the algorithms. This difference is determined by the joint effect coming from the little number of nodes and the stationarity of the traffic workload. AntNet is the only algorithm able to deliver almost all the generated data traffic: its throughput, after a short transient phase, approaches very closely the level of that delivered by the Daemon algorithm. PQ-R attains a steady value approximately 15% inferior to that obtained by AntNet. The other algorithms behave poorly, stabilizing on values of about 30% inferior to those provided by AntNet. In this case it is rather clear that AntNet is the only algorithm able to exploit at best all the three available paths $\langle 1, 8, 7, 6\rangle, \langle 1, 3, 5, 6\rangle, \langle 1, 2, 4, 5, 6\rangle$ to distribute the data traffic without generating counterproductive oscillations. The AntNet's stochastic routing of the data packet plays in this case a fundamental role to achieve results of better quality. Results for throughput are confirmed by those for packet delays, reported in the graph of Figure 8.5b. The differences in the empirical

distributions for packet delays reflect approximatively the same proportions evidenced in the throughput case.

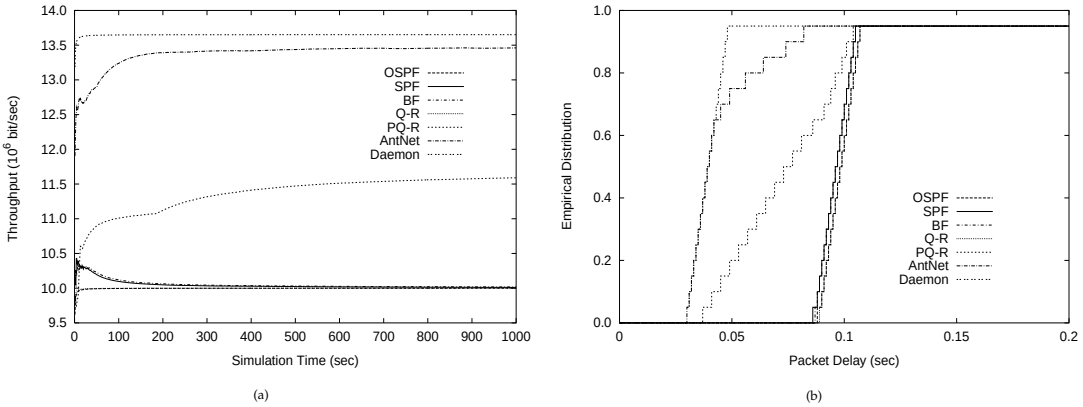

Figure 8.5: SimpleNet*: Comparison of algorithms for F-CBR traffic directed from node 1 to node 6 (MPIA = 0.0003 sec). (a) Throughput, and (b) empirical distribution of the end-to-end data packet delays.*

8.3.2 NSFNET

Performance on NSFNET have been tested using UP, RP, UP-HS and TMPHS-UP traffic patterns. For all the considered cases, all the algorithms behave similarly with respect to throughput, while major differences are apparent for packet delays. For each one of the UP, RP and UP-HS cases, we have ran a sequence of five distinct experiments sets, each of ten repeated trials. The generated workload is gradually increased at each set starting from an initial low workload until a near-saturation one is reached. The workload is increased by reducing the time interval between sessions' inter-arrivals.

UP TRAFFIC - WORKLOAD RANGING FROM LOW TO NEAR-SATURATION

In this case, differences in throughput (Figure 8.6a) are small: the best performing algorithms are BF and SPF, which can attain performance of only about 10% inferior to those of Daemon, and of the same amount better than those of AntNet, Q-R and PQ-R,[3] while OSPF behaves slightly better than these last ones. Concerning delays (Figure 8.6b) the picture is rather different: it seems that all the algorithms but AntNet have been able to produce a quite good throughput at the expenses of a much worse result for end-to-end delays, as it will also happen in the majority of the ran experiments. OSPF, Q-R and PQ-R show really poor results (delays of order 2 or more seconds have to be considered as very high values, even if considering the 90-th percentile of the distribution). BF and SPF show a similar behavior, with performance of order 50% worse than those obtained by AntNet and of order 65% worse than Daemon.

[3] In these and in some of the experiments presented in the following, PQ-R's performance is slightly worse than that of Q-R. This seems to be in contrast with the results presented by the PQ-R's authors in the article where they introduced PQ-R [84]. A possible explanation of such a behavior the fact that: (i) their link recovery rate has been designed having in mind a discrete-time system while in the ran simulations time is a continuous variable, and (ii) the experimental and simulation conditions are rather different, due to the fact that the simulator they have used is quite far from being realistic. Moreover, in the paper the way traffic patterns are generated is not specified.

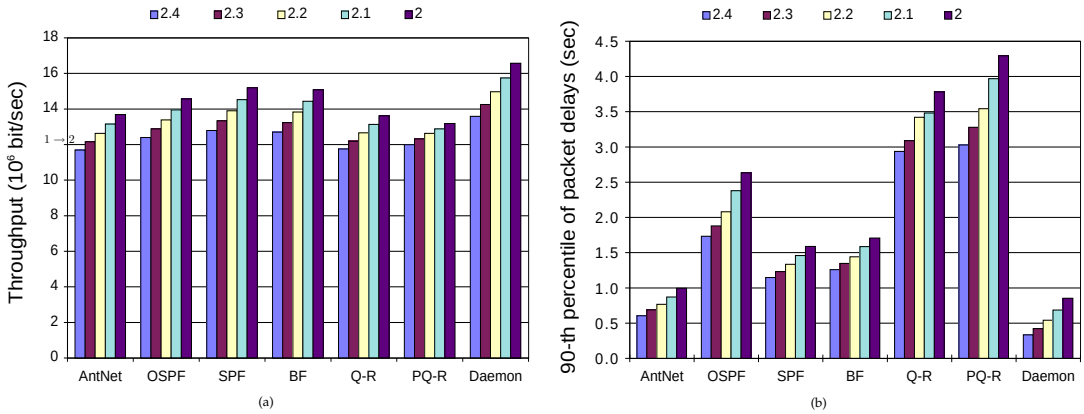

Figure 8.6: NSFNET*: Comparison of algorithms for increasing workload under UP traffic conditions. The load is increased by reducing the MSIA value from 2.4 to 2 seconds (MPIA = 0.005 sec). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

RP TRAFFIC - WORKLOAD RANGING FROM LOW TO NEAR-SATURATION

In the RP case (Figure 8.7a), the throughput generated by AntNet, SPF and BF look very similar, although AntNet shows a slightly better performance. OSPF and PQ-R behave only slightly worse than SPF and BF, while Q-R is the worst performing algorithm. Daemon is able to obtain only slightly better results than AntNet. Again, looking at the results for end-to-end delays (Figure 8.7b), OSPF, Q-R and PQ-R perform quite bad, while SPF's results are a bit better than those of BF but of order 40% worse than those of AntNet. Daemon is in this case far better, which might be an indication of the fact that the testbed was rather difficult.

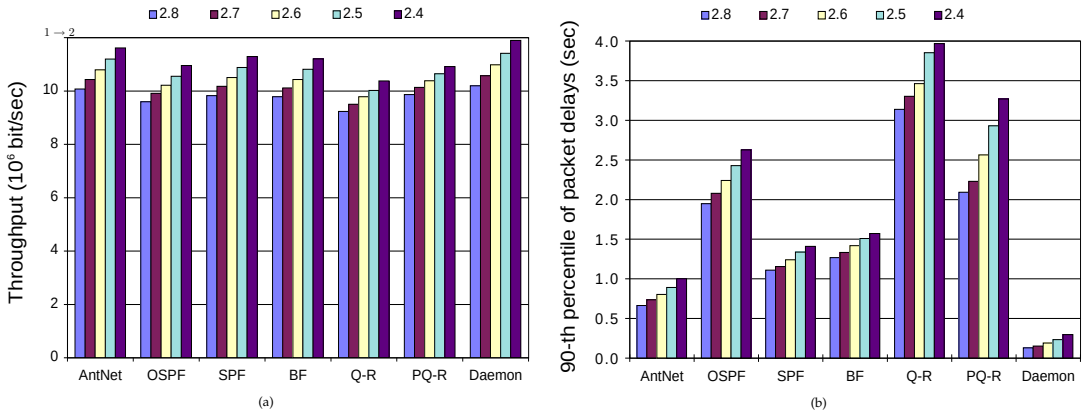

Figure 8.7: NSFNET*: Comparison of algorithms for increasing workload under RP traffic conditions. The load is increased by reducing the MSIA value from 2.8 to 2.4 seconds (MPIA = 0.005 sec). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

UP-HS TRAFFIC - WORKLOAD RANGING FROM LOW TO NEAR-SATURATION

In this case packet burstiness is set to a much lower level than in the previous cases due to the additional load given by the hot spots. Throughput (Figure 8.8a) for AntNet, SPF, BF, Q-R and Daemon are very similar, while OSPF and PQ-R clearly obtain much worse results. Again (Figure 8.8b), end-to-end delays results for OSPF, Q-R and PQ-R are much worse than those for the other algorithms (they are so much worse that actually they do not fit in the chosen scale).

AntNet is still the best performing algorithm. In this case, differences with SPF are of order 20%, and of 40% with respect to BF. Daemon performs about 50% better than AntNet and scales much better than AntNet, which, again, indicates that the testbed was rather difficult.

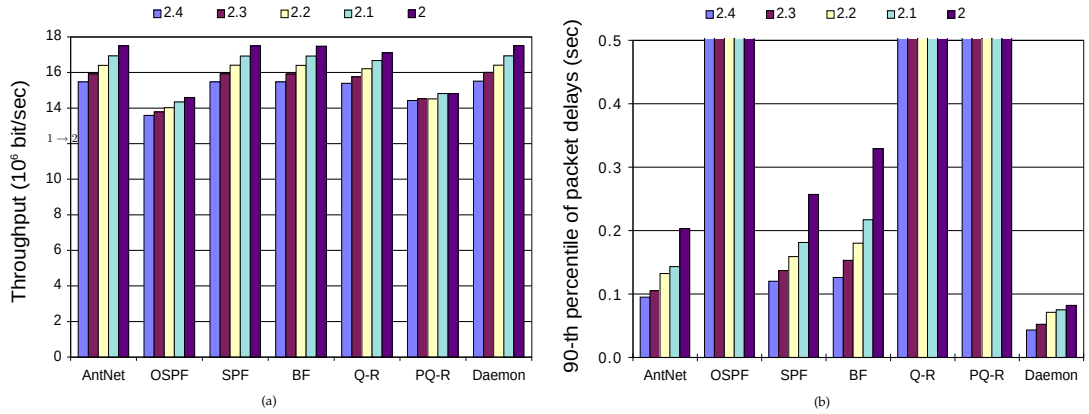

Figure 8.8: NSFNET*: Comparison of algorithms for increasing load for UP-HS traffic. The load is increased by reducing the MSIA value from 2.4 to 2.0 seconds (MPIA = 0.3 sec, HS = 4, MPIA-HS = 0.04 sec). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

TMPHS-UP TRAFFIC

Figure 8.9 shows how the algorithms behave in the case of a TMPHS-UP situation. At time $t = 400$ four hot spot nodes are turned on and superimposed to the existing, light, UP traffic (packet burstiness for the HS sessions is much higher than that for the UP sessions). The transient is held for 120 seconds. The graph shows the details of the typical answer curves observed during the experiments. Reported values are a sort of "instantaneous" values for throughput and packet delays, computed as the average of the values observed during moving time windows of 5 seconds. Most of the algorithms have a similar, very good, performance as far as throughput is concerned. Only OSPF and PQ-R, lose a few percent of the packets during the transitory period. The graph of packet delays confirms previous results. SPF and BF show similar behavior, with performance of about 20% worse than AntNet and 45% worse than Daemon. The other three algorithms show a big out-of-scale jump, clearly being unable to adapt properly to the sudden increase in the workload.

8.3.3 NTTnet

The same set of experiments run on NSFNET have been also run on NTTnet. Results are in this case even sharper than those obtained with NSFNET: AntNet clearly outperforms all the competitor algorithms.

UP TRAFFIC - WORKLOAD RANGING FROM LOW TO NEAR-SATURATION

In the UP case, as well as, in the RP and UP-HS cases, differences in throughput are not significant (Figures 8.10a, 8.11a and 8.12a). All the algorithms, with the exception of OSPF, are able to deliver more or less the same throughput as Daemon does.

Concerning delays (Figure 8.10b), differences between AntNet and the other algorithms are of one or more orders of magnitude. AntNet's packet delays are very close to those obtained by Daemon. SPF is the third best, but its performance is about of 80% worse than that of AntNet.

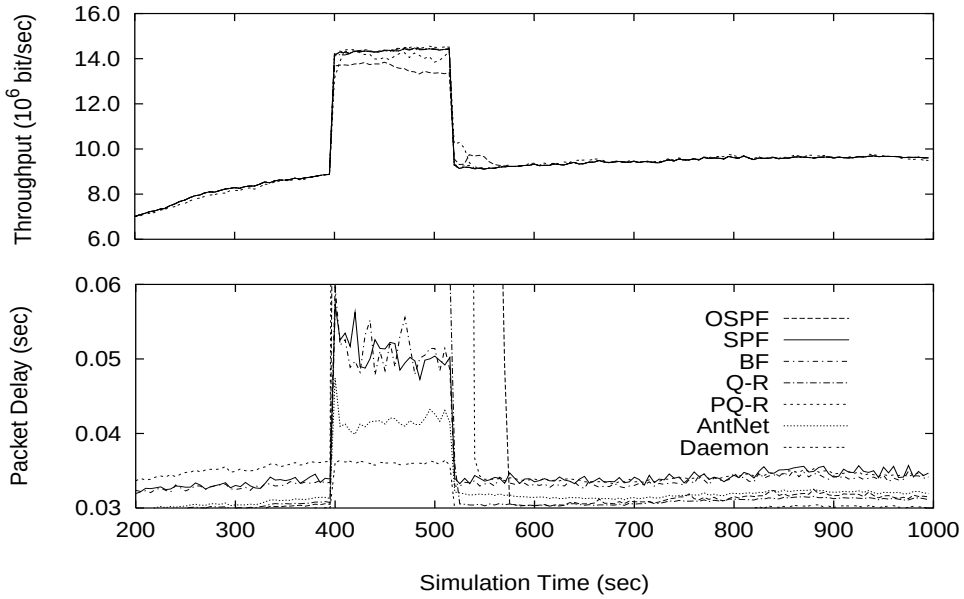

Figure 8.9: NSFNET*: Comparison of algorithms for transient saturation conditions with TMPHS-UP traffic (MSIA = 3.0 sec, MPIA = 0.3 sec, HS = 4, MPIA-HS = 0.04). (a) Throughput, and (b) end-to-end packet delays, both averaged on the basis of the values observed over 5 seconds moving windows.*

BF performs similarly to SPF but slightly worse than it. AntNet, SPF and BF all show a regular behavior with the increase of the workload, as expected. On the contrary, Q-R, PQ-R and OSPF show a somehow more irregular behavior. This might be due to the fact that the considered workload is already in saturation zone for these algorithms, such that irregular dynamics may in general appear. The performance of Q-R and PQ-R are close to each other, with Q-R slightly better than PQ-R, but however much worse than that of AntNet. OSPF response, both for throughput and packet delays is very poor: the end-to-end delays 90-th percentile for the case of the heaviest workload is about 50 times that of AntNet.

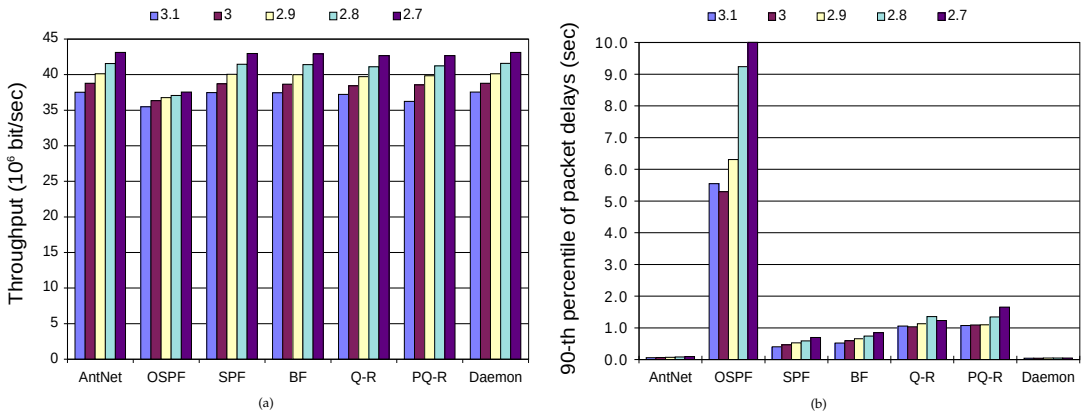

Figure 8.10: NTTnet*: Comparison of algorithms for increasing workload under UP traffic conditions. The load is increased by reducing the MSIA value from 3.1 to 2.7 seconds (MPIA = 0.005 sec). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

RP TRAFFIC - WORKLOAD RANGING FROM LOW TO NEAR-SATURATION

Again, the differences in the delivered throughput are little for all the considered algorithms but OSPF, which delivers about the 15% less than the others.

The picture for end-to-end delays is similar to that observed for the case of UP traffic. The most notable difference lies in the even larger difference between AntNet's performance, which is again very close to that of Daemon, and that of the other algorithms. SPF, which is the best performing algorithm among the competitors, has a value of 90-th percentile of end-to-end delays about ten times bigger than that provided by AntNet. The difference between SPF and BF performance is of about 10%. PQ-R and Q-R's delays are of the same order of magnitude, with PQ-R performing slightly better, but still about the double of those of SPF. OSPF values for packet delays are completely out-of-scale with respect to those of all the other algorithms.

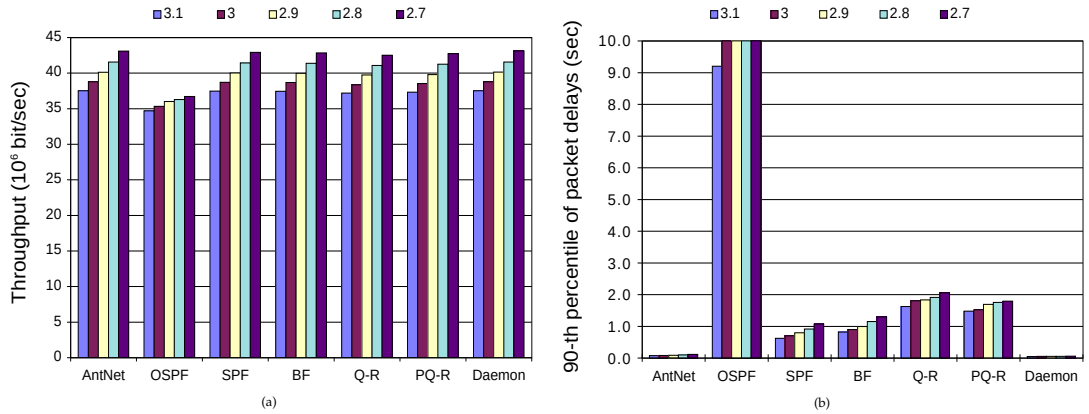

Figure 8.11: NTTnet: *Comparison of algorithms for increasing workload under RP traffic conditions. The load is increased by reducing the MSIA value from 3.1 to 2.7 seconds (MPIA = 0.005 sec). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

UP-HS TRAFFIC - WORKLOAD RANGING FROM LOW TO NEAR-SATURATION

In this case too, throughput results are practically the same for all the algorithms but OSPF, which shows a throughput more than 25% less of that of the other algorithms (Figure 8.12a). In addition to this, OSPF also shows an irregular behavior, with the throughput which is either decreasing or showing a slow increase with the increasing of the offered workload. Actually, this apparently contradictory behavior is counterbalanced by the behavior for packet delays, which is increasing with the increase of the workload, as Figure 8.12b shows. In this case, due to the heavy traffic load, a lot of packets are discarded, such that the throughput is decreasing, but, at the same time, OSPF is able to forward the surviving packets to their destinations with decreasing delays. The other algorithms show a more regular behavior. Again, the performance of AntNet for packet delays are very close to that obtained by Daemon, and much better than that of the other competitors. SPF and BF show similar results, while Q-R performs comparably but in a more irregular way. Finally, PQ-R is in this case the third best algorithm, with delays about four times larger than those of AntNet.

TMPHS-UP TRAFFIC

The TMPHS-UP sudden load variation experiment (Figure 8.13) confirms the previous results. OSPF is not able to adequately follow the variation both for throughput and delays. On the other hand, all the other algorithms are able to follow the sudden increase in the offered throughput,

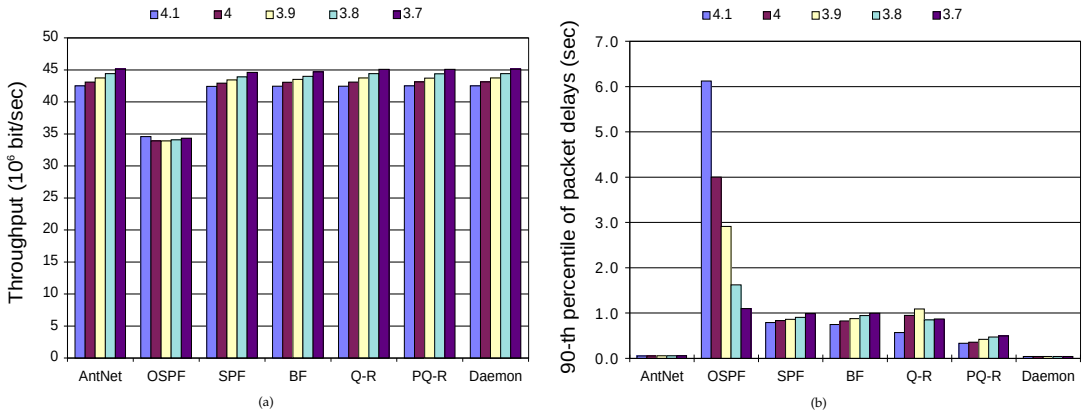

Figure 8.12: NTTnet*: Comparison of algorithms for increasing workload under UP-HS traffic conditions. The load is increased by reducing the MSIA value from 4.1 to 3.7 seconds (MPIA = 0.3 sec, HS = 4, MPIA-HS = 0.05 sec). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

but only AntNet and Daemon show a clearly regular behavior. Differences in packet delays are striking. AntNet performance is very close to that obtained by Daemon (the respective curves are practically superimposed at the scale used in the figure). Among the other algorithms, SPF and BF are the best ones, although their response is rather irregular and, in any case, much worse than that of AntNet. OSPF and Q-R are out-of-scale and show a curve with a very delayed recovering. PQ-R, after a huge jump, which takes the graph out-of-scale in the first 40 seconds after hot spots are turned on, shows a trend approaching that of BF and SPF.

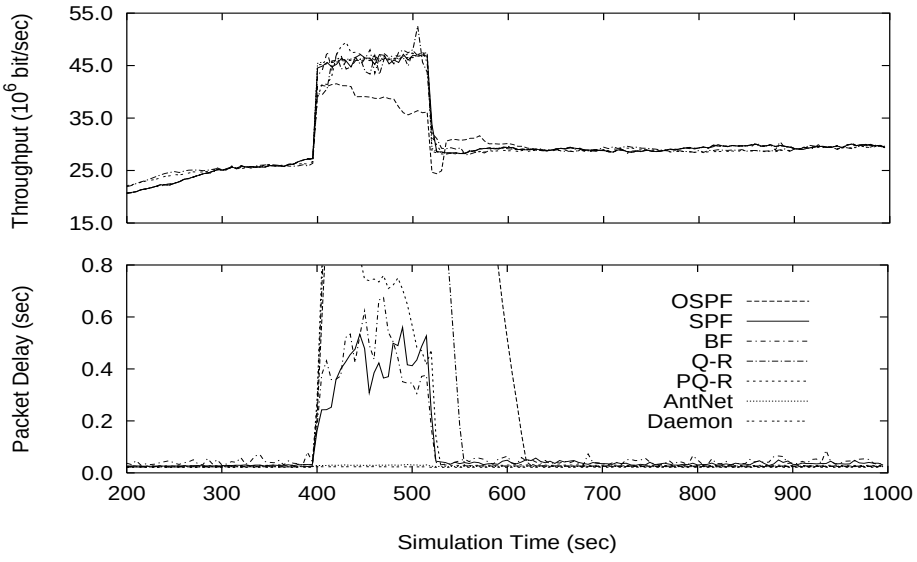

Figure 8.13: NTTnet*: Comparison of algorithms for transient saturation conditions with TMPHS-UP traffic (MSIA = 4.0 sec, MPIA = 0.3 sec, HS = 4, MPIA-HS = 0.05). (a) Throughput, and (b) end-to-end packet delays averaged over 5 seconds moving windows.*

8.3.4 6x6Net

The results reported in this subsection refer to the 6x6Net network. Only one single traffic situation is considered for this network which has been designed by the authors of Q-R. The big difference in the observed performance between AntNet and the other considered algorithms, as well as the fact that this network is a rather pathological one, did not stimulated the need for a more extensive set of experiments. The considered traffic situation is of type UP with a medium level of workload.

In Figure 8.14a throughput curves generated by AntNet, Q-R and PQ-R are quite similar even if AntNet shows slightly better performance. The other three algorithms, and OSPF in particular, are able to deliver much less throughput.

As usual, AntNet's packet delays are much lower than those of all the other algorithms (Figure 8.14b). The second best algorithm is OSPF, whose throughput was on the other hand the worst one. All the other algorithms show quite poor performance: their packet delays distribution cannot even be fully represented on the reported scale of up to 2 seconds.

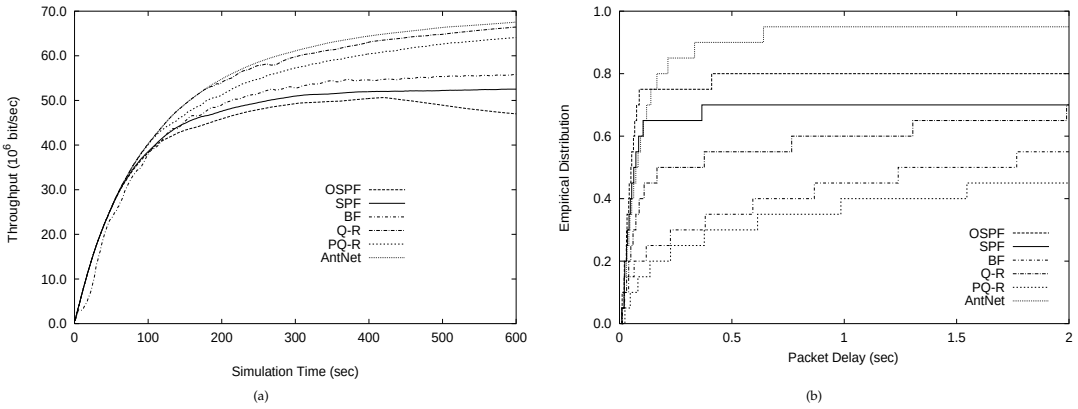

Figure 8.14: 6x6net*: Comparison of algorithms for traffic situation of type UP with medium level workload (MSIA=1.0, MPIA=0.1). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

8.3.5 Larger randomly generated networks

This subsection reports the experimental results for the case of randomly generated networks. The number of nodes of the networks is significantly larger than in the previous cases. Reported data are the average over 10 trials, where for each trial a different randomly generated network has been used.

Results also concern the performance of AntNet-FA. The performance of all the other algorithms considered so far but Daemon are also reported. Daemon has been excluded because it was too demanding from a computational point of view due to the high number of nodes.

100-NODES RANDOM NETWORKS - UP TRAFFIC

Figure 8.15 shows the experimental results for a set of 100-nodes randomly generated networks under heavy UP workload. In this case all the algorithms have been able to deliver the same amount of throughput, while, once again, differences in the distribution of end-to-end delays are striking. AntNet-FA is by far the best performing algorithm, followed by AntNet, while all the competitors perform about 30%-40% worse.

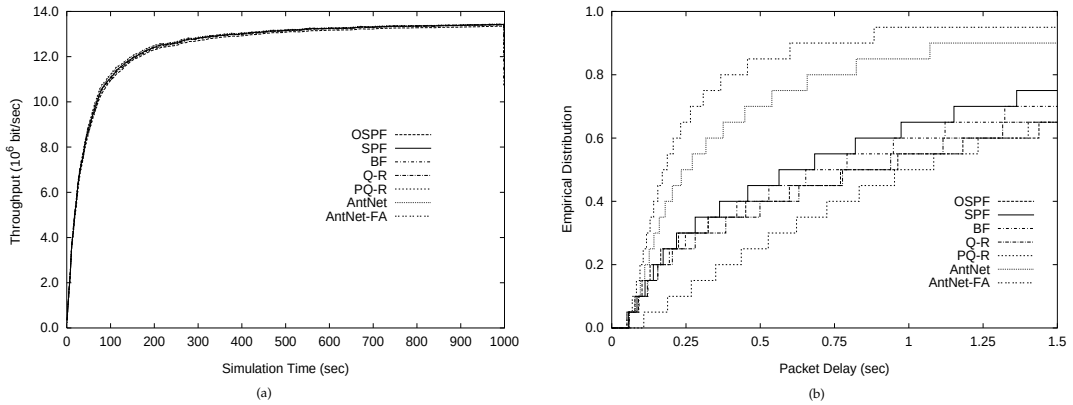

Figure 8.15: 100-Nodes Random Networks*: Comparison of algorithms for heavy UP traffic conditions. Average over 10 trials using a different randomly generated 100-node network in each trial (MSIA=15.0, MPIA=0.005). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

150-NODES RANDOM NETWORKS - RP TRAFFIC

Figure 8.16 reports the experimental results for a set of 150-nodes randomly generated networks under heavy RP workload. In this case differences are significant for both throughput and packet delays.

Only AntNet, AntNet-FA and SPF are able to follow the generated throughput without losses, OSPF behaves only slightly worse, while all the other algorithms can only deliver a throughput about 35% lower.

Concerning packet delays, AntNet-FA is again by far the best performing algorithm. AntNet is the second best one, but it keeps delays much higher than AntNet-FA, about four times higher considering the 90-th percentile. SPF keeps delays much higher than AntNet-FA, and about 60% higher than AntNet on the 85th percentile. BF follows, but it had much worse performance on throughput. OSPF, Q-R and PQ-R perform rather poorly.

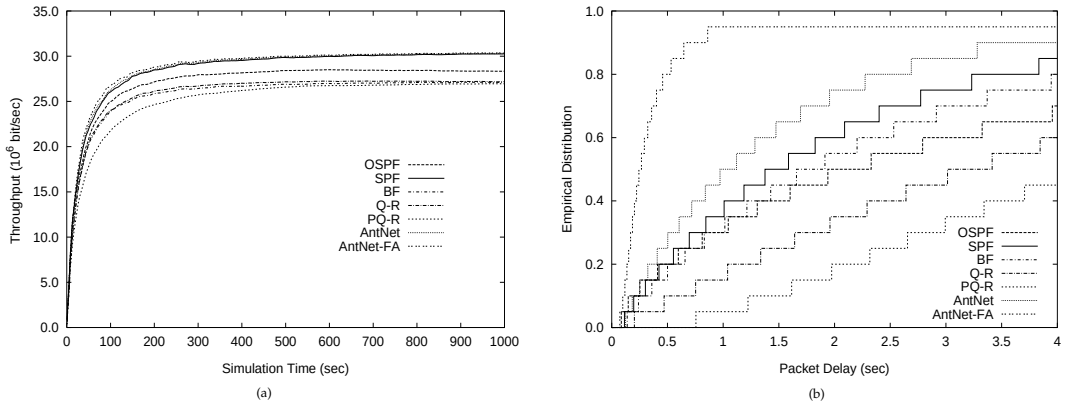

Figure 8.16: 150-Nodes Random Networks*: Comparison of algorithms for heavy RP traffic conditions. Average over 10 trials using a different randomly generated 150-node network in each trial (MSIA=10.0, MPIA=0.005). (a) Throughput, and (b) 90-th percentile of the empirical distribution of the end-to-end data packet delays.*

These results on randomly generated networks confirm the AntNet's excellent performance even on large network instances.[4] Even more interesting is the fact that actually AntNet-FA per-

[4] One of the world leaders in network technologies, CISCO, suggests not to put more than few hundred nodes on the

forms much better than AntNet itself. The difference in performance seems to increase with the size of the network. This behavior can be explained in terms of the higher reactivity of AntNet-FA with respect to AntNet. In AntNet-FA forward ants do not wait in the data queues. Accordingly, information is collected and propagated faster, and it is also more up-to-date with respect to the current network status, Clearly, these characteristics become more and more important as the diameter of the network grows and paths become longer and longer. In these cases, AntNet can show very long delays in gathering and releasing traffic information across the network, making completely out-of-date the information used to update the local traffic models and the routing tables.

8.3.6 Routing overhead

Table 8.2 shows the results concerning the overhead generated by routing packets. For each algorithm the network load generated by the routing packets is reported as the ratio between the bandwidth occupied by all the routing packets and the total available network bandwidth. Each row in the table refers to a previously discussed experimental situation (Figures 8.5, 8.6 to 8.8, and 8.10 to 8.12). Routing overhead is computed for the experiment corresponding to the case of the highest workload in the series.

Table 8.2: Routing Overhead*: ratio between the bandwidth occupied by all the routing packets and the total available network bandwidth. All data are scaled by a factor of* 10^{-3}.

	AntNet	OSPF	SPF	BF	Q-R	PQ-R
SimpleNet - F-CBR	0.33	0.01	0.10	0.07	1.49	2.01
NSFNET - UP	2.39	0.15	0.86	1.17	6.96	9.93
NSFNET - RP	2.60	0.15	1.07	1.17	5.26	7.74
NSFNET - UP-HS	1.63	0.15	1.14	1.17	7.66	8.46
NTTnet - UP	2.85	0.14	3.68	1.39	3.72	6.77
NTTnet - RP	4.41	0.14	3.02	1.18	3.36	6.37
NTTnet - UP-HS	3.81	0.14	4.56	1.39	3.09	4.81

All data are scaled by a factor of 10^{-3}. Data in the table show that the routing overhead with respect to the available bandwidth is in practice negligible for all the considered algorithms. Among the adaptive algorithms, BF shows the lowest overhead, closely followed by SPF. AntNet generates a slightly bigger consumption of network resources, but this is widely compensated by the much better performance it provides. AntNet-FA, which is not show in the table, generates slightly less overhead. Q-R and PQ-R produce an overhead a bit higher than that of AntNet. The routing load caused by the different algorithms is function of many factors, specific to each algorithm. Q-R and PQ-R are data-driven algorithms: if the number of data packets and/or the length of the followed paths grows (either because of topology or bad routing), so will do the number of generated routing packets. BF, SPF and OSPF have a more predictable behavior: the generated overhead is mainly function of the topological properties of the network and of the generation rate of the routing information packets. AntNet produces a routing overhead depending on the ants generation rate and on the length of the paths along they travel. AntNet-FA improves over AntNet since forward ants do not need to carry crossing times.

same hierarchical level given the current routing protocols and technologies. In this sense, 150 is already a reasonably high value for the number of nodes.

8.3.7 Sensitivity of AntNet to the ant launching rate

In AntNet and AntNet-FA, The ant traffic can be roughly modeled in the terms of a set of additional traffic sources, one for each network node, producing rather small data packets (and related sort of acknowledgment packets, the backward ants) at a constant bit rate. In general, ants are expected to travel over rather "short" paths and their size grows of 8 bytes at each hop during the forward (AntNet ants) or backward (AntNet-FA) phase. Therefore, each *ant traffic source* represents, in general, an additional source of light traffic. Of course, they can become heavy traffic sources if the ant launching rate is dramatically raised up. Figure 8.17 shows the sensitivity of AntNet's performance with respect to the ant launching rate and versus the generated routing overhead.

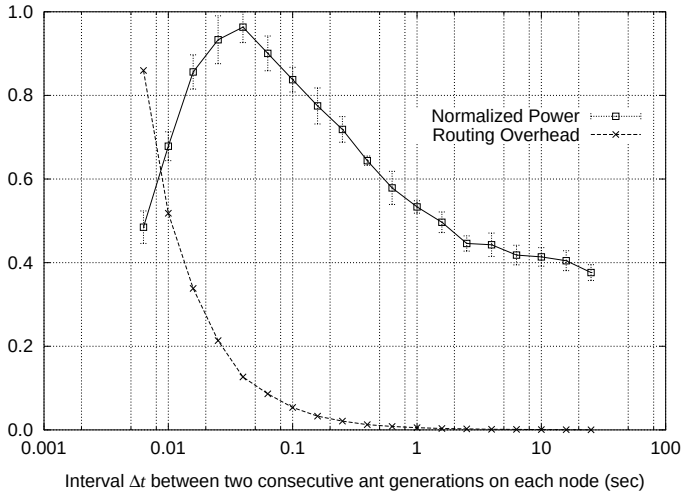

Figure 8.17: AntNet normalized power vs. routing overhead. Power *has been defined as the ratio between the delivered throughput and the 90-th percentile of packet delay. This value has been normalized between* $(0, 1]$ *by dividing it for the highest value of power obtained during the 10 trials of the experiment.*

For the sample case of a UP traffic situation on NSFNET (previously studied in Figure 8.6) the interval Δt between two consecutive ant generations at a node is progressively decreased (Δt is the same for all nodes). Δt values are sampled at regularly spaced points over a logarithmic scale ranging from about 0.006 to 25 seconds. The lower, dashed curve interpolates the generated routing overhead expressed, as before, as the fraction of the available network bandwidth used by routing packets. The upper, solid curve plots data for the obtained *power* normalized to its highest value as observed during the ten trials of the experiment. The power is defined here as the ratio between the delivered throughput and the packet delay. The value used for delivered throughput is the throughput value at time 1000 averaged over ten trials, while for packet end-to-end delay the 90-th percentile of the empirical distribution is used.

From the figure it is quite clear that using a very small Δt determines an excessive growth of the routing overhead, with consequent reduction of the algorithm power. Similarly, when Δt is too large, the power slowly diminishes and tends toward a plateau because the number of ants is not enough to generate and maintain up-to-date statistics of the network status. In between these two extreme regions, a wide range of Δt intervals gives raise to quite similar and rather good power values. In these cases, the routing overhead is practically negligible but the number of ants is enough to provide satisfactory performance. This figure strongly support the conjecture that AntNet, and, more in general, ACR algorithms, can be quite robust to internal parameter settings.

8.3.8 Efficacy of adaptive path evaluation in AntNet

At a first glance, the mechanisms used by AntNet and AntNet-FA to assign reinforcement values might seem over-complicate. Subsection 7.1.4 has discussed in depth the need for such mechanisms, and, accordingly, the need to maintain at each node a local model for the network-wide traffic patterns.

In Figure 8.18 we report the outcome of a simple experiment that compare the performance of AntNet without path evaluation (i.e., making use of an assigned constant value for the reinforcements whatever path is followed), with the performance of the usual AntNet, making use of path evaluation and therefore of possibly non-constant reinforcements.

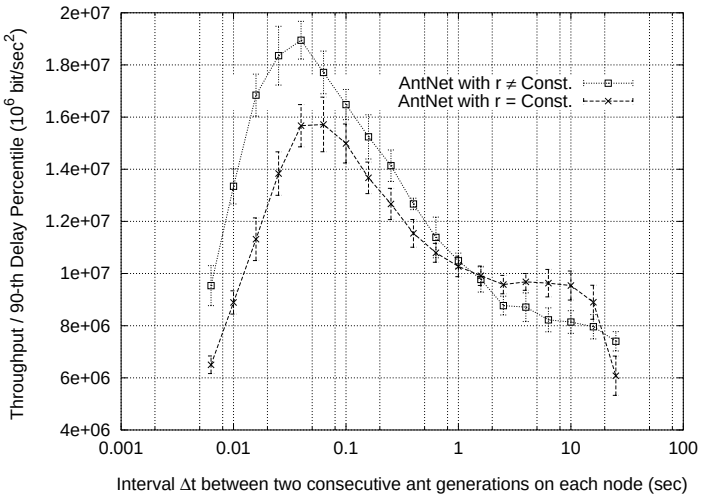

Figure 8.18: Constant *vs.* non-constant reinforcements*: AntNet power for increasing values of ant generation rates at each node. The results are averages over 10 trials for UP traffic conditions on NSFNET.*

In the case of the use of constant reinforcements, what is at work in AntNet is the implicit reinforcement mechanism due to the differentiation in the ant arrival rates explained at Page 217. Ants traveling along faster paths will arrive at a higher rate than other ants, hence their paths will receive a higher cumulative reward. In AntNet-FA this effect is much reduced. The figure shows that, in spite of its simplicity, the implicit reinforcement mechanism is already quite effective but, in the range of the ant generation rates corresponding to the higher power values, the difference between the two algorithms can reach about 30%. Such a difference surely justifies the additional complexity associated to path evaluation and to the use of non-constant reinforcements.

8.4 Experimental settings and results for AntHocNet

We report in this section some preliminary experimental results for AntHocNet. As simulation software we have used *Qualnet* [354], a discrete-event packet-level simulator developed by Scalable Networks Inc. as a follow-up of GloMoSim, which was a shareware simulator designed at University of California, Los Angeles. Qualnet is specifically optimized to simulate large-scale MANETs, and comes with correct implementations of the most important protocols for all the network layers and for routing in particular. We have compared the performance of AntHocNet with *Ad Hoc On-Demand Distance Vector (AODV)* [349] (with local route repair and expanding ring search, e.g., [268]), a state-of-the-art MANET routing algorithm and a de facto standard. AODV is a purely reactive approach. Single-path routes are established on-demand and on the

basis on a minimum-hop metric. Only the nodes along the path used by the application maintain routing information toward the destination, such that in practice a virtual-circuit is established and end-to-end signaling is used tear up and down the path, as well as to rebuild the path in case of broken links.

Most of our simulation scenarios, except for the scalability study scenario which is taken from [268], are derived from the base scenario used in [61], which is so far a constant reference in the current MANET literature, even if it is quite pathological and not really general. In this base scenario 50 nodes are randomly placed in a rectangular area of $1500 \times 300\ m^2$. Within this area, the nodes move according to the random waypoint model [236]: each node randomly chooses a destination point and a speed, and moves to this point with the chosen speed. After that it stops for a certain pause time and then randomly chooses a new destination and speed. The maximum speed in the scenario is 20 meters/sec and the pause time is 30 seconds. The total length of the simulation is 900 seconds. Data traffic is generated by 20 constant bit rate (CBR) sources sending one 64-byte packet per second. Each source starts sending at a random time between 0 and 180 seconds after the start of the simulation, and keeps sending until the end. At the physical layer we use a two-ray signal propagation model. The transmission range is around 300 meters, and the data rate is 2 Mbit/sec. At the MAC layer we use the popular 802.11 DCF protocol.

In this scenario nodes are densely packed, such that from one side there is a high probability of radio collisions but from the other side it is always possible to easily find a short route to a destination. The average path length is about two hops and the average number of neighbors of a node is about ten (due to the dimensions of the node area vs. the radio range and the fact that random waypoint movements tend to concentrate nodes in the central zone of the area). This is clearly a scenario that well match the characteristics of a purely reactive algorithm like AODV since it is relatively easy and fast to build or re-build a path while at the same time is important to keep low the routing overhead in order to reduce the risk of radio collisions. Moreover, since it is quite hard to find multiple (and good) radio-disjoint paths for the same destination given the high node density, the AODV's single-path strategy minimizing the hop number appears as the most suitable one. On the other hand, AntHocNet is a hybrid, reactive-proactive, algorithm for multi-path routing using both end-to-end delay and hop metrics to define the paths. In order to study the behavior of the two algorithms under this reference scenario but also under possibly more interesting and challenging conditions involving longer path lengths and less dense networks, we have performed extensive simulations changing the pause time, the node area and the number of nodes. For each new scenario, 5 different problems have been created, by choosing different initial placements of the nodes and different movement patterns. The reported results, in terms of delivery ratio (fraction of sent packets which actually arrives at their destination) and end-to-end delay, are averaged over 5 different runs (to account for stochastic elements, both in the algorithms and in the physical and MAC layers) on each of the 5 problems.

Since AntHocNet generate considerably more routing packets than AODV, we have also made a further study increasing the number of nodes up to 2000 in order to study the overall scalability of the approach.

Increasing number of hops and node sparseness

We have progressively extended the long side of the simulation area. This has a double effect: paths become longer and the network becomes sparser. The results are shown in Figure 8.19. In the base scenario, AntHocNet has a better delivery ratio than AODV, but a higher average delay. For the longer areas, the difference in delivery ratio becomes bigger, and AODV also looses its advantage in delay. If we take a look at the 99-th percentile of the delay, we can see that the

decrease in performance of AODV is mainly due to a small number of packets with very high delay.This means that AODV delivers packets with a very high delay jitter, that might a problem in terms of QoS. The jitter could be reduced by removing these packets with very high delay, but that would mean an even worse delivery rate for AODV.

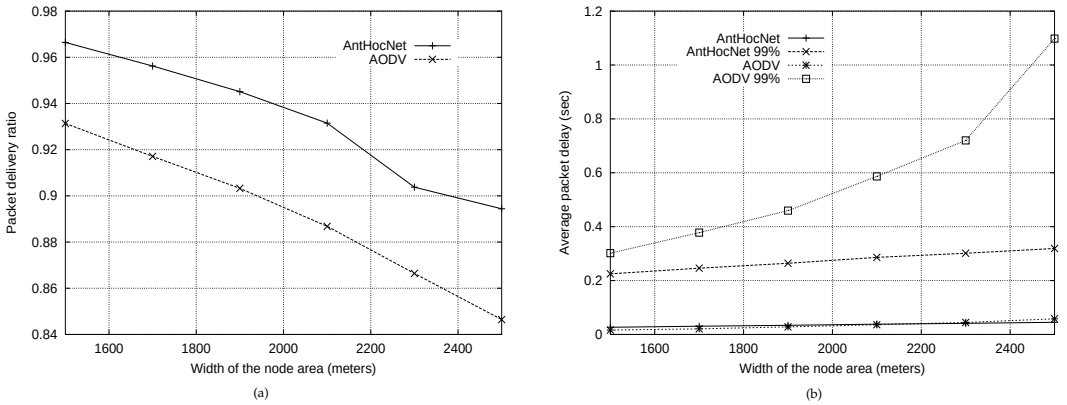

Figure 8.19: Increasing the length of the horizontal dimension of the node area: *(a) Delivery ratio, and (b) average and 99-th percentile of the packet end-to-end delays. On the x-axis is reported the increasing size of the long edge of the node area, while the other edge is fixed at 300 meters. That is, starting from the base scenario of* $1500 \times 300\ m^2$*, and ending at* $2500 \times 300\ m^2$*.*

The performance trend shown in these set of experiments will be also confirmed by the following ones: in all scenarios AntHocNet gives always better delivery ratio than AODV, while for the simpler scenarios it has a higher average delay than AODV but a lower average delay for the more difficult ones. The better performance on delivery ratio likely comes from the multipath nature of AntHocNet. The construction of multiple paths at route setup, and the continuous search for new paths with proactive ants ensures that there are often alternative paths available in case of route failures, resulting in less packet loss and quicker local recovery from failure. On the other hand, the use of multiple paths means also that not all packets are sent over the minimum-delay path, such that the resulting average delay might be slightly higher. However, since AODV relies on just one path, delays can become very bad when this path becomes inefficient or invalid, a situation that is more likely to happen in the more difficult scenarios.

Increasing node mobility

We have changed the level of mobility of the nodes, varying the pause time between 0 seconds (all nodes move constantly) and 900 seconds (all nodes are static). The terrain dimensions were kept on $2500 \times 300\ m^2$, like at the end of the previous experiment (results for $1500 \times 300\ m^2$ were similar but less pronounced). Figure 8.20 shows a trend similar to that of the previous experiment. For easy situations (long pause times, hardly any mobility), AntHocNet has a higher delivery ratio, while AODV has lower delay. As the environment becomes more difficult (higher mobility), the difference in delivery becomes bigger, while the average delay of AntHocNet becomes better than that of AODV. Again, the 99-th percentile of AODV shows that this algorithm delivers some packets with a very high delay. Also AntHocNet has some packets with a high delay (since the average is above the 99-th percentile), but this number is less than 1% of the packets.

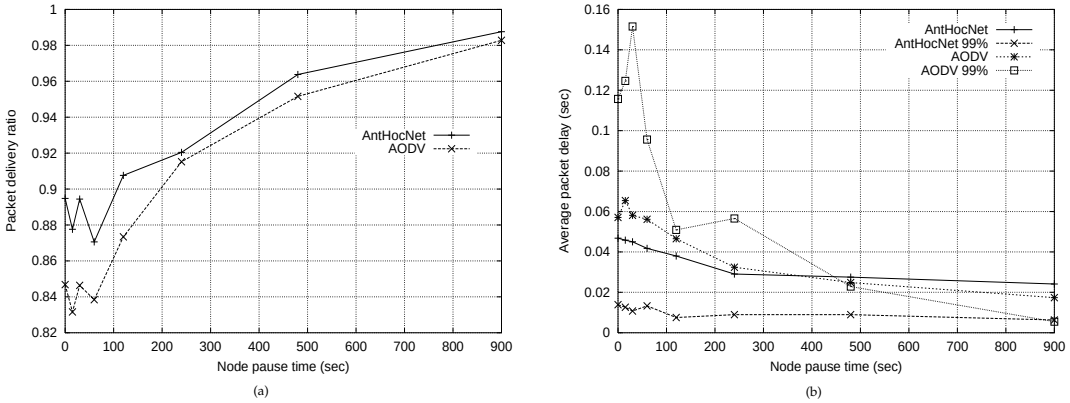

Figure 8.20: Changing node pause time: *(a) Delivery ratio, and (b) average and 99-th percentile of the packet end-to-end delays. For pause time equal to 0 seconds nodes keep moving, while for pause time equal to 900 seconds node do not move at all.*

Increasing both node area and number of nodes.

The scale of the problem is increased maintaining an approximately constant node density: starting from 50 nodes in a $1500 \times 300\ m^2$ area, we have multiplied both area edges by a scaling factor and the number of nodes by the square of this factor. The results, presented in Figure 8.21, show again the same trend: as the problem gets more difficult, the advantage of AntHocNet in terms of delivery rate increases, while the advantage of AODV in terms of average delay becomes a disadvantage. Again this is mainly due to a number of packets with a very high delay.

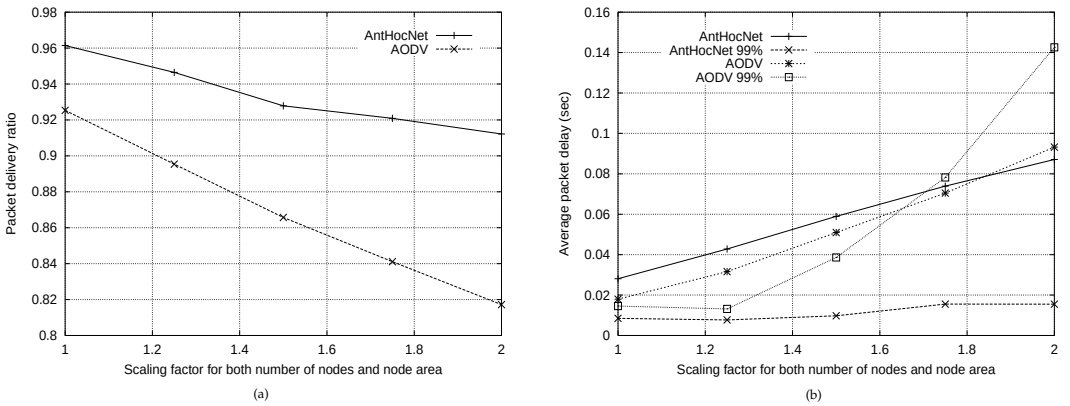

Figure 8.21: Scaling both node area and number of nodes: *(a) Delivery ratio, and (b) average and 99-th percentile of the packet end-to-end delays. On the x-axis is reported the scaling factor for the problem starting from the base scenario of 50 nodes and* $1500 \times 300\ m^2$ *(e.g., for a scaling factor of* $\sigma = 2$ *the number of nodes is* $50\sigma^2 = 200$ *and the area becomes* $(1500\sigma) \times (300\sigma) = 3000 \times 600\ m^2$*).*

In another set of similar experiments we have increased both the size of the node area (starting from $1000 \times 1000\ m^2$) and the number of nodes in the same way as in [268]. That is, such that node density stays approximately constant while increasing the number of nodes up to 2000. The number of traffic sessions is maintained constant to 20, but this time the data rate is of 4 packets/sec and each packet has a payload of 512 bytes. Due to the high computational times necessary for the simulations, we have limited the simulation time to 500 seconds, and only 3 trials per experiment point have been ran. Results in Figure 8.22 confirms the previous trend. AntHocNet always delivers a higher number of packets, while it keeps delays at a much lower

level than AODV (results of the 99-th percentile are not shown since they would be out-of-scale, however, they confirm the datum for the average). This results show the *scalability* of the approach, that, in spite of the higher number of routing packets generated with respect to AODV, is able to scale up its performance. However, for large networks more results and experiments are definitely necessary, such that results reported in Figure 8.22 must be considered as very preliminary

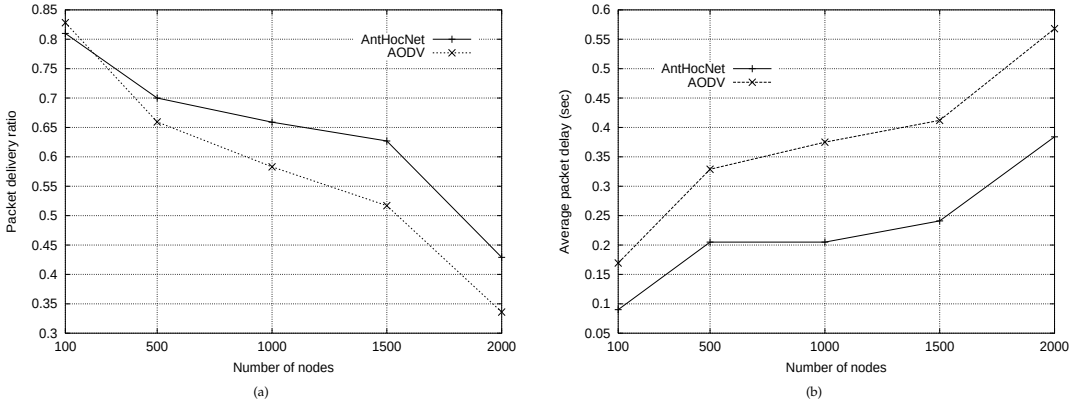

Figure 8.22: Increasing both node area and number of nodes up to a large network: *(a) Delivery ratio, and (b) average and 99-th percentile of the packet end-to-end delays. On the x-axis is reported the number of nodes. The node area is also scaled such that a node has approximately always the same of number of neighbors (about 7).*

8.5 Discussion of the results

For all the experiments reported in the previous Section 8.3, the performance of AntNet and AntNet-FA has been excellent.[5] AntNets have always provided comparable or much better performance than that of the considered competitor algorithms (of course, made exception for Daemon). The overall result can be seen as statistically significant, given that the experimental testbed, even if far from being exhaustive, has considered a set of several different realistic situations.

The positive experimental results confirm the validity of the design choices of AntNets and support the discussions of the previous chapter that have pointed out the several possibilities for improving current routing algorithms. The good performance that also AntHocNet has shown for the more challenging case of mobile ad hoc networks up to large and very dynamic scenarios further support the general validity of the ACR approach.

In the following, the reasons behind the excellent performance of AntNets are discussed by comparing the design characteristics of AntNets to those of the other algorithms that have been taken into account. Some of the issues that are going to be considered have been already discussed at a more general level in the previous chapter. While most of the arguments apply also to AntHocNet, the discussions will almost exclusively refer to AntNets.

PROACTIVE INFORMATION GATHERING AND EXPLORATORY BEHAVIOR

AntNets fully exploit the unique characteristics of networks consisting in the possibility of using the network itself as a simulator to run realistic *Monte Carlo simulations* concurrently with the

[5] Hereafter, AntNet and AntNet-FA are also jointly indicated with the term "AntNets" in order to shorten the notation.

normal operations of the system. In this sense, AntNets have been among the first examples of routing systems based on the distributed and repeated generation of mobile agents for *(pro)active* gathering of information on routing paths, and making at the same time use of a stochastic policy to direct agents' actions and realize data-independent *exploration*. AntNets make use of both the local (passive) observation of data flows (the status of the link queues) and the explicit generation of mobile agents aimed at collecting specific non-local information about network paths. Moreover, a built-in *variance reduction* effect in the Monte Carlo estimates is determined by: (i) the way ant destinations are assigned, which are biased toward the use of the most frequently requested destinations for data packets, and (ii) the fact that the ant decision policy combines both current (link queues) and past (pheromone) traffic information. In this way, paths are not sampled uniformly, but according to the specific characteristics of the traffic patterns. On the other hand, the experimental results have shown how effective can be the use of online simulation, as well as, how small, and in practice negligible, can be the extra overhead generated by the ant-like agents.

In all the other considered algorithms, routing tables are built through measures coming only from the passive observation of data flows. No actions are generated to explicitly gather additional information. Non-local information is passively acquired from the other nodes, which proactively send their local estimates. On the other side, concerning exploration, in OSPF, SPF and BF there is no exploration at all, while in Q-R and PQ-R exploration is tightly coupled to data traffic and is of local nature.

INFORMATION MAINTAINED AT THE NODES

The type of information maintained at each node to build the routing tables and the way this information is possibly propagated are key components of every routing algorithm. All the considered adaptive algorithms can be seen as maintaining at each node two types of information: a model $\mathcal{M}$ representing either the local link costs or some local view of the global input traffic, and a routing table $\mathcal{T}$, which stores either distances-to-go or the relative goodness of each local routing choice. SPF and BF make use of a model $\mathcal{M}$ to maintain smoothed averages of the local link costs, that is, of the distances to the neighbor nodes. Therefore, $\mathcal{M}$ is a model concerning only local components. This local information is, in turn, used to build the local routing tables and is also sent to other nodes. In Q-R the local model $\mathcal{M}$ is a fictitious one, since is the raw traveling time observed for each hop of a data packet which is directly used as a value to update the entries of $\mathcal{T}$, which are, on the contrary, exponential averages of these observed transmission times. PQ-R makes use of a slightly more sophisticated model with respect to Q-R, storing also a measure of the utilization of each link. All these algorithms propagate part of the locally maintained information to other nodes, which, in turn, make use of this information to update their routing tables. SPF nodes send link costs, while BF, Q-R and PQ-R nodes send distance estimates (built in different way by each algorithm). In SPF and BF the content of each $\mathcal{T}$ is updated at regular intervals through a sort of "memoryless strategy": the new entries do not depend on the old values, that are discarded. On the contrary, Q-R and PQ-R make use of a Q-learning rule to incrementally build exponential averages.

AntNets show characteristics rather different from those of the other algorithms. The local model $\mathcal{M}$ maintains for each destination adaptive information about the time-to-go. The contents of $\mathcal{M}$ allow to evaluate and reinforce the ant paths in function of what has been observed so far. The pheromone table $\mathcal{T}$ is a stochastic decision matrix which is updated according to the values of the assigned reinforcements and of the current entries. Moreover, ants decisions are also based on a model $\mathcal{L}$ of the depletion dynamics of the link queues, that is, on the current local status of the link queues. The pheromone tables are, in turn, used to build the data-routing tables, which are still stochastic matrices but somehow more biased toward the best choices.

It is apparent that AntNets make use of a richer repertoire of information than the other algorithms: (i) adaptive probabilistic models for distance estimates and to score the relative goodness of each local choice, (ii) a simple model to capture the instantaneous state of the link queues, (iii) a stochastic decision matrix for both data and ants routing. Each node in AntNets is an actor-critic agent, learning on the basis of a wise balance between long- and short-term estimates, in order to cope with the intrinsic variability of the input traffic. The use of several components at each node has also the great advantage of reducing the criticality of each of these components, since a sort of task distribution happens. In some sense, AntNets' architecture is not only using richer information but is also more robust with respect to those of the considered competitors.

STOCHASTIC MULTIPATH ROUTING BASED ON PHEROMONE TABLES

All the tested algorithms but AntNets use a *deterministic routing policy*. More in general, AntNets are the only algorithms really relying on the use of stochastic components. It has already discussed the fact that stochastic policies seem in general to be more appropriate to deal with non-Markov, non-stationary problems. In AntNets, being based on proactive Monte Carlo simulation (the ants), *stochasticity* is at the core of the mechanisms for building the routing tables. But what makes AntNets even more different from the other algorithms is the combined effect of using different stochastic matrices to route ants and data. In this way, the ants have a built-in *exploration* component that allow them to explore new routes using the ant-routing tables, while at the same time, data packets make use of the data-routing tables, *exploiting* the best of the paths discovered so far. None of the other considered algorithms keep on separate levels exploration and exploitation. Actually, none of them really do exploration.

The use of a stochastic policy to route data packets result in a better distribution of the traffic load over different paths, with a resulting better utilization of the resources and *automatic load balancing* (the experiments with SimpleNet well show this fact). In this sense, AntNets are the only algorithms which implement *multipath routing*. Data packets of a same session are concurrently spread over multiple paths, if there are several equally good possible paths that can be used. Moreover, AntNets do not have to face the problem of deciding a priori how many paths have to be used. Since a relative goodness is assigned to every local choice, the chosen transformation function of Equation 7.10 and the random selection mechanism of a next hop automatically select the appropriate number of paths that will be actually used by data packets. On the other hand, since every choice is locally scored by means of pheromone (and status of link queues) and this score is proactively and repeatedly refreshed with some non null probability, even those paths that are not actually used by data are made available and can be used in case of sudden congestion along the best paths, and/or in case of failure.

The fact that a *bundle* of paths is made available and scored by the ants, such that it can be used for both multipath and *backup routing* is quite important to explain the good performance shown by *AntHocNet* over AODV. The construction of multiple paths at route setup, and the continuous search for new paths with proactive ants ensures that there are often alternative paths available in case of route failures, resulting in less packet loss and in the higher delivery ratio shown by AntHocNet in all the experiments. Moreover, local repair of paths after link failures is made easier and quicker by the presence of the presence of multiple paths. On the other hand, AntHocNet has a higher average delay than AODV for the simpler scenarios, but a lower average delay for the more difficult ones (while results are always usually better if the 99-th percentile is considered). Again, this is in line with the multipath nature of AntHocNet: since it uses different paths simultaneously, not all packets are sent over the shortest path, and so the average delay will be slightly higher. On the other hand, since AODV relies on just one path, delays can become very bad when this path becomes inefficient or invalid. This is especially

likely to happen in difficult scenarios, with longer paths, lower node density or higher mobility, rather than in the dense and relatively easy base scenario.

Concerning the contents of the routing tables, while AntNets maintains *probabilistic measures of the relative goodness of the local choices*, all the other algorithms maintain for each choice a *distance estimate* to the destinations. The main difference consists in the fact that distance values are absolute values, while the AntNets' goodness measures are only relative values. For instance, the values in the routing table of BF say that passing by neighbor n the distance to d is t_n seconds, while passing by m is t_m seconds. This is quite different from what happens in AntNets, whose data-routing table contains $[0, 1]$ normalized entries like τ_{nd} and τ_{md} that indicate the relative goodness of choosing the route through m with respect to the route through n. On the other hand, AntNets maintain estimates for the expected and best traveling times toward the destinations from the current node in the model $\mathcal{M}$. However, distances are calculated considering the whole set of neighbors, and not separately for each neighbor.

Under rapid traffic fluctuations, it might result quite hard to keep track for each neighbor of the precise absolute distances toward the destinations of interest (as well as misleading conclusions can be easily drawn as shown in Subsection 7.1.4). On the other hand, is expected to be more robust to deal with only relative measure of goodness assigned on the basis of all the sampled information available, which is the approach followed in AntNets.

Another advantage of using normalized goodness values instead of absolute distances, consists in the possibility of *exploiting the arrival rate* of the ants as a way to assign implicit reinforcements to the sampled paths. In fact, after the arrival of a backward ant, the routing table is always updated, kicking up the path that has been followed by an amount that depends on both the quality of the path and its estimated goodness. It is not obvious how the same effect could be obtained by using routing tables containing distance estimates. In fact, in this case each new sampled value of a traveling time would have to be added to the statistical estimate, that would then oscillate around its expected value without inducing an arrival-dependent cumulative effect.

It is interesting to remark that the use of probabilistic routing tables whose entries are learned in an adaptive way by changing on positive feedback and ignoring negative feedback, is reminiscent of older *automata* approaches to routing in telecommunication networks, already mentioned in the previous chapters. In these approaches, a learning automaton is usually placed on each network node. An automaton is defined by a set of possible actions and a vector of associated probabilities, a continuous set of inputs and a learning algorithm to learn input-output associations. Automata are connected in a feedback configuration with the environment (the whole network), and a set of penalty signals from the environment to the actions is defined. Routing choices and modifications to the learning strategy are carried out in a probabilistic way and according to the network conditions (e.g., see [334, 331]). The main difference lies in the fact that in AntNet the ants are part of the environment itself, and they actively direct the learning process towards the most interesting regions of the search space. That is, the whole environment plays a key, active role in learning good state-action pairs, while learning automata essentially learn only by induction.

ROBUSTNESS TO WRONG ESTIMATES

AntNets do not propagate local estimates to other nodes, as all the other algorithms do. The statistical estimates and the routing table maintained at each node are updated after each ant experiment without relying on any form of estimate *bootstrapping*. AntNets rely on pure Monte Carlo sampling and updates. Each ant experiment affects only the estimates and the routing table entries relative to the nodes visited by the ant, and local information is updated on the basis of the "global" information collected by traveling ants along the path, implicitly reducing in this

way the variance in the local estimates. All these characteristics make AntNets particularly robust to wrong or out-of-date estimates. The information associated to each ant experiment has a limited impact. If this characteristic can be a disadvantage under conditions of traffic stationarity, it is an advantage under the more usual conditions of non-stationarity.

On the contrary, in all the other algorithms local estimates (of either link costs or distances) are propagated to other nodes and might affect the decisions concerning all the different destinations. Accordingly, an estimate which is wrong in some sense, is propagated overall the network, and can have a globally negative impact. How bad this is for the algorithm performance depends on how long the effect of the wrong estimate stay in the network. In particular, for SPF and BF this is a function of frequency updates and of the propagation time throughout the network, while for Q-R and PQ-R is a function of the learning parameters.

The issue of information bootstrapping in BF, Q-R, PQ-R and AntNets can be considered under a more general perspective. In fact, as it has been already pointed out, AntNets are a sort of parallel replicated Monte Carlo systems. As it has been shown by Singh and Sutton (1996), a first-visit Monte Carlo simulation system (only the first visit to a state is used to estimate its value during a trial) is equivalent to a batch *temporal difference* (TD) [413] method with replacing traces and decay parameter $\lambda = 1$. In some sense, TD(1) is a pathological case of the TD class of algorithms, since its results are the same obtained by a first-visit Monte Carlo, that is, without any form of bootstrapping. The advantage of TD(1) over Monte Carlo is the fact that it can be run as an online method [414, Chapter 7]. Although AntNets can be seen as first-visit Monte Carlo simulation systems, there are some important differences with the type of Monte Carlo considered by Singh and Sutton and, more in general, by other works in the field of reinforcement learning. The differences are mainly due to the differences in the characteristic of the considered classes of problems. In AntNets, outcomes of experiments are both used to update local models able to capture the global network state, which is only partially observable, and to generate a sequence of stochastic policies. On the contrary, in the Monte Carlo system considered by Singh and Sutton, the outcomes of the experiments are used to compute reduced maximum-likelihood estimates of the expected mean and variance of the states' returns (i.e., the total cost or payoff following the visit of a state) of a Markov process. Actually, batch Monte Carlo methods learn the estimates that minimize the mean-squared error on the used training set, while, for example, batch TD(0) methods find the maximum-likelihood estimates for the parameters of the underlying Markov process [414, Page 144]. In spite of the differences between AntNet and TD(1), the weak parallel with TD(λ) methods is rather interesting, and it allows to compare some of the characteristics of AntNets with those of BF, Q-R and PQ-R by reasoning within the TD class of algorithms. In fact, BF, Q-R and PQ-R are TD methods. In particular, Q-R and PQ-R, which propagate the estimation information only one step back, are precisely distributed versions of the TD(0) class of algorithms. They could be transformed into generic TD(λ), $0 \geq \lambda < 1$, by transmitting backward to all the nodes previously visited by the data packet, the information collected by the routing packet generated after each data hop. Of course, this would greatly increase the routing traffic generated, because it has to be done after each hop of each data packet, making the approach at least very costly, if feasible at all.

In general, using temporal differences methods in the context of routing presents an important problem: the key condition of the method, the self-consistency between the estimates of successive states, that is, the application of the Bellman's principle, may not be strictly satisfied in the general case. This is due to the fact that (i) the dynamics at each node are related in a highly non-linear way to the dynamics of all its neighbors, (ii) the traffic process evolves concurrently over all the nodes, and (iii) there is a recursive interaction between the traffic patterns and the control actions (that is, the modifications of the routing tables). According to these facts, the nodes cannot be correctly seen as the states of a Markov process, therefore, the assumptions for the effective application of TD methods whose final outcomes are based on information boot-

strapping are not met. In this sense, the poor performance of TD(0)-like algorithms as Q-R and PQ-R in case of highly non-stationary routing problems can be better understood. On the contrary, being AntNets a sort of batch TD(1) methods, not relying on information bootstrapping, they can be more safely applied in these cases. Although, in case of quasi-stationarity, bootstrapping methods are expected to be more effective and to converge more quickly.

UPDATE FREQUENCY OF THE ROUTING TABLES

In BF and SPF the frequency according to which routing information is transmitted to the other nodes plays a major role concerning algorithm performance. This is particularly true for BF, which maintains at each node only an incomplete representation of the network status and topology. Unfortunately, the "right" frequency for sending routing information is problem-dependent, and there is no straightforward way to make it adaptive, while, at the same time, avoiding large oscillations (as is confirmed by the early attempts in both ARPANET and Internet). In Q-R and PQ-R, routing tables updating is data driven: only the Q-values associated to the neighbor nodes visited by data packets are updated. Although this is a reasonable strategy, given that the exploration of new routes (by using data packets) could cause undesired delays to data, it causes long delays before discovering new good routes, and is a major handicap in a domain in which good routes could change all the time. In OSPF routing tables are practically never updated: link costs have been assigned on the basis of the physical characteristics of the links. This lack of an adaptive metric is the main reason of the poor performance of OSPF, as it has been already remarked.

AntNets, from one side do not have the *exploration* problems of Q-R and PQ-R, since they make use of simulation packets to explore new routes, from the other side, they do not have the same *critical* dependency of BF and SPF from the frequency for sending routing information. In fact, from the experiments carried out, AntNets have shown to be robust to changes in the ants' generation rate: for a wide range of generation rates, rather independent of the network size, the algorithm is able to provide very good performance, while, at the same time, the generated routing overhead is in practice negligible also for considerable amount of generated ant agents.

8.6 Summary

In this chapter we have reported an extensive set of experimental results based on simulation about the performance of AntNet, AntNet-FA, and AntHocNet.

Concerning AntNet and AntNet-FA, to which the majority of results refer to, in order to provide significance to our studies, we have investigated several different traffic patterns and levels of workload for six different types of networks both modeled on real-world instances and artificially designed. The performance of our algorithms have been compared to those of six other algorithms representative of state-of-the-art of both static and adaptive routing algorithms. The performance showed by AntNet and AntNet-FA are excellent. Under all the considered situations they clearly outperformed the competitor algorithms. In the case of low and quasi-static input traffic, algorithms' performance is quite similar, and has not been showed here.

On the other hand, AntHocNet has been compared only to AODV, which is however the currently most popular state-of-the-art algorithm for MANETs. We investigated the behavior of the algorithms for a range of different situations in terms of number of nodes, node mobility, and node density. We also briefly investigated the behavior in large networks (up to 2000 nodes). In general, in all scenarios AntHocNet performed comparably or better than AODV. In particular, it always gave better delivery ratio than AODV. Concerning delay, it had a slightly higher average delay than AODV for the simpler scenarios (high density and short paths) but a lower average delay for the more difficult ones. While when considering the 99-th percentile for delays,

AntHocNet had always better performance. The better performance on delivery ratio likely results from the multipath nature of AntHocNet that ensures that there are often alternative paths available in case of route failures, resulting in less packet loss and quicker local recovery from failure. On the other hand, the use of multiple paths means also that not all packets are sent over the minimum-delay path, such that the resulting average delay might be slightly higher. However, since AODV relies on just one path, delays can become very bad when this path becomes inefficient or invalid, as it might likely be the case in the more difficult scenarios. The good performance also for large networks are promising regarding the scalability of the approach.

Even if it is not really possible to draw final conclusions, it is clear that we have at least good indications that the general ACO ideas, once applied to distributed and highly dynamic problems can be indeed very effective. In the last section of the chapter we have (re)discussed the major design characteristics of AntNet and AntNet-FA, which have been directly inherited from the ACO metaheuristic, and we have pointed out why and under which conditions these characteristics are possibly an advantage with respect to those of other adaptive approaches. In particular, the use of: active information gathering and path exploration, stochastic components, and both local and non-local (brought by the ants) information, are the most distinctive and effective aspects of AntNet, AntNet-FA, and AntHocNet design, and, more in general, of instances of ACR. These algorithms can automatically provide multipath routing with an associated load balancing. More in general, a bundle of paths is adaptively made available and maintained, with each path associated to a relative measure of goodness (the pheromone), such that the best paths can be used for actual data routing, while the less good ones can be used as backup paths in case of need.

We have shown that the algorithms are quite robust to internal parameter settings and in particular to the ant generation rate, as well as to possible ant failures. Moreover, the study on routing overhead for AntNet has shown that the actual impact of the ant-like agents can be made quite negligible while at the same time still providing good performance, at least in small wired networks. Clearly much more care in the ant generation and spreading must be taken in the case of mobile ad hoc networks. And in a sense, this is one of the issues critically affecting the performance of the algorithm in this case.

Bibliography

[1] E. Aarts and J. Korst. *Simulated Annealing and Boltzmann Machines, A Stochastic Approach to Combinatorial Optimization and Neural Computing*. Wiley, 1989.

[2] E.H. Aarts and J. K. Lenstra, editors. *Local Search in Combinatorial Optimization*. John Wiley & Sons, Chichester, UK, 1997.

[3] T. Abe, S. A. Levin, and M. Higashi, editors. *Biodiversity: an ecological perspective*. Springer-Verlag, New York, USA, 1997.

[4] A. Acharya, A. Misra, and S. Bansal. A label-switching packet forwarding architecture for multi-hop wireless LANs. In *Proceedings of the 5th ACM international workshop on Wireless mobile multimedia*, 2002.

[5] C. Alaettinoğlu, A. U. Shankar, K. Dussa-Zieger, and I. Matta. Design and implementation of MaRS: A routing testbed. Technical Report UMIACS-TR-92-103, CS-TR-2964, Institute for Advanced Computer Studies and Department of Computer Science, University of Maryland, College Park (MD), August 1992.

[6] D.A. Alexandrov and Y.A. Kochetov. The behavior of the ant colony algorithm for the set covering problem. In K. Inderfurth, G. Schwödiauer, W. Domschke, F. Juhnke, P. Kleinschmidt, and G. Wäscher, editors, *Operations Research Proceedings 1999*, pages 255–260. Springer-Verlag, 2000.

[7] S. Appleby and S. Steward. Mobile software agents for control in telecommunications networks. *BT technology Journal*, 12(2), 1994.

[8] A.F. Atlasis, M.P. Saltouros, and A.V. Vasilakos. On the use of a Stochastic Estimator Learning Algorithm to the ATM routing problem: a methodology. *Computer Communications*, 21(538), 1998.

[9] D.O. Awduche, A. Chiu, A. Elwalid, I. Widjaja, and X.P. Xiao. A framework for Internet traffic engineering. Internet Draft draft-ietf-tewg-framework-05, Internet Engineering Task Force - Traffic Engineering Working Group, January 2000.

[10] T. R. Balch. Hierarchic social entropy: An information theoretic measure of robot group diversity. *Autonomous Robots*, 8(3):209–238, 2000.

[11] S. Baluja and R. Caruana. Removing the genetics from the standard genetic algorithm. In A. Prieditis and S. Russell, editors, *Proceedings of the Twelfth International Conference on Machine Learning (ML-95)*, pages 38–46. Palo Alto, CA: Morgan Kaufmann, 1995.

[12] S. Baluja and S. Davies. Fast probabilistic modeling for combinatorial optimization. In *Proceedings of AAAI-98*, 1998.

[13] B. Baran and R. Sosa. A new approach for AntNet routing. In *Proceedings of the 9th International Conference on Computer Communications Networks*, Las Vegas, USA, 2000.

[14] J.S. Baras and H. Mehta. A probabilistic emergent routing algorithm for mobile ad hoc networks. In *WiOpt03: Modeling and Optimization in Mobile, Ad Hoc and Wireless Networks*, 2003.

[15] A. G. Barto, R. S. Sutton, and C. W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transaction on Systems, Man and Cybernetics*, SMC-13:834–846, 1983.

[16] A.G. Barto, S.J. Bradtke, and S.P. Singh. Learning to act using real-time dynamic programming. *Artificial Intelligence*, 72:81–138, 1995.

[17] A. Bauer, B. Bullnheimer, R. F. Hartl, and C. Strauss. An Ant Colony Optimization approach for the single machine total tardiness problem. In *Proceedings of the 1999 Congress on Evolutionary Computation*, pages 1445–1450. IEEE Press, Piscataway, NJ, USA, 1999.

[18] R. Beckers, J. L. Deneubourg, and S. Goss. Trails and U-turns in the selection of the shortest path by the ant Lasius Niger. *Journal of Theoretical Biology*, 159:397–415, 1992.

[19] M. Beckmann, C.B. McGuire, and C.B. Winstein. *Studies in the Economices of Transportation*. Yale University Press, 1956.

[20] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[21] R. Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.

[22] H. Bersini, M. Dorigo, S. Langerman, G. Seront, and L. M. Gambardella. Results of the first international contest on evolutionary optimisation (1st ICEO). In *Proceedings of IEEE International Conference on Evolutionary Computation, IEEE-EC 96*, pages 611–615. IEEE Press, 1996.

[23] D. Bertsekas. *Dynamic Programming and Optimal Control*, volume I–II. Athena Scientific, 1995.

[24] D. Bertsekas. *Network Optimization: Continuous and Discrete Models*. Athena Scientific, 1998.

[25] D. Bertsekas and D. Castanon. Rollout algorithms for stochastic scheduling problems. *Journal of Heuristics*, 5(1):89–108, 1999.

[26] D. Bertsekas and R. Gallager. *Data Networks*. Prentice–Hall, Englewood Cliffs, NJ, USA, 1992.

[27] D. Bertsekas and J. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

[28] D. Bertsekas, J. N. Tsitsiklis, and Cynara Wu. Rollout algorithms for combinatorial optimization. *Journal of Heuristics*, 3(3):245–262, 1997.

[29] S. Besher and J.H. Fewell. Models of division of labor in social insects. *Annual Review of Entomology*, 46:413–440, 2001.

[30] L. Bianchi, L. M. Gambardella, and M. Dorigo. An ant colony optimization approach to the probabilistic traveling salesman problem. In *Proceedings of PPSN-VII, Seventh International Conference on Parallel Problem Solving from Nature*, volume 2439 of *Lecture Notes in Computer Science*. Springer Verlag, Berlin, Germany, 2002.

[31] L. Bianchi, L. M. Gambardella, and M. Dorigo. Solving the homogeneous probabilistic traveling salesman problem by the ACO metaheuristic. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 176–187. Springer-Verlag, 2002.

[32] N. Biggs, E. Lloyd, and R. Wilson. *Graph theory 1736-1936*. Oxford University Press, 1976.

[33] M. Birattari, G. Di Caro, and M. Dorigo. For a formal foundation of the Ant Programming approach to combinatorial optimization. Part 1: The problem, the representation, and the general solution strategy. Technical Report TR-H-301, ATR Human Information Processing Research Laboratories, Kyoto, Japan, December 2000.

[34] M. Birattari, G. Di Caro, and M. Dorigo. Toward the formal foundation of ant programming. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms, Brussels, Belgium, September 12–14, 2002*, volume 2463 of *Lecture Notes in Computer Science*, pages 188–201. Springer-Verlag, 2002.

[35] C. M. Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, 1995.

[36] J.A. Bland. Layout of facilities using an Ant System approach. *Engineering optimization*, 32(1), 1999.

[37] J.A. Bland. Space-planning by Ant Colony Optimization. *International Journal of Computer Applications in Technology*, 12, 1999.

[38] J.A. Bland. Optimal structural design by Ant Colony Optimization. *Engineering optimization*, 33: 425–443, 2001.

[39] L. Blazevic, L. Buttyan, S. Capkun, S. Giordano, J.-P. Hubaux, and J.-Y. Le Boudec. Self-organization in mobile ad-hoc networks: the approach of terminodes. *IEEE Communications Magazine*, 39(6), June 2001.

[40] L. Blazevic, S. Giordano, and J.-Y. Le Boudec. Self organized terminode routing. *Journal of Cluster Computing*, April 2002.

[41] C. Blum. Aco applied to group shop scheduling: a case study on intensification and diversification. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 14–27. Springer-Verlag, 2002.

[42] C. Blum. Ant colony optimization for the edge-weighted k-cardinality tree problem. In *GECCO 2002: Proceedings of the Genetic and Evolutionary Computation Conference*, pages 27–34, 2002.

[43] C. Blum and A. Roli. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys*, 35(3):268–308, 2003.

[44] Christian Blum. *Theoretical and practical aspects of ant colony optimization*. PhD thesis, Faculté des Sciences Appliquées, Université Libre de Bruxelles, Brussels, Belgium, March 2004.

[45] K. Bolding, M. L. Fulgham, and L. Snyder. The case for chaotic adaptive routing. Technical Report CSE-94-02-04, Department of Computer Science, University of Washington, Seattle, 1994.

[46] M. Bolondi and M. Bondanza. Parallelizzazione di un algoritmo per la risoluzione del problema del commesso viaggiatore. Master's thesis, Dipartimento di Elettronica e Informazione, Politecnico di Milano, Italy, 1993.

[47] V. Boltyanskii. *Optimal Control of Discrete Systems*. John Wiley & Sons, New York, NY, USA, 1978.

[48] E. Bonabeau, M. Dorigo, and G. Theraulaz. *Swarm Intelligence: From Natural to Artificial Systems*. Oxford University Press, 1999.

[49] E. Bonabeau, M. Dorigo, and G. Theraulaz. Inspiration for optimization from social insect behavior. *Nature*, 406:39–42, 2000.

[50] E. Bonabeau, F. Henaux, S. Guérin, D. Snyers, P. Kuntz, and G. Theraulaz. Routing in telecommunication networks with "Smart" ant-like agents. In *Proceedings of IATA'98, Second Int. Workshop on Intelligent Agents for Telecommunication Applications*, volume 1437 of *Lecture Notes in Artificial Intelligence*. Springer Verlag, 1998.

[51] E. Bonabeau, A. Sobkowski, G. Theraulaz, and J.-L. Deneubourg. Adaptive task allocation inspired by a model of division of labor in social insects. In D. Lundh, B. Olsson, and A. Narayanan, editors, *Biocomputing and Emergent Computation*, pages 36–45. World Scientific, 1997.

[52] E. Bonabeau and G. Theraulaz. Swarm smarts. *Scientific American*, 282(3):54–61, 2000.

[53] R. Borndörfer, A. Eisenblätter, M. Grötschel, and A. Martin. The orientation model for frequency assignment problems. Technical Report TR-98-01, Konrad-Zuse-Zentrum für Informationstechnik, Berlin, Germany, 1998.

[54] J. Boyan and W. Buntine (Editors). Statistical machine learning for large scale optimization. *Neural Computing Surveys*, 3, 2000.

[55] J. Boyan and A. Moore. Learning evaluation functions to improve optimization by local search. *Journal of Machine Learning Research*, 1:77–112, 2000.

[56] J.A. Boyan and M.L. Littman. Packet routing in dinamically changing networks: A reinforcement learning approach. In J. D. Cowan, G. Tesauro, and J. Alspector, editors, *Advances in Neural Information Processing Systems 6 (NIPS6)*, pages 671–678. Morgan Kaufmann, San Francisco, CA, USA, 1994.

[57] D. Braess. Uber ein paradoxon der verkehrsplanung. *Unternehmensforschung*, 12:258–268, 1968.

[58] L. S. Brakmo, S. W. O'Malley, and L. L. Peterson. TCP vegas: New techniques for congestion detection and avoidance. *ACM Computer Communication Review (SIGCOMM'94)*, 24(4), 1994.

[59] J. Branke, M. Middendorf, and F. Schneider. Improved heuristics and a genetic algorithm for finding short supersequences. *OR-Spektrum*, 20:39–46, 1998.

[60] D. Brelaz. New methods to color vertices of a graph. *Communications of the ACM*, 22:251–256, 1979.

[61] J. Broch, D.A. Maltz, D.B. Johnson, Y.-C. Hu, and J. Jetcheva. A performance comparison of multi-hop wireless ad hoc network routing protocols. In *Proceedings of the Fourth Annual ACM/IEEE International Conference on Mobile Computing and Networking (MobiCom98)*, 1998.

[62] B. Bullnheimer, R. F. Hartl, and C. Strauss. Applying the ant system to the vehicle routing problem. In S. Voß, S. Martello, I. H. Osman, and C. Roucairol, editors, *Meta-Heuristics: Advances and Trends in Local Search Paradigms for Optimization*, pages 285–296. Kluwer Academics, 1998.

[63] B. Bullnheimer, R. F. Hartl, and C. Strauss. Applying the ant system to the vehicle routing problem. In *Proc. of the Metaheuristics International Conference (MIC97)*, pages 109–120. Kluwer Academics, 1998.

[64] B. Bullnheimer, R. F. Hartl, and C. Strauss. An improved ant system algorithm for the vehicle routing problem. *Annals of Operations Research*, 89:319–328, 1999. Also TR-POM-10/97.

[65] B. Bullnheimer, R. F. Hartl, and C. Strauss. A new rank-based version of the ant system: a computational study. *Central European Journal for Operations Research and Economics*, 7(1), 1999. Also TR-POM-03/97.

[66] B. Bullnheimer, G. Kotsis, and C. Strauss. Parallelization strategies for the ant system. In R. De Leone, A. Murli, P. Pardalos, and G. Toraldo, editors, *High Performance Algorithms and Software in Nonlinear Optimization*, volume 24 of *Applied Optimization*, pages 87–100. Kluwer Academic Publishers, 1998.

[67] B. Bullnheimer and C. Strauss. Tourenplanung mit dem Ant System. Technical Report 6, Institut für Betriebwirtschaftslehre, Universität Wien, 1996.

[68] D. Câmara and A.F. Loureiro. A novel routing algorithm for ad hoc networks. In *Proceedings of the 33rd Hawaii International Conference on System Sciences*, 2000.

[69] D. Câmara and A.F. Loureiro. Gps/ant-like routing in ad hoc networks. *Telecommunication Systems*, 18(1–3):85–100, 2001.

[70] S. Camazine, J.-L. Deneubourg, N. R. Franks, J. Sneyd, G. Theraulaz, and E. Bonabeau. *Self-Organization in Biological Systems*. Princeton University Press, 2001.

[71] R. Campanini, G. Di Caro, M. Villani, I. D'Antone, and G. Giusti. Parallel architectures and intrinsically parallel algorithms: Genetic algorithms. *International Journal of Modern Physics C*, 5(1):95–112, 1994.

[72] G. Canright. Ants and loops. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 235–242. Springer-Verlag, 2002.

[73] L. Carrillo, J.L. Marzo, L. Fàbrega, , P. Vilà, and C. Guadall. Ant colony behaviour as routing mechanism to provide quality of service. In M. Dorigo, M. Birattari, C. Blum, L. M. Gambardella, F. Mondada, and T. Stützle, editors, *Ants Algorithms - Proceedings of ANTS 2004, Fourth International Workshop on Ant Algorithms*, volume 3172 of *Lecture Notes in Computer Science*. Springer-Verlag, 2004.

[74] L. Carrillo, J.L. Marzo, D. Harle, and P. Vilà. A review of scalability and its application in the evaluation of the scalability measure of antnet routing. In C.E. Palau Salvador, editor, *Proceedings of the IASTED Conference on Communication Systems and Networks (CSN'03)*, pages 317–323. ACTA Press, 2003.

[75] A. Cassandra, M. L. Littman, and N. L. Zhang. Incremental pruning: A simple, fast, exact algorithm for partially observable markov decision processes. In *Proceedings of the 13th Conference on Uncertainties in Artificial Intelligence*, 1997.

[76] H.S. Chang, W. Gutjahr, J. Yang, and S. Park. An Ant System approach to Markov Decision Processes. Technical Report 2003-10, Department of Statistics and Decision Support Systems, University of Vienna, Vienna, Austria, September 2003.

[77] J. Chen, P. Druschel, and D. Subramanian. An efficient multipath forwarding method. In *Proceedings of IEEE Infocom '98,* San Francisco, CA, USA, 1998.

[78] K. Chen. A simple learning algorithm for the traveling salesman problem. *Physical Review E*, 55, 1997.

[79] S. Chen and K. Nahrstedt. Distributed QoS routing with imprecise state information. In *Proceedings of 7th IEEE International Conference on Computer, Communications and Networks*, pages 614–621, Lafayette, LA, USA, October 1998.

[80] S. Chen and K. Nahrstedt. An overview of quality-of-service routing for the next generation high-speed networks: Problems and solutions. *IEEE Network Magazine, Special issue on Transmission and Distribution of Digital Video*, Nov./Dec., 1998.

[81] S. Chen and K. Nahrstedt. Distributed quality-of-service routing in ad-hoc networks. *IEEE Journal on Selected Areas in Communications*, 17(8), 1999.

[82] C. Cheng, R. Riley, S.P. Kumar, and J.J. Garcia-Luna-Aceves. A loop-free extended bellman-ford routing protocol without bouncing effect. *ACM Computer Communication Review (SIGCOMM '89)*, 18 (4):224–236, 1989.

[83] B. V. Cherkassky, A. V. Goldberg, and T. Radzik. Shortest paths algorithms: Theory and experimental evaluation. In D. D. Sleator, editor, *Proceedings of the 5th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 94)*, pages 516–525, Arlington, VA, January 1994. ACM Press.

[84] S.P. Choi and D.-Y. Yeung. Predictive Q-routing: A memory-based reinforcement learning approach to adaptive traffic control. In *Advances in Neural Information Processing Systems 8 (NIPS8)*, pages 945–951. MIT Press, 1996.

[85] L. Chrisman. Reinforcement learning with perceptual aliasing: The perceptual distinctions approach. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, pages 183–188, 1992.

[86] C.-J. Chung and R.G. Reynolds. A testbed for solving optimization problems using cultural algorithms. In *Proceedings of the 5th Annual Conference on Evolutionary Programming*, 1996.

[87] I. Cidon, R. Rom, and Y. Shavitt. Multi-path routing combined with resource reservation. In *IEEE INFOCOM'97*, pages 92–100, 1997.

[88] I. Cidon, R. Rom, and Y. Shavitt. Analysis of multi-path routing. *IEEE/ACM Transactions on Networking*, 7(6):885–896, 1999.

[89] G. Clarke and J. W. Wright. Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12:568–581, 1964.

[90] E. G. Coffman, M. R. Garey, and D. S. Johnson. Approximation algorithms for bin packing: a survey. In D. HochBaum, editor, *Approximation algorithms for NP-hard problems*, pages 46–93. PWS Publishing, Boston, MA, USA, 1996.

[91] A. Colorni, M. Dorigo, and V. Maniezzo. Distributed optimization by ant colonies. In *Proceedings of the European Conference on Artificial Life (ECAL 91)*, pages 134–142. Elsevier, 1991.

[92] A. Colorni, M. Dorigo, V. Maniezzo, and M. Trubian. Ant system for job-shop scheduling. *Belgian Journal of Operations Research, Statistics and Computer Science (JORBEL)*, 34:39–53, 1994.

[93] D. E. Comer. *Internetworking with TCP/IP. Principles, Protocols, and Architecture*. Prentice–Hall, Englewood Cliffs, NJ, USA, third edition, 1995.

[94] O. Cordón, I. Fernández de Viana, and F. Herrera. Analysis of the Best-Worst Ant System and its variants on the qap. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 228–234. Springer-Verlag, 2002.

[95] D. Costa and A. Hertz. Ants can colour graphs. *Journal of the Operational Research Society*, 48:295–305, 1997.

[96] D. Costa, A. Hertz, and O. Dubuis. Embedding of a sequential algorithm within an evolutionary algorithm for coloring problems in graphs. *Journal of Heuristics*, 1:105–128, 1995.

[97] M. Cottarelli and A. Gobbi. Estensioni dell'algoritmo formiche per il problema del commesso viaggiatore. Master's thesis, Dipartimento di Elettronica e Informazione, Politecnico di Milano, Italy, 1997.

[98] G.A. Croes. A method for solving traveling salesman problems. *Operations Research*, 6:791–812, 1958.

[99] J. P. Crutchfield. Semantics and thermodynamics. In M. Casdagli and S. Eubank, editors, *Nonlinear modeling and forecasting*, pages 317–359. Addison Wesley, Redwood City, 1992.

[100] M.E. Csete and J.C. Doyle. Reverse engineering of biological complexity. *Science*, 295(1):1664–11669, March 2002.

[101] G. B. Dantzig. Maximization of a linear function of variables subject to linear inequalities. In T. C. Koopmans, editor, *Activity analysis of production and allocation*, pages 339–347. John Wiley & Sons, New York, USA, 1951.

[102] P. B. Danzig, Z. Liu, and L. Yan. An evaluation of TCP Vegas by live emulation. Technical Report UCS-CS-94-588, Computer Science Department, University of Southern California, Los Angeles, 1994.

[103] S.R. Das, C.E. Perkins, and E.M. Royer. Performance comparison of two on-demand routing protocols for ad hoc networks. *IEEE Personal Communications Magazine special issue on Ad hoc Networking*, pages 16–28, February 2001.

[104] J. de Jong and M. Wiering. Multiple ant colony systems for the busstop allocation problem. In *Proceedings of the Thirteenth Belgium-Netherlands Conference on Artificial Intelligence (BNAIC'01)*, pages 141–148, 2001.

[105] J. de Jong and M.A. Wiering. Multiple ant colony systems for the busstop allocation problem. In *Proceedings of the Thirteenth Belgium-Netherlands Conference on Artificial Intelligence*, pages 141–148, 2001.

[106] P. Delisle, M. Krajecki, M. Gravel, and C. Gagné. Parallel implementation of an ant colony optimization metaheuristic with OpenMP. In *Proceedings of the 3rd European Workshop on OpenMP (EWOMP'01)*, 2001.

[107] M. den Besten, T. Stützle, and M. Dorigo. Scheduling single machines by ants. Technical Report IRIDIA/99-16, IRIDIA, Université Libre de Bruxelles, Belgium, 1999.

[108] M. den Besten, T. Stützle, and M. Dorigo. Ant colony optimization for the total weighted tardiness problem. In M. Schoenauer, K. Deb, G. Rudolph, X. Yao, E. Lutton, J.J. Merelo, and H. Schwefel, editors, *Proceedings of PPSN-VI, Sixth International Conference on Parallel Problem Solving from Nature*, volume 1917 of *Lecture Notes in Computer Science*, pages 611–620. Springer-Verlag, 2000.

[109] J.-L. Denebourg, A. lioni, and C. Detrain. Dynamics of aggregation and emergence of cooperation. *Biological Bulletin*, 202:262–267, June 2002.

[110] J.-L. Deneubourg, S. Aron, S. Goss, and J.-M. Pasteels. The self-organizing exploratory pattern of the argentine ant. *Journal of Insect Behavior*, 3:159–168, 1990.

[111] J.L. Deneubourg, S. Goss, N. Franks, A. Sendova-Franks, C. Detrain, and L. Chretien. The dynamics of collective sorting: robot-like ants and ant-like robots. In S. Wilson, editor, *Proceedings of the First International Conference on Simulation of Adaptive Behaviors: From Animals to Animats*, pages 356–365. MIT Press, Cambridge, MA, USA, 1991.

[112] C. Detrain, J.-L. Denebourg, and J. Pasteels. Decision-making in foraging by social insects. In C. Detrain, J.-L. Denebourg, and J. Pasteels, editors, *Information processing in social insects*, pages 331–354. Birkhauser, Basel, CH, 1999.

[113] G. Di Caro. A society of ant-like agents for adaptive routing in networks. DEA thesis in Applied Sciences, Polytechnic School, Université Libre de Bruxelles, Brussels (Belgium), May 2001.

[114] G. Di Caro and M. Dorigo. Distributed reinforcement agents for adaptive routing in communication networks. *Third European Workshop on Reinforcement Learning (EWRL-3)*, Rennes, France, October 13-14, 1997.

[115] G. Di Caro and M. Dorigo. Adaptive learning of routing tables in communication networks. In *Proceedings of the Italian Workshop on Machine Learning*, Torino, Italy, December 9-10 1997.

[116] G. Di Caro and M. Dorigo. AntNet: A mobile agents approach to adaptive routing. Technical Report 97–12, IRIDIA, Université Libre de Bruxelles, Brussels (Belgium), June 1997.

[117] G. Di Caro and M. Dorigo. A study of distributed stigmergetic control for packet-switched communications networks. Technical Report 97-18, IRIDIA, Université Libre de Bruxelles, Brussels (Belgium), November 1997.

[118] G. Di Caro and M. Dorigo. Mobile agents for adaptive routing. In *Proceedings of the 31st International Conference on System Sciences (HICSS-31)*, volume 7, pages 74–83. IEEE Computer Society Press, 1998.

[119] G. Di Caro and M. Dorigo. AntNet: Distributed stigmergetic control for communications networks. *Journal of Artificial Intelligence Research (JAIR)*, 9:317–365, 1998.

[120] G. Di Caro and M. Dorigo. Ant colony routing. *PECTEL 2 Workshop on Parallel Evolutionary Computation in Telecommunications*, Reading, England, April 6-7, 1998.

[121] G. Di Caro and M. Dorigo. Ant colonies for adaptive routing in packet-switched communications networks. In A. E. Eiben, T. Back, M. Schoenauer, and H.-P. Schwefel, editors, *Proceedings of PPSN-V, Fifth International Conference on Parallel Problem Solving from Nature*, volume 1498 of *LNCS*, pages 673–682. Springer-Verlag, 1998.

[122] G. Di Caro and M. Dorigo. An adaptive multi-agent routing algorithm inspired by ants behavior. In *Proceedings of PART98 - 5th Annual Australasian Conference on Parallel and Real-Time Systems*, pages 261–272. Springer-Verlag, 1998.

[123] G. Di Caro and M. Dorigo. Extending AntNet for best-effort Quality-of-Service routing. *ANTS'98 - From Ant Colonies to Artificial Ants: First International Workshop on Ant Colony Optimization*, Brussels (Belgium), October 15-16, 1998.

[124] G. Di Caro and M. Dorigo. Two ant colony algorithms for best-effort routing in datagram networks. In *Proceedings of the Tenth IASTED International Conference on Parallel and Distributed Computing and Systems (PDCS'98)*, pages 541–546. IASTED/ACTA Press, 1998.

[125] G. Di Caro and M. Dorigo. AntNet: Distributed stigmergetic control for communications networks. *Vivek, a Quarterly in Artificial Intelligence*, 12(3 & 4):2–37, 1999. Reprinted from JAIR.

[126] G. Di Caro, F. Ducatelle, and L.M. Gambardella. AntHocNet: an ant-based hybrid routing algorithm for mobile ad hoc networks. In *Proceedings of Parallel Problem Solving from Nature (PPSN) VIII*, volume 3242 of *Lecture Notes in Computer Science*, pages 461–470. Springer-Verlag, 2004. (Conference best paper award).

[127] G. Di Caro, F. Ducatelle, and L.M. Gambardella. AntHocNet: probabilistic multi-path routing in mobile ad hoc networks using ant-like agents. Technical Report 16-04, IDSIA, Lugano (Switzerland), April 2004.

[128] G. Di Caro, F. Ducatelle, and L.M. Gambardella. AntHocNet: an adaptive nature-inspired algorithm for routing in mobile ad hoc networks. *European Transactions on Telecommunications*, 2005 (to appear). (Technical Report IDSIA 27-04).

[129] G. Di Caro, F. Ducatelle, N. Ganguly, P. Heegarden, M. Jelasity, R. Montemanni, and A. Montresor. Models for basic services in ad-hoc, peer-to-peer and grid networks. Internal Deliverable D05 of Shared-Cost RTD Project (IST-2001-38923) *BISON* funded by the Future & Emerging Technologies initiative of the Information Society Technologies Programme of the European Commission, 2003.

[130] G. Di Caro and T. Vasilakos. Ant-SELA: Ant-agents and stochastic automata learn adaptive routing tables for QoS routing in ATM networks. *ANTS'2000 - From Ant Colonies to Artificial Ants: Second International Workshop on Ant Colony Optimization*, Brussels (Belgium), September 8-9, 2000.

[131] E. W. Dijkstra. A note on two problems in connection with graphs. *Numerical Mathematics*, 1:269–271, 1959.

[132] E.W. Dijkstra and C.S. Scholten. Termination detection for diffusing computations. *Information Processing Letters*, 11:1–4, August 1980.

[133] S. Doi and M. Yamamura. BntNetL: Evaluation of its performance under congestion. *Journal of IEICE B* (in Japanese), pages 1702–1711, 2000.

[134] S. Doi and M. Yamamura. BntNetL and its evaluation on a situation of congestion. *Electronics and Communications in Japan (Part I: Communications)*, 85(9):31–41, 2002.

[135] M. Dorigo. *Optimization, Learning and Natural Algorithms* (in Italian). PhD thesis, Dipartimento di Elettronica e Informazione, Politecnico di Milano, IT, 1992.

[136] M. Dorigo. Parallel ant system: An experimental study. Unpublished manuscript, 1993.

[137] M. Dorigo, M. Birattari, C. Blum, L. M. Gambardella, F. Mondada, and T. Stützle, editors. *Ant Colony Optimization and Swarm Intelligence, 4th International Workshop, ANTS 2004*, volume 3172 of *LNCS*. Springer-Verlag, Berlin, Germany, 2004.

[138] M. Dorigo, E. Bonabeau, and G. Theraulaz. Ant algorithms and stigmergy. *Future Generation Computer Systems*, 16(8):851–871, 2000.

[139] M. Dorigo and G. Di Caro. Ant colony optimization: A new meta-heuristic. In *Proceedings of CEC99 - Congress on Evolutionary Computation*, Washington DC, July 6-9 1999.

[140] M. Dorigo and G. Di Caro. The ant colony optimization meta-heuristic. In D. Corne, M. Dorigo, and F. Glover, editors, *New Ideas in Optimization*, pages 11–32. McGraw-Hill, 1999.

[141] M. Dorigo, G. Di Caro, and T. Stützle (Editors). Ant algorithms. Special Issue on *Future Generation Computer Systems (FGCS)*, 16(8), 2000.

[142] M. Dorigo, G. Di Caro, and L. M. Gambardella. Ant algorithms for discrete optimization. *Artificial Life*, 5(2):137–172, 1999.

[143] M. Dorigo, G. Di Caro, and M. Sampels, editors. *Ant Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms, Brussels, Belgium, September 12–14, 2002*, volume 2463 of *Lecture Notes in Computer Science*, 2002. Springer-Verlag.

[144] M. Dorigo and L. M. Gambardella. A study of some properties of Ant-Q. In *Proceedings of PPSN-IV, Fourth International Conference on Parallel Problem Solving From Nature*, pages 656–665. Berlin: Springer-Verlag, 1996.

[145] M. Dorigo and L. M. Gambardella. Ant colonies for the traveling salesman problem. *BioSystems*, 43: 73–81, 1997.

[146] M. Dorigo and L. M. Gambardella. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, 1997.

[147] M. Dorigo, L.M Gambardella, M. Middendorf, and T. Stützle (Eds.). Ant colony optimization. Special Section on *IEEE Transactions on Evolutionary Computation*, 6(4), 2002.

[148] M. Dorigo and V. Maniezzo. Parallel genetic algorithms: Introduction and overview of the current research. In J. Stender, editor, *Parallel Genetic Algorithms: Theory and Applications*, pages 5–42. IOS Press, 1993.

[149] M. Dorigo, V. Maniezzo, and A. Colorni. The ant system: An autocatalytic optimizing process. Technical Report 91-016 Revised, Dipartimento di Elettronica, Politecnico di Milano, IT, 1991.

[150] M. Dorigo, V. Maniezzo, and A. Colorni. Positive feedback as a search strategy. Technical Report 91-016, Dipartimento di Elettronica, Politecnico di Milano, IT, 1991.

[151] M. Dorigo, V. Maniezzo, and A. Colorni. The ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics–Part B*, 26(1):29–41, 1996.

[152] M. Dorigo and T. Stützle. *Ant Colony Optimization*. MIT Press, Cambridge, MA, 2004.

[153] M. Dorigo, M. Zlochin, N. Meuleau, and M. Birattari. Updating ACO pheromones using stochastic gradient ascent and cross-entropy methods. In S. Cagnoni, J. Gottlieb, E. Hart, M. Middendorf, and G. R. Raidl, editors, *Applications of Evolutionary Computing, Proceedings of EvoWorkshops 2002*, volume 2279 of *Lecture Notes in Computer Science*, pages 21–30. Springer Verlag, Berlin, Germany, 2002.

[154] F. Ducatelle, G. Di Caro, and L.M. Gambardella. Ant agents for hybrid multipath routing in mobile ad hoc networks. In *Proceedings of the Second Annual Conference on Wireless On demand Network Systems and Services (WONS)*, St. Moritz, Switzerland, January 18–19, 2005. (Technical Report IDSIA 26-04).

[155] F. Ducatelle and J. Levine. Ant colony optimisation for bin packing and cutting stock problems. In *Proceedings of the UK Workshop on Computational Intelligence*, Edinburgh, UK, September 2001.

[156] W. Durham. *Co-Evolution: Genes, Culture, and Human Diversity*. Stanford University Press, Stanford, CA, USA, 1984.

[157] A. Dussutour, V. Fourcassié, D. Helbing, and J.-L. Denebourg. Optimal traffic organization in ants under crowded conditions. *Nature*, 428:70–73, March 2004.

[158] H. Dyckhoff. A typology of cutting and packing problems. *European Journal of Operational Research*, 44:145–159, 1990.

[159] D. Eppstein, Z. Galil, and G. Italiano. Dynamic graph algorithms. In *Handbook of Algorithms and Theory of Computation*, chapter 22. CRC Press, 1997.

[160] L. F. Escudero. An inexact algorithm for the sequential ordering problem. *European Journal of Operations Research*, 37:232–253, 1988.

[161] G.R. Brookes et al. *Inside the Transputer*. Blackwell Scientific Publications, 1990.

[162] C. J. Eyckelhof and M. Snoek. Ant systems for a dynamic TSP. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 88–99. Springer-Verlag, 2002.

[163] D. Farinachi. *Introduction to enhanced IGRP (EIGRP)*. Cisco System Inc., 1993.

[164] J. Feigenbaum and S. Kannan. Dynamic graph algorithms. In K. H. Rosen, J. G. Michaels, J. L. Gross, J. W. Grossman, and D. R. Shier, editors, *Handbook of Discrete and Combinatorial Mathematics*. CRC Press, 2000.

[165] J. A. Feldman and D. H. Ballard. Connectionist models and their properties. *Cognitive Science*, 6: 205–254, 1982.

[166] S. Fenet and S. Hassas. A.N.T.: a distributed network control framework based on mobile agents. In *Proceedings of the International ICSC Congress on Intelligent Systems And Applications*, 2000.

[167] S. Fenet and S. Hassas. A.N.T.: a distributed problem-solving framework based on mobile agents. In *Advances in Mobile Agents and System Research*, volume 1, 2000.

[168] J. H. Fewell. Social insect networks. *Science*, 301(26):1867–1869, September 2003.

[169] S. Fidanova. ACO algorithm for MKP using different heuristic information. In *Proceedings of NM&A'02 - Fifth International Conference on Numerical Methods and Applications*, Lecture Notes in Computer Science. Springer Verlag, Berlin, Germany, 2002.

[170] C. Fleurent and J. Ferland. Genetic and hybrid algorithms for graph coloring. *Annals of Operations Research*, 63:437–461, 1996.

[171] M.J. Flynn. Some computer organization and their effectiveness. *IEEE Transactions on Computers*, C-21(9), September 1972.

[172] D. B. Fogel. *Evolutionary Computation*. IEEE Press, 1995.

[173] L. Ford and D. Fulkerson. *Flows in Networks*. Prentice-Hall, 1962.

[174] B. Fortz and M. Thorup. Increasing internet capacity using local search. Technical report, AT&T Labs Research, 2000. Short version published as "Internet traffic engineering by optimizing OSPF weights", in Proc. IEEE INFOCOM 2000 - The Conference on Computer Communication.

[175] D. E. Foulser, M. Li, and Q. Yang. Theory and algorithms for plan merging. *Artificial Intelligence*, 57: 143–181, 1992.

[176] B. Freisleben and P. Merz. Genetic local search algorithms for solving symmetric and asymmetric traveling salesman problems. In *Proceedings of IEEE International Conference on Evolutionary Computation, IEEE-EC96*, pages 616–621. IEEE Press, 1996.

[177] B. Freisleben and P. Merz. New genetic local search operators for the traveling, salesman problem. In H.-M. Voigt, W. Ebeling, I. Rechenberg, and H.-P. Schwefel, editors, *Proceedings of PPSN-IV, Fourth International Conference on Parallel Problem Solving from Nature*, pages 890–899. Berlin: Springer-Verlag, 1996.

[178] K. Fujita, A. Saito, T. Matsui, and H. Matsuo. An adaptive ant-based routing algorithm used routing history in dynamic networks. In *4th Asia-Pacific Conference on Simulated Evolution and Learning*, pages 46–50, 2002.

[179] C. Gagné, W.L. Price, and M. Gravel. Comparing an aco algorithm with other heuristics for the single machine scheduling problem with sequence dependent setup times. *Journal of the Operational Research Society*, 2002.

[180] R. Gallager. A minimum delay routing algorithm using distributed computation. *IEEE Transactions on Communications*, 25:73–84, January 1977.

[181] M. Gallego-Schmid. Modified AntNet: Software application in the evaluation and management of a telecommunication network. Genetic and Evolutionary Computation Conference (GECCO-99), Student Workshop, 1999.

[182] L. M. Gambardella and M. Dorigo. Ant-Q: A reinforcement learning approach to the traveling salesman problem. In *Proceedings of the Twelfth International Conference on Machine Learning, ML-95*, pages 252–260. Palo Alto, CA: Morgan Kaufmann, 1995.

[183] L. M. Gambardella and M. Dorigo. Solving symmetric and asymmetric TSPs by ant colonies. In *Proceedings of the IEEE Conference on Evolutionary Computation, ICEC96*, pages 622–627. IEEE Press, 1996.

[184] L. M. Gambardella and M. Dorigo. HAS-SOP: An hybrid ant system for the sequential ordering problem. Technical Report 11-97, IDSIA, Lugano, CH, 1997.

[185] L. M. Gambardella and M. Dorigo. An ant colony system hybridized with a new local search for the sequential ordering problem. *INFORMS Journal on Computing*, 12(3):237–255, 2000.

[186] L. M. Gambardella, E. Taillard, and G. Agazzi. Ant colonies for vehicle routing problems. In D. Corne, M. Dorigo, and F. Glover, editors, *New Ideas in Optimization*, pages 63–76. London,UK: McGraw-Hill, 1999.

[187] L. M. Gambardella, E. D. Taillard, and M. Dorigo. Ant colonies for the QAP. Technical Report 4-97, IDSIA, Lugano, Switzerland, 1997.

[188] L. M. Gambardella, É. D. Taillard, and M. Dorigo. Ant colonies for the quadratic assignment problem. *Journal of the Operational Research Society (JORS)*, 50(2):167–176, 1999. Also TR-IDSIA-4-97.

[189] J.J. Garcia-Luna-Aceves. Loop-free routing using diffusing computations. *IEEE/ACM Transactions on Networking*, 1:130–141, February 1993.

[190] J.J. Garcia-Luna-Aceves and J. Behrens. Distributed, scalable routing based on vectors of link states. *IEEE Journal on Selected Areas in Communications*, 13, October 1995.

[191] J.J. Garcia-Luna-Aceves and S. Murthy. A path-finding algorithm for loop-free routing. *IEEE/ACM Transactions on Networking*, February 1997.

[192] M. R. Garey and D. S. Johnson. *Computers and Intractability*. W.H. Freeman and Company, 1979.

[193] M. R. Garey, D. S. Johnson, and R. Sethi. The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 2(2):117–129, 1976.

[194] R.M. Garlick and R.S. Barr. Dynamic wavelength routing in WDM networks via Ant Colony Optimization. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 250–255. Springer-Verlag, 2002.

[195] J. Gavett. Three heuristics rules for sequencing jobs to a single production facility. *Management Science*, 11:166–176, 1965.

[196] S. Geman, E. Bienenstock, and R. Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4:1–58, 1992.

[197] M. Gendrau, A. Hertz, and G. Laporte. New insertion and postoptimization procedures for the traveling salesman problem. *Operations Research*, 40:1086–1094, 1992.

[198] Comte de Buffon Georges-Luis Lecrerc. *Essai d'aritmetique morale*. France, 1777.

[199] F. Glover. Tabu search, part I. *ORSA Journal on Computing*, 1:190–206, 1989.

[200] F. Glover. Tabu search, part II. *ORSA Journal on Computing*, 2:4–32, 1989.

[201] F. Glover and G. Kochenberger, editors. *Handbook of Metaheuristics*, volume 57 of *International Series in Operations Research and Management Science*. Kluwer Academic Publishers, 2003.

[202] D. E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.

[203] S. Goss, S. Aron, J. L. Deneubourg, and J. M. Pasteels. Self-organized shortcuts in the Argentine ant. *Naturwissenschaften*, 76:579–581, 1989.

[204] P. P. Grassé. *Les Insects dans Leur Univers*. Ed. du Palais de la découverte, Paris, 1946.

[205] P. P. Grassé. La reconstruction du nid et les coordinations interindividuelles chez *bellicositermes natalensis et cubitermes sp.* La théorie de la stigmergie: essai d'interprétation du comportement des termites constructeurs. *Insectes Sociaux*, 6:41–81, 1959.

[206] R. S. Gray, G. Cybenko, D. Kotz, and D. Rus. Mobile agents: Motivations and state of the art. In J. Bradshaw, editor, *Handbook of Agent Technology*. AAAI/MIT Press, 2001.

[207] M. Grötschel and M. W. Padberg. Polyhedral theory. In E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy-Kan, and D. B. Shmoys, editors, *The Travelling Salesman Problem*. John Wiley & Sons, Chichester, UK, 1985.

[208] R. Guerin and A. Orda. QoS-based routing in networks with inaccurate information: Theory and algorithms. In *Proceedings of IEEE INFOCOM'97*, May 1997.

[209] F. Guerriero and M. Mancini. A cooperative parallel rollout algorithm for the sequential ordering problem. *Parallel Computing*, 29(5):663–677, 2003.

[210] M. Günes, M. Kähmer, and I. Bouazizi. Ant-routing-algorithm (ARA) for mobile multi-hop ad-hoc networks - new features and results. In *Proceedings of the 2nd Mediterranean Workshop on Ad-Hoc Networks (Med-Hoc-Net'03)*, Mahdia, Tunisia, June 2003.

[211] M. Günes, U. Sorges, and I. Bouazizi. ARA - the ant-colony based routing algorithm for MANETS. In *Proceedings of the 2002 ICPP International Workshop on Ad Hoc Networks (IWAHN 2002)*, pages 79–85, August 2002.

[212] M. Guntsch and M. Middendorf. Applying population based ACO to dynamic optimization problems. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 111–122. Springer-Verlag, 2002.

[213] M. Guntsch and M. Middendorf. A population based approach for ACO. In S. Cagnoni, J. Gottlieb, E. Hart, M. Middendorf, and G. Raidl, editors, *Applications of Evolutionary Computing, Proceedings of EvoWorkshops2002: EvoCOP, EvoIASP, EvoSTim*, volume 2279, pages 71–80, Kinsale, Ireland, 3–4 2002. Springer-Verlag.

[214] W. Gutjahr. A graph-based ant system and its convergence. Special issue on Ant Algorithms, *Future Generation Computer Systems*, 16(8):873–888, 2000.

[215] W. Gutjahr. Aco algorithms with guaranteed convergence to the optimal solution. *Information Processing Letters*, 82:145–153, 2002.

[216] W. Gutjahr. A converging aco algorithm for stochastic combinatorial optimization. In A. Albrecht and K. Steinhoefl, editors, *Stochastic Algorithms: Foundations and Applications - Proceedings of SAGA 2003*, volume 2827 of *Lecture Notes in Computer Science*, pages 10–25. Springer-Verlag, 2003.

[217] B. Halabi. *Internet Routing Architectures*. New Riders Publishing, Cisco Press, 1997.

[218] J. Handl, J. Knowles, and M. Dorigo. On the performance of ant-based clustering. In *Design and application of hybrid intelligent systems*, volume 104 of *Frontiers in Artificial intelligence and Applications*. IOS Press, Amsterdam, The Netherlands, 2003.

[219] M. Hauskrecht. *Planning and control in stochastic domains with imperfect information*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1997.

[220] A.L. Hayzelden and J. Bigham. Agent technology in communications systems: An overview. *Knowledge Engineering Review*, 1999.

[221] M. Heissenbüttel and T. Braun. Ants-based routing in large scale mobile ad-hoc networks. In *Kommunikation in verteilten Systemen (KiVS03)*, March 2003.

[222] C. K. Hemelrijk. Self-organization and natural selection in the evolution of complex despotic societies. *Biological Bulletin*, 202:283–288, 2002.

[223] J. Hertz, A. Krogh, and R. G. Palmer. *Introduction to the Theory of Neural Computation*. Addison Wesley, 1991.

[224] M. Heusse, D. Snyers, S. Guérin, and P. Kuntz. Adaptive agent-driven routing and load balancing in communication networks. *Advances in Complex Systems*, 1(2):237–254, 1998.

[225] W. D. Hillis. *The Connection Machine*. MIT Press, 1982.

[226] J. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.

[227] B. Hölldobler and E.O. Wilson. *The Ants*. Springer-Verlag, Berlin, Germany, 1990.

[228] J. Hopfield and D. W. Tank. Neural computation of decisions in optimization problems. *Biological Cybernetics*, 52, 1985.

[229] R. A. Howard. *Dynamic Programming and Markov Processes*. MIT Press, Cambridge, 1960.

[230] International Standards Organization. Intra-domain IS-IS routing protocol. ISO/IEC JTC1/SC6 WG2 N323, September 1989.

[231] S. Iredi, D. Merkle, and M. Middendorf. Bi-criterion optimization with multi colony ant algorithms. In *Proceedings of the First International Conference on Multi-Criterion Optimization (EMO'01), Zurich, Switzerland*, pages 359–372, 2001.

[232] G. Israel. *La visione matematica della realtà* (in Italian). Editori Laterza, 2003. Translated from French, *La mathématisation du réel*, Éditions du Seuil, Paris, 1996.

[233] J.M. Jaffe and F.H. Moss. A responsive distributed routing algorithm for computer networks. *IEEE Transactions on Communications*, 30:1758–1762, July 1982.

[234] P. Jain. Validation of AntNet as a superior single path, single constrained routing protocol. Master's thesis, Department of Computer Science and Engineering, University of Minnesota, June 2002.

[235] M. Jerrum and A. Sinclair. Conductance and the rapid mixing property for markov chains: the approximation of the permanent resolved. In *Proceedings of the ACM Symposium on Theoretical Computing*, pages 235–244, 1988.

[236] D. B. Johnson and D. A. Maltz. *Mobile Computing*, chapter Dynamic Source Routing in Ad Hoc Wireless Networks, pages 153–181. Kluwer, 1996.

[237] D. S. Johnson and L. A. McGeoch. The traveling salesman problem: A case study. In E.H. Aarts and J. K. Lenstra, editors, *Local Search in Combinatorial Optimization*, pages 215–310. John Wiley & Sons, Chichester, UK, 1997.

[238] D. S. Johnson and M. A. Trick, editors. *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, 1996.

[239] N. L. Johnson. Developmental insights into evolving systems: Roles of diversity, non-selection, self-organization, symbiosis. In M. Bedau, editor, *Artificial Life VII*. MIT Press, Cambridge, MA, 2000.

[240] N. L. Johnson. Importance of diversity: Reconciling natural selection and noncompetitive processes. In J. L. R. Chandler and G. V. d. Vijer, editors, *Closure: Emergent Organizations and Their Dynamics*, volume 901 of *Annals of the New York Academy of Sciences*, New York, USA, 2000.

[241] L. P. Kaebling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.

[242] N. Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4:373–395, 1984.

[243] I. Kassabalidis, A.K. Das, M.A. El-Sharkawi, R.J. Marks II, P. Arabshahi, and A.A. Gray. Intelligent routing and bandwidth allocation in wireless networks. In *Proceedings of the NASA Earth Science Technology Conference, College Park, MD, USA, August 28–30, 2001*, 2002.

[244] I. Kassabalidis, M.A. El-Sharkawi, R.J. Marks II, P. Arabshahi, and A.A. Gray. Adaptive-SDR: Adaptive swarm-based distributed routing. In *Proceedings of IEEE Globecom 2001*, 2002.

[245] I. Kassabalidis, M.A. El-Sharkawi, R.J. Marks II, P. Arabshahi, and A.A. Gray. Swarm intelligence for routing in communication networks. In *Proceedings of the IEEE World Congress on Computational Intelligence, Hawaii, May 12–17 2002*, 2002.

[246] H. Kawamura, M. Yamamoto, and A. Ohuchi. Improved multiple ant colonies systems for traveling salesman problems. In E. Kozan and A. Ohuchi, editors, *Operations research / management science at work*, pages 41–59. Kluwer, Boston, USA, 2002.

[247] H. Kawamura, M. Yamamoto, K. Suzuki, and A. Ohuchi. Multiple ant colonies algorithm based on colony level interactions. *IEICE Transactions, Fundamentals*, E83-A:371–379, 2000.

[248] M. Kearns and S. Singh. Near-optimal reinforcement learning in polynomial time. In *Proc. 15th International Conf. on Machine Learning*, pages 260–268. Morgan Kaufmann, San Francisco, CA, 1998.

[249] J. Kennedy and R.C. Eberhart. *Swarm Intelligence*. Morgano Kaufmann, 2001.

[250] J. Kephart and D. Chess. The vision of autonomic computing. *IEEE Computer magazine*, pages 41–50, January 2003.

[251] L. G. Khachiyan. A polynomial algorithm for linear programming. *Soviet Mathematics Doklady*, 15: 434–437, 1979.

[252] A. Khanna and J. Zinky. The revised ARPANET routing metric. *ACM SIGCOMM Computer Communication Review*, 19(4):45–56, 1989.

[253] S. Kirkpatrick, D. C. Gelatt, and M. P. Vecchi. Statistical mechanics algorithm for Monte Carlo optimization. *Physics Today*, May:17–19, 1982.

[254] C. G. Knott. *Life and Scientific Work of Peter Guthrie Tait*, pages 214–215. London, UK, 1911.

[255] Young-Bae Ko and Nitin H. Vaidya. Location-aided routing (LAR) in mobile ad hoc networks. In *Proceedings of the Fourth Annual ACM/IEEE International Conference on Mobile Computing and Networking*, pages 66–75, Dallas, TX, USA, October 1998.

[256] R. Kolisch. *Project scheduling under resource constraints*. Production and Logistics. Springer, 1995.

[257] T. C. Koopmans and M. J. Beckmann. Assignment problems and the location of economic activities. *Econometrica*, 25:53–76, 1957.

[258] Y.A. Korli, A.A. Lazar, and A. Orda. Capacity allocation under noncooperative routing. *IEEE Transactions on Automatic Control*, 43(3):309–325, 1997.

[259] D. Kotz and R. S. Gray. Mobile agents and the future of the Internet. *ACM Operating Systems Review*, 33(3):7–13, August 1999.

[260] J. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.

[261] K. Krishnaiyer and S.H. Cheraghi. Ant algorithms: Review and future applications. In *Proceedings of the 2002 IERC Conference*, May 2002.

[262] M. J. Krone. *Heuristic programming applied to scheduling problems*. PhD thesis, Princeton University, September 1970.

[263] F. Krüger, D. Merkle, and M. Middendorf. Studies on a parallel ant system for the BSP model. Unpublished manuscript, 1998.

[264] S. Kumar and R. Miikulainen. Dual reinforcement Q-routing: an on-line adaptive routing algorithm. In *Artificial Neural Networks In Engineering (ANNIE97)*, 1997.

[265] A. Küpper and A. S. Park. Stationary vs. mobile user agents in future mobile telecommunication networks. In *Proceedings if Mobile Agents '98*, volume 1477 of *Lecture Notes in Computer Science*, pages 112–124. Springer-Verlag: Heidelberg, Germany, 1998.

[266] P. Larranaga. A review of estimation of distribution algorithms. In P. Larranaga and J.A. Lozano, editors, *Estimation of distribution algorithms*, pages 80–90. Kluwer Academic Publisher, 2002.

[267] E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy-Kan, and D. B. Shmoys, editors. *The Travelling Salesman Problem*. John Wiley & Sons, Chichester, UK, 1985.

[268] S.-J. Lee, E. M. Royer, and C. E. Perkins. Scalability study of the ad hoc on-demand distance vector routing protocol. *ACM/Wiley International Journal of Network Management*, 13(2):97–114, March 2003.

[269] G. Leguizamón and Z. Michalewicz. A new version of Ant System for subset problems. In *Proceedings of the 1999 Congress on Evolutionary Computation*, pages 1459–1464. IEEE Press, Piscataway, NJ, USA, 1999.

[270] F. Leighton. A graph coloring algorithm for large scheduling problems. *Journal of Research of the National Bureau of Standards*, 84:489–505, 1979.

[271] J. Levine and F. Ducatelle. Ants can solve difficult bin packing problems. In *Proceedings of the 1st Multidisciplinary International Conference on Scheduling: Theory and Applications (MISTA 2003)*, Nottingham, UK, August 2003.

[272] J. Levine and F. Ducatelle. Ant colony optimisation and local search for bin packing and cutting stock problems. *Journal of the Operational Research Society, Special Issue on Local Search*, 55(7), July 2004.

[273] Y. Li, P.M. Pardalos, and M.G. Resende. A greedy randomized adaptive search procedure for the quadratic assignment problem. In P.M. Pardalos and H. Wolkowicz, editors, *Quadratic assignment and related problems*, volume 16 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 237–261. 1994.

[274] S. Liang, A.N. Zincir-Heywood, and M.I. Heywood. Intelligent packets for dynamic network routing using distributed genetic algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO'02)*, 2002.

[275] Y.-C. Liang and A. E. Smith. An Ant System approach to redundancy allocation. In *Proceedings of the 1999 Congress on Evolutionary Computation*, pages 1478–1484. IEEE Press, Piscataway, NJ, USA, 1999.

[276] S. Lin. Computer solutions of the traveling salesman problem. *Bell Systems Journal*, 44:2245–2269, 1965.

[277] M. Littman. Memoryless policies: Theoretical limitations and practical results. In *Proceedings of the International Conference on Simulation of Adaptive Behavior: From Animals to Animats 3*, 1994.

[278] M. L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the 11th International Conference on Machine Learning*, pages 157–163, New Brunswick, NJ, 1994.

[279] M. L. Littman, T. L. Dean, and L. P. Kaelbling. On the complexity of solving Markov decision processes. In *Proceedings of the Eleventh International Conference on Uncertainty in Artificial Intelligence*, 1995.

[280] J. Loch and S. Singh. Using eligibility traces to find the best memoryless policy in partially observable markov decision processes. In *Proceedings of the Fifteenth International Conference on Machine Learning*, 1998.

[281] F.-X. Le Louarn, M. Gendrau, and J.-Y. Potvin. GENI ants for the traveling salesman problem. In *INFORMS Fall 2000 Meeting, November 5–8*, San Antonio, 2000.

[282] F.-X. Le Louarn, M. Gendrau, and J.-Y. Potvin. GENI ants for the traveling salesman problem. Technical report, Centre de recherche sur le transports (CRT), University of Montreal, Quebec, Canada, 2001.

[283] W.S. Lovejoy. A survey of algorithmic methods for partially observed Markov decision processes. *Annals of Operations Research*, 28:47–66, 1991.

[284] M.G.C. Resende M. Ericsson and P.M. Pardalos. A genetic algorithm for the weight setting problem in OSPF routing. *Journal of Combinatorial Optimization*, 6:299–333, 2002.

[285] Q. Ma. *Quality of Service routing in integrated services networks*. PhD thesis, Department of Computer Science, Carnegie Mellon University, 1998.

[286] Q. Ma, P. Steenkiste, and H. Zhang. Routing in high-bandwidth traffic in max-min fair share networks. *ACM Computer Communication Review (SIGCOMM'96)*, 26(4):206–217, 1996.

[287] S. Mahadevan. Average reward reinforcement learning: Foundations, algorithms, and empirical results. *Machine Learning*, 22:159–196, 1996.

[288] G. S. Malkin. *RIP: An Intra-Domain Routing Protocol*. Addison-Wesley, 1999.

[289] G. S. Malkin and M. E. Steenstrup. Distance-vector routing. In M. E. Steenstrup, editor, *Routing in Communications Networks*, chapter 3, pages 83–98. Prentice-Hall, 1995.

[290] V. Maniezzo. Exact and approximate nondeterministic tree-search procedures for the quadratic assignment problem. Technical Report CSR 98-1, C. L. In Scienze dell'Informazione, Universitá di Bologna, sede di Cesena, Italy, 1998.

[291] V. Maniezzo. Exact and approximate nondeterministic tree-search procedures for the quadratic assignment problem. *INFORMS Journal of Computing*, 1999.

[292] V. Maniezzo and A. Carbonaro. An ANTS heuristic for the frequency assignment problem. Technical Report CSR 98-4, Scienze dell'Informazione, Universitá di Bologna, Sede di Cesena, Italy, 1998.

[293] V. Maniezzo and A. Carbonaro. An ANTS heuristic for the frequency assignment problem. *Future Generation Computer Systems*, 16(8):927–935, 2000.

[294] V. Maniezzo and A. Carbonaro. Ant colony optimization: an overview. In C. Ribeiro, editor, *Essays and Surveys in Metaheuristics*, pages 21–44. Kluwer, 2001.

[295] V. Maniezzo and A. Colorni. The Ant System applied to the quadratic assignment problem. *IEEE Transactions on Knowledge and Data Engineering*, 11(5):769–778, 1999.

[296] V. Maniezzo, A. Colorni, and M. Dorigo. The ant system applied to the quadratic assignment problem. Technical Report IRIDIA/94-28, Université Libre de Bruxelles, Belgium, 1994.

[297] V. Maniezzo and M. Milandri. An ant-based framework for very strongly constrained problems. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 222–227. Springer-Verlag, 2002.

[298] S. Mannor, R. Y. Rubinstein, and Y. Gat. The cross-entropy method for fast policy search. In *Proceedings of the Twelfth International Conference on Machine Learning (ICML-2003)*, Washington DC, USA, 2003.

[299] S. Martello and P. Toth. *Knapsack Problems. Algorithms and Computer Implementations*. John Wiley & Sons, West Sussex, UK, 1990.

[300] O. Martin, S. W. Otto, and E. W. Felten. Large-step Markov chains for the TSP incorporating local search heuristics. *Operations Research Letters*, 11(4):219–224, 1992.

[301] S. Marwaha, C. K. Tham, and D. Srinivasan. Mobile agents based routing protocol for mobile ad hoc networks. In *Proceedings of IEEE Globecom*, 2002.

[302] H. Matsuo and K. Mori. Accelerated ants routing in dynamic networks. In *2nd International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing*, pages 333–339, 2001.

[303] J. C. Maxwell. *Theory of Heat*. London, UK, 1872.

[304] J. M. McQuillan, I. Richer, and E. C. Rosen. The new routing algorithm for the ARPANET. *IEEE Transactions on Communications*, 28:711–719, 1980.

[305] D. Merkle and M. Middendorf. An ant algorithm with a new pheromone evaluation rule for total tardiness problems. In *Proceedings of the EvoWorkshop 2000*, volume 1803 of *Lecture Notes in Computer Science*, pages 287–296. Springer-Verlag, 2000.

[306] D. Merkle and M. Middendorf. A new approach to solve permutation scheduling problems with ant colony optimization. In E.J. Boers, S. Cagnoni, J. Gottlieb, E. Hart, P.L. Lanzi, G. Raidl, R.E. Smith, and H. Tijink, editors, *Applications of Evolutionary Computing: Proceedings of the EvoWorkshop 2001*, volume 2037 of *Lecture Notes in Computer Science*, pages 213–222. Springer-Verlag, 2001.

[307] D. Merkle and M. Middendorf. Ant colony optimization with global pheromone evaluation for scheduling a single machine. *Applied Intelligence*, 18(1):105–111, 2003.

[308] D. Merkle, M. Middendorf, and H. Schmeck. Ant colony optimization for resource-constrained project scheduling. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2000)*, pages 893–900. Morgan Kauffmann, 2000.

[309] D. Merkle, M. Middendorf, and H. Schmeck. Ant colony optimization for resource-constrained project scheduling. *IEEE Transactions on Evolutionary Computation*, 6(4):333–346, 2002.

[310] P.M. Merlin and A. Segall. A failsafe distributed routing protocol. *IEEE Transactions on Communications*, COM-27(9):1280–1287, September 1979.

[311] Metaheuristics Network: Project sponsored by the Improving Human Potential program of the European Community. `http://www.metaheuristics.org`.

[312] N. Metropolis, A.W. Rosenbluth, M.N. Rosenbluth, A.H. Teller, and E. Teller. Equations of state calculations by fast computing machines. *Journal of Chemical Physics*, 21:1087–1092, 1953.

[313] N. Meuleau and M. Dorigo. Ant colony optimization and stochastic gradient descent. *Artificial Life*, 8(2):103–121, 2002.

[314] T. Michalareas and L. Sacks. Link-state and ant-like algorithm behaviour for single-constrained routing. *IEEE Workshop on High Performance Switching and Routing, HPSR 2001*, May 2001.

[315] T. Michalareas and L. Sacks. Stigmergic techniques for solving multi-constraint routing for packet networks. In P. Lorenz, editor, *Networking - ICN 2001, Proceedings of the First International Conference on Networking, Part II, Colmar, France July 9-13, 2001*, volume 2094 of *Lecture Notes in Computer Science*, pages 687–697. Springer-Verlag, 2001.

[316] R. Michel and M. Middendorf. An island model based ant system with lookahead for the shortest supersequence problem. In A. E. Eiben, T. Back, M. Schoenauer, and H.-P. Schwefel, editors, *Proceedings of PPSN-V, Fifth International Conference on Parallel Problem Solving from Nature*, pages 692–701. Springer-Verlag, 1998.

[317] R. Michel and M. Middendorf. An ACO algorithm for the shortest supersequence problem. In D. Corne, M. Dorigo, and F. Glover, editors, *New Ideas in Optimization*, pages 51–61. London,UK: McGraw-Hill, 1999.

[318] M. Middendorf, F. Reischle, and H. Schmeck. Information exchange in multi colony ant algorithms. In J. Rolim, editor, *Parallel and Distributed Computing, Proceedings of the 15th IPDPS 2000 Workshops, Third Workshop on Biologically Inspired Solutions to Parallel Processing Problems (BioSP3)*, volume 1800 of *Lecture Notes in Computer Science*, pages 645–652. Springer-Verlag, Berlin, Germany, 2000.

[319] C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer programming formulation and traveling salesman problems. *Journal of the ACM*, 7:326–329, 1960.

[320] N. Minar, K. H. Kramer, and P. Maes. Cooperating mobile agents for dynamic network routing. In Alex Hayzelden and John Bigham, editors, *Software Agents for Future Communication Systems*, chapter 12. Springer-Verlag, 1999.

[321] F. Mondada, G. C. Pettinaro, A. Guignard, I. W. Kwee, D. Floreano, J.-L. Deneubourg, S. Nolfi, L. M. Gambardella, and M. Dorigo. Swarm-Bot: a New Distributed Robotic Concept. *Autonomous Robots*, 17(2–3), 2004.

[322] R. Montemanni, L.M. Gambardella, A.E. Rizzoli, and A.V. Donati. A new algorithm for a dynamic vehicle routing problem based on ant colony system. In *Proceedings of ODYSSEUS 2003, Palermo, Italy, May 27–30, 2003*, 2003. (Extended abstract).

[323] R. Montemanni, D. Smith, and S. Allen. An *ANTS* algorithm for the minimum-span frequency-assignment problem with multiple interference. *IEEE Transactions on Vehicular Technology*, 51(5):949–953, September 2002.

[324] J. Montgomery and M. Randall. Anti-pheromones as a tool for better exploration of search space. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 100–110. Springer-Verlag, 2002.

[325] A. Montresor, G. Di Caro, and P. Heegarden. Architecture of the simulation environment. Internal Deliverable D11 of Shared-Cost RTD Project (IST-2001-38923) *BISON* funded by the Future & Emerging Technologies initiative of the Information Society Technologies Programme of the European Commission, 2003.

[326] J. Moy. OSPF version 2. Request For Comments (RFC) 1583, Network Working Group, 1994.

[327] J. Moy. Link-state routing. In M. E. Steenstrup, editor, *Routing in Communications Networks*, chapter 5, pages 135–157. Prentice-Hall, 1995.

[328] J. Moy. *OSPF Anatomy of an Internet Routing Protocol*. Addison-Wesley, 1998.

[329] H. Mühlenbein and G. Paass. From recombination of genes to the estimation of distributions: I. binary parameters. In *Proceedings of PPSN-IV, Second International Conference on Parallel Problem Solving from Nature*, volume 1141 of *Lecture Notes in Computer Science*, pages 178–187. Springer-Verlag, 1996.

[330] K. Narendra and M Thathachar. *Learning Automata: An Introduction*. Prentice-Hall, 1989.

[331] K. S. Narendra and M. A. Thathachar. On the behavior of a learning automaton in a changing environment with application to telephone traffic routing. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-10(5):262–269, 1980.

[332] V. Naumov. Aggregate multipath QoS routing in the Internet. In *Proceedings of the Next Generation Network Technologies International Workshop NGNT*, pages 25–30, 2001.

[333] G. Navarro Varela and M. C. Sinclair. Ant colony optimisation for virtual-wavelength-path routing and wavelength allocation. In *Proceedings of the 1999 Congress on Evolutionary Computation*, pages 1809–1816. IEEE Press, Piscataway, NJ, USA, 1999.

[334] O. V. Nedzelnitsky and K. S. Narendra. Nonstationary models of learning automata routing in data communication networks. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-17:1004–1015, 1987.

[335] S. Nelakuditi and Z.-L. Zhang. On selection of paths for multipath routing. In *Proceedings of the Interational Workshop on QoS (IWQoS)*, volume 2092 of *Lecture Notes in Computer Science*, pages 170–182, 2001.

[336] N. J. Nilsson. *Artificial Intelligence: A New Synthesis*. Morgan Kauffmann, 1998.

[337] K. Oida and A. Kataoka. Lock-free AntNet and its evaluation for adaptiveness. *Journal of IEICE B* (in Japanese), J82-B(7):1309–1319, 1999.

[338] K. Oida and M. Sekido. An agent-based routing system for QoS guarantees. In *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics*, pages 833–838, 1999.

[339] K. Oida and M. Sekido. ARS: an efficient agent-based routing system for QoS guarantees. *Computer communications*, 23:1437–1447, 2000.

[340] A. Orda, R. Rom, and N. Shimkin. Competitive routing in multi-user communication networks. *IEEE/ACM Transactions on Networking*, 1:510–521, 1993.

[341] G. Owen. *Game Theory*. Academic Press, third edition, 1995.

[342] H. Paessens. The savings algorithm for the vehicle routing problem. *European Journal of Operational Research*, 34:336–344, 1988.

[343] R. E. Page and J. Erber. Levels of behavioral organization and the evolution of division of labor. *Naturwissenschaften*, 89(3):91–106, March 2002.

[344] C.H. Papadimitriou and K. Steiglitz. *Combinatorial Optimization*. Prentice-Hall, New Jersey, 1982.

[345] A. Papoulis. *Probability, Random Variables and Stochastic Process*. McGraw-Hill, third edition, 1991.

[346] R.S. Parpinelli, H.S. Lopes, and A.A. Freitas. An ant colony algorithm for classification rule discovery. In H. Abbass, R. Sarker, and C. Newton, editors, *Data Mining: a Heuristic Approach*, pages 191–208. Idea Group Publishing, London, UK, 2002.

[347] R.S. Parpinelli, H.S. Lopes, and A.A. Freitas. Data mining with an ant colony optimization algorithm. *IEEE Transactions on Evolutionary Computation*, 6(4), August 2002.

[348] J. M. Pasteels, J.-L. Deneubourg, and S. Goss. Self-organization mechanisms in ant societies (i): Trail recruitment to newly discovered food sources. *Experientia Supplementum*, 54:155–175, 1987.

[349] C. E. Perkins and E. M. Royer. Ad-hoc on-demand distance vector routing. In *Proceedings of the Second IEEE Workshop on Mobile Computing Systems and Applications*, 1999.

[350] L. Peshkin. *Reinforcement learning by policy search*. PhD thesis, Department of Computer Science, Brown University, Providence, Rhode Island, April 2002.

[351] L. L. Peterson and B. Davie. *Computer Networks: A System Approach*. Morgan Kaufmann, 1996.

[352] M. Pinedo. *Scheduling: theory, algorithms, and systems*. Prentice–Hall, Englewood Cliffs, NJ, USA, 1995.

[353] M.L. Puterman. *Markov Decision Problems*. John Wiley & Sons, 1994.

[354] Qualnet Simulator, Version 3.6. Scalable Network Technologies, Inc., Culver City, CA, USA, 2003. `http://stargate.ornl.gov/trb/tft.html`.

[355] M. Rahoual, R. Hadji, and V. Bachelet. Parallel Ant System for the set covering problem. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 262–267. Springer-Verlag, 2002.

[356] K.-J. Räihä and E. Ukkonen. The shortest common supersequence problem over binary alphabet is NP-complete. *Theoretical Computer Science*, 16:187–198, 1981.

[357] H. Ramalhinho Lourenço, O. Martin, and T. Stüzle. Iterated local search. In F. Glover and G. Kochenberger, editors, *Handbook of Metaheuristics*, pages 321–353. Kluwer Academic Publishers, Norwell, MA, USA, 2002.

[358] H. Ramalhinho Lourenço and D. Serra. Adaptive approach heuristics for the generalized assignment problem. Technical Report EWP Series No. 304, Universitat Pompeu Fabra, Dept. of Economics and Management, Barcelona, 1998.

[359] S. Ramanathan and M. Steenstrup. A survey of routing techniques for mobile communications networks. *Mobile Networks and Applications*, 1:89–104, 1996.

[360] M. Randall and A. Lewis. A parallel implementation of ant colony optimisation. *Journal of Parallel and Distributed Computing*, 62(9):1421–1432, 2004.

[361] G. Reinelt. TSPLIB - A traveling salesman problem library. *ORSA Journal on Computing*, 3:376–384, 1991.
`http://www.iwr.uni-heidelberg.de/iwr/comopt/soft/TSPLIB95/TSPLIB.html`.

[362] G. Reinelt. *The Traveling Salesman Problem: Computational Solutions for TSP Applications*. Berlin: Springer-Verlag, 1994.

[363] M. Reinmann, K. Doerner, and R. Hartl. Insertion based ants for vehicle routing problems with backhauls and time windows. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 135–148. Springer-Verlag, 2002.

[364] Y. Rekhter. Inter-Domain Routing Protocolo (IDRP). *Internetworking: research and experience*, 4(2): 61–80, June 1993.

[365] Y. Rekhter and T. Li. A Border Gateway Protocol 4 (BGP-4). Request For Comments (RFC) 1644, Network Working Group, 1994.

[366] R. G. Reynolds. An introduction to cultural algorithms. In *Proceedings of the 3rd Annual Conference on Evolutionary Programming*, pages 131–139. World Scientific Publishing, 1994.

[367] C. P. Robert and G. Casella. *Monte Carlo Statistical Methods*. Springer-Verlag, 1999.

[368] A. Roli, C. Blum, and M. Dorigo. ACO for maximal constraint satisfaction problems. In *Proceedings of MIC'2001 – Meta–heuristics International Conference*, volume 1, pages 187–191, 2001.

[369] S. Ross. *Introduction to Stochastic Dynamic Programming*. Academic Press, New York, USA, 1983.

[370] T. Roughgarden and E. Tardos. How bad is selfish routing? In *IEEE Symposium on Foundations of Computer Science*, pages 93–102, 2000.

[371] E. M. Royer and C.-K. Toh. A review of current routing protocols for ad hoc mobile wireless networks. *IEEE Personal Communications*, 1999.

[372] R. Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 2:127–190, 1999.

[373] R. Y. Rubinstein. Combinatorial optimization, cross-entropy, ants and rare events. In S. Uryasev and P.M. Pardalos, editors, *Stochastic Optimization: Algorithms and Applications*. Kluwer Academic Publisher, 2000.

[374] R. Y. Rubistein. *Simulation and the Monte Carlo Method*. John Wiley & Sons, 1981.

[375] Günter Rudolph. Massively parallel simulated annealing and its relation to evolutionary algorithms. *Evolutionary Computation*, 1(4):361–382, 1993.

[376] Günter Rudolph. Convergence analysis of canonical genetic algorithms. *IEEE Transactions on Neural Networks*, 5(1):96–101, 1994.

[377] D. E. Rumelhart, J. L. McClelland, and the PDP Research Group, editors. *Parallel Distributed Processing*. Cambridge, MA: MIT Press, 1986.

[378] H. Sandalidis, K. Mavromoustakis, and P. Stavroulakis. Performance measures of an ant based decentralized routing scheme for circuit switching communication networks. *Soft Computing*, 5(4):313–317, 2001.

[379] H. Sandalidis, K. Mavromoustakis, and P. Stavroulakis. Ant-based probabilistic routing with pheromone and antipheromone mechanisms. *International Journal of Communication Systems (IJCS)*, 17:55–62, January 2004.

[380] L. Schoofs and B. Naudts. Ant colonies are good at solving constraint satisfaction problems. In *Proceedings of the 2000 Congress on Evolutionary Computation*, volume 2, pages 1190–1195. IEEE Press, 2000.

[381] R. Schoonderwoerd, O. Holland, and J. Bruten. Ant-like agents for load balancing in telecommunications networks. In *Proceedings of the First International Conference on Autonomous Agents*, pages 209—216. ACM Press, 1997.

[382] R. Schoonderwoerd, O. Holland, J. Bruten, and L. Rothkrantz. Ant-based load balancing in telecommunications networks. *Adaptive Behavior*, 5(2):169–207, 1996.

[383] N. Secomandi. Comparing neuro-dynamic programming algorithms for the vehicle routing problem with stochastic demands. *Computer and Operations Research*, 27:1201–1225, 2000.

[384] P. Serafini. *Ottimizzazione* (in Italian). Zanichelli, Bologna, Italy, 2000.

[385] S. Shani and T. Gonzales. P-complete approximation problems. *Journal of ACM*, 23:555–565, 1976.

[386] A. U. Shankar, C. Alaettinoğlu, K. Dussa-Zieger, and I. Matta. Performance comparison of routing protocols under dynamic and static file transfer connections. *ACM Computer Communication Review*, 22(5):39–52, 1992.

[387] A. U. Shankar, C. Alaettinoğlu, K. Dussa-Zieger, and I. Matta. Transient and steady-state performance of routing protocols: Distance-vector versus link-state. Technical Report UMIACS-TR-92-87, CS-TR-2940, Institute for Advanced Computer Studies and Department of Computer Science, University of Maryland, College Park (MD), August 1992.

[388] J. Shen, J. Shi, and J. Crowcroft. Proactive multipath routing. In *Proceedings of Quality of future Internet Services (QofIS) 2002*, 2002.

[389] E. Sigel, B. Denby, and S. Le Heárat-Mascle. Application of ant colony optimization to adaptive routing in a LEO telecommunications satellite network. *Annals of Telecommunications*, 57(5–6):520–539, May-June 2002.

[390] K.M. Sim and W.H. Sun. Ant colony optimization for routing and load-balancing: Survey and new directions. *IEEE Transactions on Systems, Man, and Cybernetics–Part A*, 33(5):560–572, September 2003.

[391] S. P. Singh, T. Jaakkola, and M. I. Jordan. Learning without state estimation in partially observable Markovian decision processes. In *Proceedings of the Eleventh Machine Learning Conference*, pages 284–292. New Brunswick, NJ: Morgan Kaufmann, 1994.

[392] S. P. Singh and R. S. Sutton. Reinforcement learning with replacing eligibility traces. *Machine Learning*, 22:123–158, 1996.

[393] M. J. Smith. The existence, uniqueness and stability of traffic equilibria. *Transportation Research*, 13B: 295–304, 1979.

[394] K. Socha, J. Knowles, and M. Sampels. A $\mathcal{MAX}$-$\mathcal{MIN}$ Ant System for the University Timetabling Problem. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 1–13. Springer-Verlag, 2002.

[395] K. Socha, M. Sampels, and M. Manfrin. Ant Algorithms for the University Course Timetabling Problem with regard to the state-of-the-art. In *Proceedings of EvoCOP 2003 – 3rd European Workshop on Evolutionary Computation in Combinatorial Optimization*, volume 2611 of *Lecture Notes in Computer Science*, pages 334–345. Springer-Verlag, 2003.

[396] C. Solnon. Solving permutation constraint satisfaction problems with artificial ants. In *Proceedings of the 14th European Conference on Artificial Intelligence (ECAI'2000)*, pages 118–122. IOS Press, 2000.

[397] C. Solnon. Ants can solve constraint satisfaction problems. *IEEE Transactions on Evolutionary Computation*, 6(4):347–357, August 2002.

[398] M. E. Steenstrup, editor. *Routing in Communications Networks*. Prentice-Hall, 1995.

[399] Ivan Stojmenović, editor. *Mobile Ad-Hoc Networks*. John Wiley and Sons, January 2002.

[400] P. Stone and M. M. Veloso. Multiagent systems: A survey from a machine learning persective. *Autonomous Robots*, 8(3), 2000.

[401] T. Stützle. An Ant Approach to the Flow Shop Problem. Technical Report AIDA–97–07, FG Intellektik, FB Informatik, TH Darmstadt, September 1997.

[402] T. Stützle. Parallelization strategies for ant colony optimization. In A. E. Eiben, T. Back, M. Schoenauer, and H.-P. Schwefel, editors, *Proceedings of PPSN-V, Fifth International Conference on Parallel Problem Solving from Nature*, pages 722–731. Springer-Verlag, 1998.

[403] T. Stützle and M. Dorigo. A short convergence proof for a class of ant colony optimization algorithms. *IEEE Transactions on Evolutionary Computation*, 6(4):358–365, 2002.

[404] T. Stützle and H. Hoos. The $\mathcal{MAX}$–$\mathcal{MIN}$ Ant System and local search for the traveling salesman problem. In T. Baeck, Z. Michalewicz, and X. Yao, editors, *Proceedings of IEEE-ICEC-EPS'97, IEEE International Conference on Evolutionary Computation and Evolutionary Programming Conference*, pages 309–314. IEEE Press, 1997.

[405] T. Stützle and H. Hoos. Improvements on the Ant System: Introducing $\mathcal{MAX}$–$\mathcal{MIN}$ ant system. In *Proceedings of the International Conference on Artificial Neural Networks and Genetic Algorithms*, pages 245–249. Springer Verlag, Wien, 1997.

[406] T. Stützle and H. Hoos. MAX–MIN Ant System for the quadratic assignment problem. Technical Report AIDA–97–4, FG Intellektik, TH Darmstadt, July 1997.

[407] T. Stützle and H. Hoos. MAX–MIN Ant system and local search for combinatorial optimization problems. In S. Voß, S. Martello, I.H. Osman, and C. Roucairol, editors, *Meta-Heuristics: Advances and Trends in Local Search Paradigms for Optimization*, pages 313–329. Boston,MA:Kluwer Academics, 1999.

[408] T. Stützle and H. Hoos. $\mathcal{MAX}$–$\mathcal{MIN}$ Ant System. *Future Generation Computer Systems*, 16(8), 2000.

[409] Thomas Stützle. An Ant Approach to the Flow Shop Problem. In *Proceedings of the 6th European Congress on Intelligent Techniques and Soft Computing (EUFIT'98)*, volume 3, pages 1560–1564, 1998.

[410] Z. Subing and L. Zemin. A QoS routing algorithm based on ant algorithm. In *Proceedings of the IEEE International Conference on Communications (ICC'01)*, pages 1587–1591, 2001.

[411] D. Subramanian, P. Druschel, and J. Chen. Ants and reinforcement learning: A case study in routing in dynamic networks. In *Proceedings of IJCAI-97, International Joint Conference on Artificial Intelligence*, pages 832–838. Morgan Kaufmann, 1997.

[412] X. Sun and G. Veciana. Dynamic multi-path routing: asymptotic approximation and simulation. In *Proceedings of Sigmetrics*, 2001.

[413] R. S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3:9–44, 1988.

[414] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 1998.

[415] R.S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224, 1990.

[416] S. Tadrus and L. Bai. A QoS network routing algorithm using multiple pheromone tables. In *Proceedings of 2003 IEEE/WIC International Conference on Web Intelligence, Halifax, Canada, October 13–16, 2003*, pages 132–138, 2003.

[417] E. Taillard. Robust taboo search for the quadratic assignment problem. *Parallel Computing*, 17:443–455, 1991.

[418] E.-G. Talbi, O. Roux, C. Fonlupt, and D. Robillard. Parallel ant colonies for the quadratic assignment problem. *Future Generation Computer Systems*, 17:441–449, 2001.

[419] A. Tanenbaum. *Computer Networks*. Prentice-Hall, 4th edition, 2002.

[420] D. L. Tennenhouse, J. M. Smith, W. D. Sincoskie, D. J. Wetherall, and G. J. Minden. A survey of active network research. *IEEE Communications Magazine*, 35(1):80–86, 1997.

[421] G. Theraulaz and E. Bonabeau. A brief history of stigmergy. *Artificial Life,* Special Issue on Stigmergy, 5:97–116, 1999.

[422] W. Thomson. *Mathematical and Physical Papers*, volume V of 6 voll., page 12. Cambridge, UK, 1882–1911.

[423] S. B. Thrun. Efficient exploration in reinforcement learning. Technical Report CMU-CS-92-102, Carnegie Mellon University, Computer Science Department, Pittsburgh, Pennsylvania, 1992.

[424] R. A. Tintin and Dong I. Lee. Intelligent and mobile agents over legacy, present and future telecommunication networks. In A. Karmouch and R. Impley, editors, *First International Workshop on Mobile Agents for Telecommunication Applications (MATA'99)*, pages 109–126. World Scientific Publishing Ltd., 1999.

[425] E.P.K. Tsang. *Foundations of constraint satisfaction*. Academic Press, London, UK, 1993.

[426] R. van der Put. Routing in the faxfactory using mobile agents. Technical Report R&D-SV-98-276, KPN Research, 1998.

[427] R. van der Put and L. Rothkrantz. Routing in packet switched networks using agents. Submitted to *Simulation Practice and Theory*, 1999.

[428] M. Van der Zee. Quality of service routing: state of the art report. Technical report, Ericsson, 1999.

[429] J. van Hemert and C. Solnon. A study into ant colony optimization, evolutionary computation and constraint programming on binary constraint satisfaction problems. In *EvoCOP 2004*, Lecture Notes in Computer Science. Springer-Verlag, April 2004.

[430] A.V. Vasilakos and G.A. Papadimitriou. A new approach to the design of reinforcement scheme for learning automata: Stochastic Estimator Learning Algorithms. *Neurocomputing*, 7(275), 1995.

[431] A.F. Veinott. On discrete dynamic programming with sensitive discount optimality criteria. *Annals of Mathematical Statistics*, 40:1635–1660, 1969.

[432] G. Vigna, editor. *Mobile Agents and Security*, volume 1419 of *Lecture Notes in Computer Science*. Springer-Verlag, Berlin Germany, 1998.

[433] C. Villamizar. OSPF Optimized Multipath (OSPF-OMP). Internet Draft, February 1999.

[434] A. Vogel, M. Fischer, H. Jaehn, and T. Teich. Real-world shop floor scheduling by ant colony optimization. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ants Algorithms - Proceedings of ANTS 2002, Third International Workshop on Ant Algorithms*, volume 2463 of *Lecture Notes in Computer Science*, pages 268–273. Springer-Verlag, 2002.

[435] S. Vutukury. *Multipath routing mechanisms for traffic engineering and quality of service in the Internet*. PhD thesis, University of California, Santa Cruz, CA, USA, March 2001.

[436] S. Vutukury and J.J. Garcia-Luna-Aceves. An algorithm for multipath computation using distance-vectors with predecessor information. In *Proceedings of the IEEE International Conference on Computer Communications and Networks (ICCCN)*, pages 534–539, Boston, MA, USA, 1999.

[437] I.A. Wagner and A.M. Bruckstein. Cooperative cleaners: A case of distributed ant-robotics. In *Communications, Computation, Control, and Signal Processing: A Tribute to Thomas Kailath*, pages 289–308. Kluwer Academic Publishers, The Netherlands, 1997.

[438] N. Wakamiya, M. Solarski, and J. Sterbenz, editors. *Active Networks - IFIP TC6 5th International Workshop, IWAN 2003, Kyoto, Japan, December 10-12, 2003, Revised Papers*, volume 2982 of *Lecture Notes in Computer Science*, 2004. Springer-Verlag.

[439] J. Walrand and P. Varaiya. *High-performance Communication Networks*. Morgan Kaufmann, 1996.

[440] Z. Wang. *Internet QoS: Architectures and Mechanisms for Quality of Service*. Morgan Kaufmann, 2001.

[441] Z. Wang and J. Crowcroft. Analysis of shortest-path routing algorithms in a dynamic network environment. *ACM Computer Communication Review*, 22(2), 1992.

[442] C. J. Watkins. *Learning with Delayed Rewards*. PhD thesis, Psychology Department, University of Cambridge, UK, 1989.

[443] H.F. Wedde, M. Farooq, and Y. Zhang. BeeHive: An efficient fault tolerant routing algorithm under high loads inspired by honey bee behavior. In M. Dorigo, M. Birattari, C. Blum, L. M. Gambardella, F. Mondada, and T. Stützle, editors, *Ants Algorithms - Proceedings of ANTS 2004, Fourth International Workshop on Ant Algorithms*, volume 3172 of *Lecture Notes in Computer Science*. Springer-Verlag, 2004.

[444] G. Weiss, editor. *Multiagent Systems - A Modern Approach to Distributed Artificial Intelligence*. The MIT Press, 1999.

[445] T. White, B. Pagurek, and F. Oppacher. ASGA: improving the ant system by integration with genetic algorithms. In *Proceedings of the Third Genetic Programming Conference*, pages 610–617, 1998.

[446] T. White, B. Pagurek, and F. Oppacher. Connection management using adaptive mobile agents. In H.R. Arabnia, editor, *Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA'98)*, pages 802–809. CSREA Press, 1998.

[447] D.H. Wolpert and K. Tumer. An introduction to collective intelligence. In J. Bradshaw, editor, *Handbook of Agent Technology*. AAAI/MIT Press, 2001.

[448] L. A. Zadeh and C. A. Desoer. *Linear System Theory*. McGraw-Hill, New York, NY, USA, 1963.

[449] J. Zhang. A distributed representation approach to group problem solving. *Journal of the American Society of Information Science*, 49(9):801–809, 1998.

[450] L. Zhang, S. Deering, and D. Estrin. RSVP: A new resource ReSerVation protocol. *IEEE Networks*, 7 (5):8–18, September 1993.

[451] N. L. Zhang, S. S. Lee, and W. Zhang. A method for speeding up value iteration in partially observable markov decision processes. In *Proceedings of the 15th Conference on Uncertainties in Artificial Intelligence*, pages 696–703, 1999.

[452] W. Zhong. When ants attack: Security issues for stigmergic systems. Master's thesis, University of Virginia, Master of Computer Science, Charlottesville, VA (USA), April 2002.

[453] J. Zinky, G. Vichniac, and A. Khanna. Performance of the revised routing metric in the ARPANET and MILNET. In *MILCOM 89*, 1989.

[454] M. Zlochin, M. Birattari, N. Meuleau, and M. Dorigo. Model-base search for combinatorial optimization. Technical Report 2001-15, IRIDIA, Université Libre de Bruxelles, Brussels (Belgium), 2001.

[455] M. Zlochin, M. Birattari, N. Meuleau, and M. Dorigo. Model-based search for combinatorial optimization: A critical survey. *Annals of Operations Research*, 131, 2004.